

HO CHI MINH CITY NATIONAL UNIVERSITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE & ENGINEERING



KIẾN TRÚC PHẦN MỀM (CO3017)

Bài tập lớn - HK 251

INTELLIGENT TUTORING SYSTEM

GVHD: Trần Trương Tuấn Phát

Sinh viên:	Phạm Gia Bảo	2210254
	Nguyễn Trương Thái Bảo	2210251
	Châu Quốc Bảo	2210195
	Võ Lê Quang Vinh	2213982
	Phan Đình Khôi Nguyên	2212318
	Lâm Vũ	2213994
	Võ Quang Đại Việt	2213954
	Nguyễn Duy Vũ	2213997

TP. HỒ CHÍ MINH, 11/2025

MỤC LỤC

1	Tổng Quan Hệ Thống	5
1.1	Phân Tích Ngữ Cảnh	5
1.2	Phạm Vi Hệ Thống	5
1.3	Các Bên Liên Quan	6
1.4	Yêu Cầu Chức Năng	6
1.5	Yêu Cầu Phi Chức Năng	7
2	Mô Hình Hóa Hệ Thống	9
2.1	Sơ Đồ Use Case	9
2.2	Mô Tả Use Case	9
3	Kiến Trúc Phần Mềm	67
3.1	Tổng Quan Kiến Trúc	67
3.2	Góc Nhìn Phân Rã	69
3.3	Góc Nhìn Sử Dụng	72
3.4	Góc Nhìn Thành Phần - Kết Nối	74
3.5	Góc Nhìn Triển Khai	77
3.6	Phân Công Nhiệm Vụ	80
3.7	Lý Do Thiết Kế	81
4	Thiết Kế Chi Tiết	83
4.1	Thiết Kế Module	83
4.2	Thiết Kế Hành Vi	100
5	Hiện Thực Hệ Thống	115
5.1	Mã Nguồn	115
5.2	Slide Thuyết Trình	115
5.3	Các Module Được Hiện Thực	115
6	Hướng Phát Triển Trong Tương Lai	116
7	Kết Luận	117
8	Tài Liệu Tham Khảo	119
9	Phụ Lục	120
9.1	Hồ Sơ Quyết Định Kiến Trúc	120

DANH SÁCH HÌNH ẢNH

2.1	Sơ đồ Use Case	9
3.1	Sơ đồ tổng quan kiến trúc	67
3.2	Sơ đồ góc nhìn phân rã	69
3.3	Sơ đồ góc nhìn sử dụng	72
3.4	Sơ đồ góc nhìn thành phần - kết nối	74
3.5	Sơ đồ triển khai	77
4.1	Sơ đồ lớp module Learning Manager	83
4.2	Sơ đồ lớp module Teaching Assistant	86
4.3	Sơ đồ lớp module Assessment Tracker	90
4.4	Sơ đồ lớp module Assessment Manager	94
4.5	Sơ đồ lớp module Learning Analyzer	97
4.6	Sơ đồ hoạt động use case View Recommended Courses	100
4.7	Sơ đồ hoạt động use case Enroll a Course	102
4.8	Sơ đồ hoạt động use case Take a Quiz/Exam	103
4.9	Sơ đồ hoạt động use case Submit an Assignment	104
4.10	Sơ đồ hoạt động use case View Learning Analytics Report	106
4.11	Sơ đồ hoạt động use case Create a Course	107
4.12	Sơ đồ hoạt động use case Add a Quiz	108
4.13	Sơ đồ hoạt động use case Add an Exam	110
4.14	Sơ đồ hoạt động use case Add an Assignment	111
4.15	Sơ đồ hoạt động use case Grade an Assignment	113

DANH SÁCH BẢNG

1.1	Bảng các bên liên quan	6
2.1	Bảng mô tả use case View Recommended Courses	10
2.2	Bảng mô tả use case Enroll a Course	12
2.3	Bảng mô tả use case Take a Quiz/Exam	14
2.4	Bảng mô tả use case Submit an Assignment	16
2.5	Bảng mô tả use case View Learning Materials	17
2.6	Bảng mô tả use case View an Assignment Result	18
2.7	Bảng mô tả use case View an Exam Result	19
2.8	Bảng mô tả use case View Course Progress	20
2.9	Bảng mô tả use case View Learning Analytics Report	21
2.10	Bảng mô tả use case View Course Feedback	22
2.11	Bảng mô tả use case Manage Personal Information	24
2.12	Bảng mô tả use case Sign In as Student	26
2.13	Bảng mô tả use case Sign Up as Student	27
2.14	Bảng mô tả use case Create a Course	29
2.15	Bảng mô tả use case Update a Course	31
2.16	Bảng mô tả use case Add a Quiz	34
2.17	Bảng mô tả use case Update a Quiz	36
2.18	Bảng mô tả use case Delete a Quiz	37
2.19	Bảng mô tả use case View a Quiz Analytics Report	39
2.20	Bảng mô tả use case Add an Exam	41
2.21	Bảng mô tả use case Update an Exam	44
2.22	Bảng mô tả use case Add an Assignment	46
2.23	Bảng mô tả use case View an Exam Analytics Report	48
2.24	Bảng mô tả use case View an Update an Assignment	51
2.25	Bảng mô tả use case Delete an Assignment	53
2.26	Bảng mô tả use case Manage Learning Materials	56
2.27	Bảng mô tả use case Provide Student Learning Feedback	58
2.28	Bảng mô tả use case View Student Analytics Report	60
2.29	Bảng mô tả use case Grade an Assignment	62
2.30	Bảng mô tả use case Delete an Exam	64
2.31	Bảng mô tả use case Sign In as Tutor	66
3.1	Bảng phân công nhiệm vụ	80

1 Tổng Quan Hệ Thống

1.1 Phân Tích Ngữ Cảnh

Trong xu thế chuyển đổi số trong giáo dục, các mô hình học tập truyền thống “one-size-fits-all” đang dần được thay thế bởi các hệ thống học tập **cá nhân hóa** và **thích ứng**. Các hệ thống quản lý học tập (LMS) hiện nay thường chỉ đóng vai trò như kho lưu trữ nội dung và công cụ quản trị, thiếu khả năng điều chỉnh nội dung theo đặc điểm, tiến độ và năng lực riêng của từng người học.

IntelliLearn được phát triển như một **Hệ thống Gia sư Thông minh (Intelligent Tutoring System - ITS)**, ứng dụng các kỹ thuật trí tuệ nhân tạo (AI) nhằm cung cấp lộ trình học tập được tùy chỉnh cho từng cá nhân. Hệ thống có khả năng đánh giá hiệu suất học tập, đưa ra gợi ý, phản hồi và điều chỉnh chiến lược giảng dạy dựa trên hành vi cũng như tiến trình của người học. Mục tiêu của IntelliLearn là mô phỏng một phần vai trò của gia sư con người — người có thể tương tác, hướng dẫn và thích ứng với học viên theo thời gian thực.

1.2 Phạm Vi Hệ Thống

Hệ thống được thiết kế để hỗ trợ nhiều môn học, chủ đề và cấp độ khó khác nhau, đồng thời cung cấp các công cụ quản lý cho giảng viên và nhà quản trị. Cụ thể, IntelliLearn bao gồm các chức năng chính sau:

- **Cá nhân hóa học tập:** Đánh giá điểm mạnh, điểm yếu và phong cách học tập của từng học viên để gợi ý nội dung và lộ trình phù hợp.
- **Phản hồi và hướng dẫn:** Cung cấp phản hồi về kết quả học tập, kèm theo gợi ý hoặc trợ giúp khi cần thiết.
- **Đánh giá và theo dõi tiến độ:** Bao gồm các bài kiểm tra, bài tập hoặc dự án giúp đánh giá mức độ hiểu biết và tiến triển của học viên.
- **Bảng điều khiển giảng viên:** Cho phép giáo viên theo dõi tiến độ học tập, quản lý nội dung giảng dạy và tạo báo cáo phân tích.
- **Quản lý nội dung học tập:** Hỗ trợ quản lý, cập nhật và biên soạn tài liệu học theo nhiều chủ đề và định dạng khác nhau (văn bản, video, bài tập tương tác, v.v.).



1.3 Các Bên Liên Quan

Bên liên quan	Vai trò	Mối quan tâm chính
Người học	Người dùng chính	Giao diện dễ dùng, đề xuất học tập cá nhân hóa, theo dõi tiến độ rõ ràng
Giảng viên	Tạo và quản lý nội dung	Công cụ chấm điểm, phân tích tiến độ, quản lý khóa học
Quản trị viên	Vận hành hệ thống	Quản lý tài khoản, đảm bảo hệ thống hoạt động ổn định

Table 1.1: Bảng các bên liên quan

1.4 Yêu Cầu Chức Năng

1.4.1 Đối với Học viên

- Quản lý thông tin cá nhân: điểm số, năng lực, sở thích.
- Nhận phản hồi từ giảng viên.
- Theo dõi tiến độ học tập qua biểu đồ, chỉ số.
- Tạo báo cáo kết quả theo chu kỳ: ngày / tuần / tháng.

1.4.2 Đối với Giảng viên

- Quản lý nội dung: thêm, sửa, xóa khóa học.
- Chấm điểm và phản hồi thủ công.
- Theo dõi tiến độ học viên (cá nhân và tập thể).
- Tạo báo cáo phân tích hỗ trợ giảng dạy.
- Đánh giá học viên tự động qua AI.
- Phân phối nội dung thông minh dựa trên hành vi, năng lực.

1.4.3 Đối với Quản trị viên

- Tạo/xóa tài khoản người dùng.
- Giám sát hoạt động hệ thống.

1.5 Yêu Cầu Phi Chức Năng

- **Độ Tin Cậy (Reliability):** Hệ thống phải đảm bảo khả năng vận hành liên tục, chính xác và ổn định trong quá trình cung cấp dịch vụ học tập. Độ tin cậy cao giúp giảm thiểu gián đoạn và bảo đảm trải nghiệm học tập không bị ảnh hưởng bởi lỗi hệ thống.
 - Áp dụng cơ chế tự động khôi phục khi xảy ra sự cố như lỗi mạng, mất kết nối cơ sở dữ liệu hoặc ngừng dịch vụ tạm thời.
 - Hệ thống phải duy trì tỷ lệ sẵn sàng tối thiểu 95% trong khung giờ hoạt động chính của học viên và giảng viên (từ 6:00 đến 23:00 mỗi ngày).
- **Tính khả dụng (Usability):** Hệ thống cần mang lại trải nghiệm người dùng thân thiện và đáp ứng tốt trên nhiều thiết bị. Tính khả dụng cao giúp giảm độ phức tạp trong thao tác, rút ngắn thời gian làm quen và nâng cao hiệu quả tương tác giữa người học và hệ thống.
 - Giao diện đáp ứng cho cả máy tính và điện thoại.
 - Người dùng có thể sử dụng các tính năng cơ bản sau 10 phút hướng dẫn.
- **Hiệu suất (Performance):** Hệ thống phải đảm bảo khả năng xử lý nhanh và ổn định để duy trì trải nghiệm học tập liên tục. Hiệu suất cao giúp duy trì tính liên tục của trải nghiệm người dùng, đặc biệt trong các tác vụ tính toán hoặc phân tích do mô-đun AI thực hiện.
 - Thời gian tải trang, truy xuất nội dung hoặc tạo báo cáo không vượt quá 5 giây.
 - Hỗ trợ 10.000 người dùng đồng thời.
 - Các tác vụ AI như gợi ý lộ trình học hoặc phân tích năng lực phải hoàn tất trong vòng 15 giây.
- **Bảo mật (Security):** Hệ thống lưu trữ dữ liệu nhạy cảm như thông tin cá nhân, điểm số và lịch sử học tập của người dùng. Do đó, bảo mật là yêu cầu bắt buộc để bảo vệ người dùng khỏi việc rò rỉ hoặc khai thác dữ liệu trái phép.



- Dữ liệu người dùng (bao gồm điểm, tiến độ, hồ sơ học tập) phải được mã hóa trong quá trình truyền tải (end-to-end encryption).
- Các API nhạy cảm hoặc liên quan đến dữ liệu người dùng phải yêu cầu ủy quyền truy cập thông qua cơ chế OAuth 2.0 để đảm bảo chỉ các thực thể được cấp quyền mới có thể gọi dịch vụ.
- **Khả năng mở rộng (Scalability):** Hệ thống cần có khả năng mở rộng linh hoạt để đáp ứng sự gia tăng về số lượng người dùng, dữ liệu và hoạt động học tập trong tương lai. Việc mở rộng không được làm giảm hiệu suất hay gây gián đoạn dịch vụ, đảm bảo hệ thống vẫn hoạt động ổn định ngay cả khi quy mô sử dụng tăng cao.
 - Hệ thống có thể mở rộng để phục vụ số lượng người dùng tăng mà không ảnh hưởng đáng kể đến thời gian phản hồi.
 - Có thể bổ sung tài nguyên tính toán, dung lượng lưu trữ hoặc chức năng mới khi cần mà không yêu cầu thay đổi lớn trong hệ thống hiện có.

2 Mô Hình Hóa Hệ Thống

2.1 Sơ Đồ Use Case

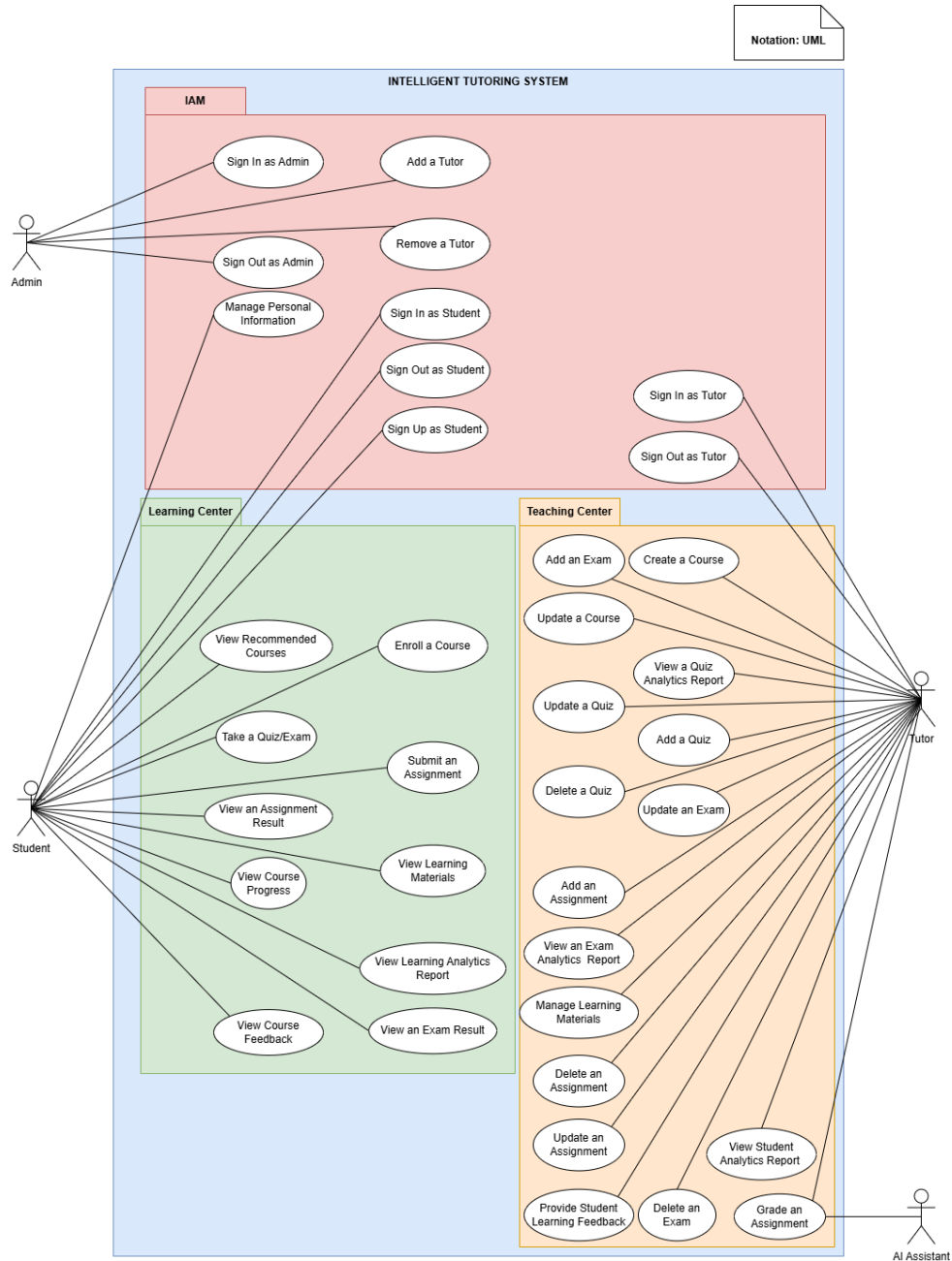


Figure 2.1: Sơ đồ Use Case

2.2 Mô Tả Use Case



Use Case ID	UC-STUDENT-01
Use Case Name	View Recommended Courses
Primary Actors	Học viên
Description	Học viên xem danh sách khóa học được hệ thống AI đề xuất dựa trên năng lực và tiến độ học.
Trigger	Học viên truy cập vào dashboard hoặc mục dành riêng cho gợi ý khóa học
Preconditions	• Học viên đã đăng nhập • Hệ thống đã thu thập đủ thông tin về học viên để có thể đưa ra quyết định
Postconditions	Học viên nhận được danh sách khóa học phù hợp với tình hình học tập hiện tại
Normal Flow	1. Học viên truy cập vào dashboard dành cho học viên 2. Hệ thống sử dụng mô hình AI để phân tích dữ liệu của học viên (gồm các thông tin đã thu thập trước đó như các khóa đã học, các khóa học đã tìm kiếm gần đây, ...) 3. Hệ thống hiển thị một danh sách các nội dung được đề xuất kèm theo lý do
Alternative Flows	A1. Đối với học viên mới 3a. Nếu chưa có đủ dữ liệu, hệ thống có thể hiển thị các khóa học phổ biến nhất hoặc các khóa nhập môn

Table 2.1: Bảng mô tả use case View Recommended Courses



Use Case ID	UC-STUDENT-02
Use Case Name	Enroll a Course
Primary Actors	Học viên
Description	Học viên đăng ký tham gia một khóa học cụ thể.
Trigger	Học viên bấm vào nút đăng kí tại trang chi tiết của một khóa học
Preconditions	• Học viên đã đăng nhập vào hệ thống • Khóa học người học muốn đăng kí tồn tại • Học viên chưa đăng kí khóa học này
Postconditions	• Hệ thống ghi nhận học viên là thành viên của khóa học • Khóa học xuất hiện trong danh sách “Các khóa học đã tham gia” của học viên
Normal Flow	1. Học viên bấm nút đăng kí cho một khóa học cụ thể 2. Hệ thống hiển thị xác nhận “Bạn có muốn tham gia khóa học này?” 3. Học viên bấm “Xác nhận” 4. Khóa học kiểm tra các tiền điều kiện (Học viên chưa tham gia, khóa học còn tồn tại, Sĩ số lớp chưa đủ) 5. Hệ thống xác nhận Học viên đủ điều kiện ghi danh 6. Hệ thống lưu thông tin ghi danh, liên kết tài khoản Học viên với Khóa học 7. Hệ thống hiển thị thông báo “Đăng kí thành công” 8. Hệ thống điều hướng học viên đến trang thông tin của Khóa học vừa mới đăng kí



Alternative Flows	A1. Học viên hủy bỏ khi đăng kí 3a. Học viên bấm “Hủy bỏ” 4a. Hệ thống đóng hộp thoại xác nhận. Use case kết thúc
Exceptions	E1. Người dùng đã đăng kí từ trước 5b. Hệ thống phát hiện học viên đã là thành viên của khóa học này 6b. Hệ thống hiển thị thông báo lỗi: “Bạn đã tham gia khóa học này” E2. Khóa học không còn tồn tại hoặc đã bị đóng 5c. Hệ thống phát hiện Giảng viên đã xóa hoặc đóng khóa học 6c. Hệ thống hiển thị thông báo lỗi: “Khóa học không còn tồn tại hoặc đã đóng đăng kí” E3. Lỗi hệ thống khi lưu 7c. Hệ thống gặp sự cố khi cố gắng ghi bản ghi đăng kí 8c. Hệ thống hiển thị thông báo lỗi chung: “Đã có lỗi xảy ra. Vui lòng thử lại sau.”

Table 2.2: Bảng mô tả use case Enroll a Course



Use Case ID	UC-STUDENT-04
Use Case Name	Take a Quiz/Exam
Primary Actors	Học viên
Description	Học viên trả lời các câu hỏi trong một bài quiz hoặc bài thi để đánh giá kiến thức
Trigger	Học viên nhấn nút “Bắt đầu làm bài” trên trang giới thiệu về bài thi/quiz
Preconditions	1. Học viên đã đăng nhập và ghi danh vào khóa học 2. Bài thi/quiz đang trong thời gian cho phép làm bài 3. Học viên chưa hết số lần làm bài cho phép
Postconditions	1. Bài làm của học viên được hệ thống ghi nhận và chấm điểm 2. Điểm số được lưu vào hồ sơ tiến độ của học viên
Normal Flow	1. Học viên chọn bài thi/quiz muốn làm 2. Hệ thống hiển thị các quy định (thời gian, số lần làm bài) và yêu cầu xác nhận bắt đầu 3. Học viên xác nhận, hệ thống bắt đầu tính giờ và hiển thị các câu hỏi 4. Học viên chọn/điền đáp án cho các câu hỏi 5. Học viên nhấn nút “Nộp bài” 6. Hệ thống xác nhận bài nộp, tính điểm và hiển thị kết quả
Alternative Flows	A1. Hết thời gian làm bài 6a. Nếu hết giờ, hệ thống sẽ tự động nộp bài với các câu trả lời mà học viên đã chọn



Exceptions	1. Mất kết nối mạng: Trình duyệt mất kết nối mạng với máy chủ. • Hệ thống (phía client) nên tự động lưu các câu trả lời vào bộ nhớ cục bộ (local storage). • Khi có kết nối lại, hệ thống sẽ cố gắng đồng bộ 2. Lỗi máy chủ hệ thống giữa chừng: Máy chủ gặp sự cố khi học viên đang làm bài. • Hệ thống phải có cơ chế lưu lại trạng thái bài làm (câu trả lời, thời gian còn lại) một cách định kỳ. • Khi hệ thống hoạt động trở lại, học viên có thể tiếp tục từ điểm đã lưu 3. Lỗi dịch vụ chấm điểm: Học viên nộp bài thành công nhưng dịch vụ chấm điểm tự động bị lỗi. • Hệ thống cần ghi nhận bài nộp, đánh dấu trạng thái là “Chờ chấm lại” và thông báo cho quản trị viên
-------------------	---

Table 2.3: Bảng mô tả use case Take a Quiz/Exam



Use Case ID	UC-STUDENT-05
Use Case Name	Submit an Assignment
Primary Actors	Học Viên
Description	Học viên nộp bài làm của mình cho một bài tập hoặc dự án cụ thể trong một khóa học để Giảng viên chấm điểm.
Trigger	Học viên nhấn vào nút “Nộp bài” trên trang chi tiết của một bài tập
Preconditions	1. Học viên đã đăng nhập vào hệ thống 2. Học viên đã ghi danh vào khóa học có chứa bài tập này 3. Thời hạn nộp bài chưa kết thúc
Postconditions	1. Bài nộp của học viên được lưu trữ thành công trên hệ thống 2. Trạng thái bài tập của học viên được cập nhật thành “Đã nộp”
Normal Flow	1. Học viên truy cập vào trang chi tiết của khóa học 2. Học viên chọn bài tập/dự án cần nộp 3. Hệ thống hiển thị giao diện cho phép tải lên tệp tin hoặc nhập nội dung 4. Học viên tải lên tệp bài làm và/hoặc điền thông tin cần thiết 5. Học viên nhấn nút “Xác nhận nộp bài” 6. Hệ thống kiểm tra tệp tin (định dạng, dung lượng) và lưu lại bài nộp 7. Hệ thống hiển thị thông báo “Nộp bài thành công” kèm theo thời gian nộp

Alternative Flows	A1. Nộp bài sau hạn chót: 7a. Nếu thời hạn nộp bài đã qua, hệ thống sẽ hiển thị cảnh báo và có thể chặn việc nộp bài hoặc đánh dấu là “Nộp muộn” A2. Tập tin không hợp lệ: 7b. Nếu tập tin không đúng định dạng hoặc vượt quá dung lượng cho phép, hệ thống sẽ báo lỗi và yêu cầu học viên tải lên tập tin khác
Exceptions	1. Mất kết nối mạng trong lúc tải lên: Hệ thống không nhận được tập tin hoàn chỉnh. • Hệ thống nên hiển thị thông báo lỗi kết nối và yêu cầu học viên thử lại. • Bài nộp không được ghi nhận 2. Lỗi máy chủ lưu trữ: Hệ thống không thể lưu tập tin do lỗi máy chủ (ví dụ: hết dung lượng, dịch vụ down). • Hệ thống phải thông báo cho học viên rằng việc nộp bài đã thất bại do lỗi kỹ thuật và khuyên họ thử lại sau. • Trạng thái bài tập không được thay đổi 3. Lỗi cơ sở dữ liệu (Database Error): Tập tin đã được tải lên thành công nhưng hệ thống không thể ghi nhận thông tin bài nộp vào CSDL. • Hệ thống cần ghi nhận lỗi này vào log, thông báo cho học viên rằng có lỗi xảy ra và liên hệ bộ phận hỗ trợ

Table 2.4: Bảng mô tả use case Submit an Assignment



Use Case ID	UC-STUDENT-03
Use Case Name	View Learning Materials
Primary Actors	Học viên
Description	Học viên truy cập và xem nội dung học tập của khóa học (tài liệu, video, bài giảng, v.v.).
Trigger	Học viên truy cập vào trang chi tiết của khóa học, hệ thống hiển thị tài liệu học tập của khóa học ở mục "Tài liệu khóa học"
Preconditions	1. Học viên đã đăng nhập 2. Học viên đã ghi danh vào khóa học chứa dữ liệu này
Postconditions	Hệ thống ghi nhận lại hoạt động của học viên (ví dụ: đánh dấu bài học là “đã hoàn thành”, ghi lại thời gian học) để phục vụ cho việc theo dõi tiến độ
Normal Flow	1. Học viên điều hướng đến trang nội dung của khóa học 2. Học viên chọn một bài học từ danh sách 3. Hệ thống hiển thị nội dung của bài học (video, văn bản, hình ảnh...) 4. Học viên tương tác với nội dung (xem, đọc...) 5. Sau khi hoàn tất, học viên nhấn nút “Đánh dấu đã hoàn thành” hoặc hệ thống có thể tự động đánh dấu dựa trên các tiêu chí: thời gian học viên ở trong trang tài liệu, thời lượng video còn lại,.... 6. Hệ thống cập nhật trạng thái của bài học này cho học viên
Alternative Flows	A1. Dữ liệu bị khóa: 3a. Nếu bài học yêu cầu hoàn thành các nội dung/bài học trước đó, hệ thống sẽ không cho phép truy cập và hiển thị thông báo yêu cầu

Table 2.5: Bảng mô tả use case View Learning Materials



Use Case ID	UC-STUDENT-06
Use Case Name	View an Assignment Result
Primary Actors	Học viên
Secondary Actors	Giảng viên
Description	Học viên xem điểm số và các nhận xét chi tiết cho bài tập đã được giảng viên chấm điểm.
Trigger	Học viên nhấn vào một bài tập từ danh sách bài tập hoặc bảng điểm.
Preconditions	1. Học viên đã đăng nhập 2. Học viên đã nộp bài tập tương ứng 3. Giảng viên đã chấm điểm và công bố kết quả.
Postconditions	Học viên đã xem được điểm và nhận xét.
Normal Flow	1. Học viên truy cập vào mục “Điểm số” hoặc trang chi tiết của bài tập đã nộp 2. Hệ thống hiển thị điểm số cuối cùng 3. Hệ thống hiển thị các nhận xét, góp ý chi tiết do giảng viên nhập vào 4. Hệ thống cũng có thể hiển thị các phân tích hoặc gợi ý tự động từ AI
Alternative Flows	A1. Bài chưa được chấm: 3a. Nếu Giảng viên chưa chấm bài, hệ thống sẽ hiển thị trạng thái “Chưa có điểm” hoặc “Đang chờ chấm”

Table 2.6: Bảng mô tả use case View an Assignment Result



Use Case ID	UC-STUDENT-07
Use Case Name	View an Exam Result
Primary Actors	Học viên
Secondary Actors	Giảng viên
Description	Học viên xem kết quả và phân tích bài thi.
Trigger	Học viên nhấn vào một bài thi từ danh sách bài thi hoặc bảng điểm.
Preconditions	1. Học viên đã đăng nhập 2. Học viên đã nộp bài tập tương ứng 3. Giảng viên đã chấm điểm và công bố kết quả.
Postconditions	Học viên đã xem được điểm và nhận xét.
Normal Flow	1. Học viên truy cập vào mục “Điểm số” hoặc trang chi tiết của bài thi đã nộp 2. Hệ thống hiển thị điểm số cuối cùng 3. Hệ thống hiển thị các nhận xét, góp ý chi tiết do giảng viên nhập vào 4. Hệ thống cũng có thể hiển thị các phân tích hoặc gợi ý tự động từ AI
Alternative Flows	A1. Bài chưa được chấm: 3a. Nếu Giảng viên chưa chấm bài, hệ thống sẽ hiển thị trạng thái “Chưa có điểm” hoặc “Đang chờ chấm”

Table 2.7: Bảng mô tả use case View an Exam Result



Use Case ID	UC-STUDENT-08
Use Case Name	View Course Progress
Primary Actors	Học viên
Description	Học viên theo dõi kết quả và tiến độ học tập của bản thân trong từng khóa học.
Trigger	Học viên chọn mục “Tiến độ của tôi” trong một khóa học hoặc truy cập vào trang dashboard cá nhân.
Preconditions	Học viên đã đăng nhập và có ít nhất một khóa học đang hoặc đã tham gia.
Postconditions	Học viên nắm được tình hình học tập của mình.
Normal Flow	1. Học viên truy cập vào mục “Tiến độ của tôi” trong một khóa học hoặc trang dashboard 2. Hệ thống tổng hợp dữ liệu từ các hoạt động của học viên trong khóa học cụ thể (bài đã học, điểm quiz, dự án đã nộp) 3. Hệ thống trình bày dữ liệu dưới dạng các biểu đồ (tỷ lệ hoàn thành khóa học), bảng điểm, các chỉ số quan trọng 4. Học viên có thể lọc xem tiến độ theo từng khóa học hoặc theo thời gian
Alternative Flows	A1. Chưa có hoạt động: 3a. Nếu học viên chưa có hoạt động nào trong khóa học, hệ thống sẽ hiển thị thông báo khuyến khích bắt đầu học. A2. Chưa có khóa học: 3b. Nếu học viên chưa tham gia khóa học nào, hệ thống sẽ hiển thị thông báo khuyến khích bắt đầu học, hiển thị gợi ý các khóa học để tham gia.

Table 2.8: Bảng mô tả use case View Course Progress



Use Case ID	UC-STUDENT-09
Use Case Name	View Learning Analytics Report
Primary Actors	Học viên
Description	Học viên theo dõi quá trình và kết quả học tập tổng thể của mình thông qua các số liệu thống kê, biểu đồ và báo cáo trực quan
Trigger	Học viên chọn mục “Báo cáo học tập” tại trang dashboard cá nhân.
Preconditions	Học viên đã đăng nhập và có ít nhất một khóa học đang hoặc đã tham gia.
Postconditions	Học viên nắm được tình hình học tập tổng thể của mình.
Normal Flow	1. Học viên truy cập mục “Báo cáo học tập” trên trang chủ (dashboard). 2. Hệ thống phân tích dữ liệu của học viên (điểm mạnh, điểm yếu, sở thích). 3. Hệ thống hiển thị một danh sách các nội dung được đề xuất kèm theo lý do (ví dụ: “Để cải thiện kỹ năng X”, “Dựa trên khóa học Y bạn đã thích”)
Alternative Flows	A1. Chưa có hoạt động: 3a. Nếu học viên chưa có khóa học hoặc chưa có hoạt động nào trong khóa học, hệ thống sẽ hiển thị thông báo khuyến khích bắt đầu học và các khóa học nhập môn.

Table 2.9: Bảng mô tả use case View Learning Analytics Report



Use Case ID	UC-STUDENT-09
Use Case Name	View Course Feedback
Primary Actors	Học viên
Description	Học viên xem phản hồi từ giảng viên về quá trình học tập trong một khóa học.
Trigger	Học viên truy cập mục "Đánh giá" trong một khóa học cụ thể
Preconditions	1. Học viên đã đăng nhập vào hệ thống 2. Học viên đã đăng ký ít nhất một khóa học
Postconditions	Học viên xem được đánh giá của giảng viên về tiến trình học tập của mình
Normal Flow	1. Học viên chọn một khóa học cụ thể từ dashboard 2. Học viên bấm vào mục "Đánh giá" 3. Hệ thống hiển thị các đánh giá của giảng viên theo bài quiz, bài thi, bài tập lớn, ..., của khóa học.

Table 2.10: Bảng mô tả use case View Course Feedback



Use Case ID	UC-STUDENT-10
Use Case Name	Manage Personal Information
Primary Actors	Học viên
Description	Học viên cập nhật các thông tin cá nhân, sở thích, mục tiêu học tập và các thông tin khác liên quan đến tài khoản của mình.
Trigger	Học viên truy cập vào dashboard và bấm vào nút "Tài khoản" trên thanh điều hướng
Preconditions	Học viên đã đăng nhập vào hệ thống
Postconditions	1. Học viên kiểm tra được thông tin tài khoản của mình 2. Học viên có thể chỉnh sửa các thông tin chưa phù hợp/không đúng
Normal Flow	1. Học viên bấm vào nút "Tài khoản" trên thanh điều hướng của hệ thống 2. Hệ thống hiển thị thông tin tài khoản cũng như các cấu hình tài khoản 3. Học viên kiểm tra/cập nhật thông tin/hành động cần thiết 4. Học viên bấm vào nút "Chỉnh sửa" để bắt đầu chỉnh sửa thông tin 5. Sau khi chỉnh sửa hoàn tất, học viên bấm vào nút "Hoàn tất" để kết thúc chỉnh sửa 6. Hệ thống hiển thị thông báo rằng học viên có chắc chắn muốn thay đổi thông tin không 7. Học viên bấm vào nút "Xác nhận" 8. Hệ thống lưu lại thay đổi 9. Hệ thống thông báo thành công.



Alternative Flows	A1. Học viên hủy bỏ việc cập nhật thông tin 4a. Học viên bấm vào nút "Hủy bỏ" 4b. Hệ thống tắt thông báo xác nhận.
Exceptions	E1. Hệ thống gặp lỗi khi lưu dữ liệu: Hệ thống thông báo lỗi: "Lỗi khi cập nhật thông tin tài khoản. Vui lòng thử lại sau"

Table 2.11: Bảng mô tả use case Manage Personal Information



Use Case ID	UC-STUDENT-11
Use Case Name	Sign In as Student
Primary Actors	Học viên
Description	Học viên đăng nhập vào hệ thống để học tập và làm bài.
Trigger	Học viên truy cập trang đăng nhập của hệ thống và chọn “Đăng nhập bằng tài khoản sinh viên”
Preconditions	• Học viên có một tài khoản hợp lệ. • Tài khoản của học viên đã được cấp phép và liên kết với vai trò “Student” trong cơ sở dữ liệu của hệ thống.
Postconditions	• Hệ thống xác thực danh tính học viên thành công. • Hệ thống cấp quyền truy cập cho học viên dựa trên vai trò “Student” đã được định nghĩa. • Hệ thống chuyển hướng học viên đến bảng điều khiển cá nhân của họ.
Normal Flow	1. Học viên truy cập trang đăng nhập của hệ thống. 2. Hệ thống hiển thị các tùy chọn đăng nhập. 3. Học viên nhấn vào nút “Đăng nhập bằng tài khoản sinh viên” 4. Học viên nhập thông tin đăng nhập (username, password) của họ. 5. Hệ thống xác thực thông tin đăng nhập. 6. Hệ thống tìm thấy tài khoản, thiết lập phiên làm việc và áp dụng các quyền của vai trò “Student”. 7. Hệ thống chuyển hướng học viên đến bảng điều khiển.



Exceptions	E1: Xác thực tài khoản thất bại • Hệ thống thông báo lỗi đến học viên: “Tài khoản hoặc mật khẩu không chính xác. Vui lòng thử lại sau.”
-------------------	--

Table 2.12: Bảng mô tả use case Sign In as Student



Use Case ID	UC-STUDENT-12
Use Case Name	Sign Up as Student
Primary Actors	Học viên
Description	Học viên mới đăng ký tài khoản để bắt đầu sử dụng hệ thống.
Trigger	Học viên truy cập trang đăng ký của hệ thống và chọn “Đăng ký tài khoản học viên mới”
Preconditions	• Tài khoản học viên đăng ký chưa có trong hệ thống
Postconditions	• Học viên đăng ký tài khoản thành công. • Các quyền của “Học viên” được hệ thống gán vào tài khoản vừa đăng ký
Normal Flow	1. Học viên truy cập trang đăng ký của hệ thống. 2. Hệ thống hiển thị các tùy chọn đăng ký. 3. Học viên nhấn vào nút “Đăng ký tài khoản học viên mới” 4. Học viên nhập thông tin cơ bản của tài khoản (username, password, mã số sinh viên,...). 5. Hệ thống xác thực thông tin đăng ký. 6. Hệ thống tạo thấy tài khoản mới thành công. 7. Hệ thống thông báo tạo tài khoản thành công 8. Hệ thống chuyển hướng học viên đến trang đăng nhập.
Exceptions	E1: Tạo tài khoản mới thất bại • Hệ thống thông báo lỗi đến học viên: “Lỗi khi đăng ký tài khoản. Vui lòng thử lại sau.”

Table 2.13: Bảng mô tả use case Sign Up as Student



Use Case ID	UC-TEACHER-01
Use Case Name	Create a Course
Primary Actors	Giảng viên
Description	Giảng viên tạo mới một khóa học bao gồm nội dung và thông tin cơ bản.
Trigger	Giảng viên bấm vào nút "Tạo khóa học mới" trên dashboard
Preconditions	1. Người dùng đã đăng nhập vào hệ thống 2. Tài khoản của người dùng được xác nhận là giảng viên
Postconditions	Một khóa học mới được thêm vào hệ thống
Normal Flow	1. Giảng viên bấm vào nút "Tạo khóa học" trên dashboard 2. Hệ thống hiển thị biểu mẫu gồm các thông tin cơ bản của khóa học 3. Giảng viên điền đầy đủ các thông tin của khóa học vào biểu mẫu 4. Giảng viên bấm vào nút "Tạo khóa học" để hoàn tất tạo khóa học 5. Hệ thống hiển thị thông báo xác nhận tạo khóa học 6. Giảng viên bấm vào nút "Xác nhận" để chắc chắn họ muốn tạo khóa học mới 7. Khóa học mới được hệ thống tạo thành công 8. Hệ thống thông báo tạo khóa học thành công
Alternative Flows	A1. Giảng viên không muốn tạo khóa học nữa 6a. Giảng viên bấm vào nút "Hủy" 6b. Hệ thống đóng biểu mẫu tạo khóa học
Exceptions	E1. Lỗi hệ thống khi tạo khóa học: Hệ thống thông báo lỗi: "Lỗi khi tạo khóa học. Vui lòng thử lại sau!"



Table 2.14: Bảng mô tả use case Create a Course



Use Case ID	UC-TEACHER-02
Use Case Name	Update a Course
Primary Actors	Giảng viên
Description	Chỉnh sửa nội dung hoặc thông tin của khóa học hiện có.
Trigger	Giảng viên bấm vào nút "Chỉnh sửa" trong trang chi tiết của một khóa học
Preconditions	1. Giảng viên đã đăng nhập vào hệ thống 2. Giảng viên có ít nhất một khóa học đã được tạo thành công
Postconditions	Thông tin cơ bản của khóa học được cập nhật thành công trên hệ thống
Normal Flow	1. Giảng viên bấm vào nút "Chỉnh sửa" trên trang chi tiết của khóa học 2. Hệ thống hiển thị một biểu mẫu gồm thông tin cơ bản của khóa học đó 3. Giảng viên tiến hành chỉnh sửa các thông tin cần thiết 4. Giảng viên bấm vào nút "Xác nhận" để hoàn tất chỉnh sửa 5. Hệ thống hiển thị thông báo xác nhận chỉnh sửa 6. Giảng viên bấm vào nút "Xác nhận" 7. Hệ thống cập nhật thông tin khóa học 8. Hệ thống thông báo cập nhật thành công
Alternative Flows	A1. Giảng viên không muốn thay đổi thông tin nữa 4a. Giảng viên bấm vào nút "Hủy bỏ" 4b. Hệ thống đóng thông báo và biểu mẫu chỉnh sửa thông tin khóa học
Exceptions	E1. Lỗi khi cập nhật thông tin: Hệ thống thông báo lỗi: "Lỗi khi cập nhật thông tin khóa học. Vui lòng thử lại sau!"



Table 2.15: Bảng mô tả use case Update a Course



Use Case ID	UC-TEACHER-03
Use Case Name	Add a Quiz
Primary Actors	Giảng viên
Description	Giảng viên thêm bài kiểm tra ngắn (quiz) vào khóa học.
Trigger	Giảng viên chọn chức năng “Tạo Quiz mới” trong giao diện quản lý Khóa học
Preconditions	1. Giảng viên đã đăng nhập vào hệ thống ITS 2. Tài khoản của giảng viên có quyền quản lý nội dung.
Postconditions	• Một Quiz mới được tạo và lưu trữ trong cơ sở dữ liệu. • Quiz này xuất hiện trong danh sách quản lý Quiz. • Hệ thống thông báo tạo thành công.



Normal Flow	1. Giảng viên truy cập vào trang chi tiết của một khóa học. 2. Giảng viên bấm vào nút "Tạo mới" trong mục Quiz của khóa học 3. Hệ thống hiển thị biểu mẫu "Tạo Quiz mới". Biểu mẫu yêu cầu các thông tin cơ bản như: Tiêu đề, Mô tả, Mức độ khó. 4. Giảng viên nhập thông tin cơ bản và chọn "Thêm câu hỏi". 5. Hệ thống hiển thị giao diện để thêm câu hỏi vào Quiz. 6. Giảng viên chọn loại câu hỏi (ví dụ: trắc nghiệm, điền từ, tự luận) và nhập nội dung câu hỏi, các phương án trả lời (nếu có), đáp án đúng và điểm số cho từng câu. 7. Giảng viên lặp lại bước 5 cho đến khi hoàn tất việc thêm câu hỏi. 8. Giảng viên chọn "Lưu". 9. Hệ thống xác thực dữ liệu đầu vào (ví dụ: Quiz phải có tiêu đề, mỗi câu hỏi phải có đáp án đúng). 10. Hệ thống lưu Quiz vào cơ sở dữ liệu. 11. Hệ thống hiển thị thông báo "Tạo Quiz thành công" và điều hướng Giảng viên về trang quản lý Quiz.
Alternative Flows	A1. Hủy bỏ tạo Quiz: Tại bất kỳ bước nào trong quá trình (từ bước 2 đến 7), Giảng viên chọn nút "Hủy". 1. Hệ thống hiển thị thông báo xác nhận: "Bạn có chắc muốn hủy? Mọi thay đổi sẽ không được lưu." 2. Giảng viên xác nhận "Đồng ý". 3. Hệ thống hủy bỏ thao tác, không lưu Quiz và quay lại trang quản lý Quiz. Usecase kết thúc.



Exceptions	E1. Xác thực dữ liệu thất bại: Tại bước 9, nếu dữ liệu không hợp lệ 1. Hệ thống không lưu Quiz. 2. Hệ thống hiển thị thông báo lỗi ngay tại trường dữ liệu vi phạm. 3. Giảng viên quay lại bước 4 hoặc 6. E2. Lỗi kết nối cơ sở dữ liệu: Tại bước 10, nếu hệ thống không thể kết nối CSDL để lưu. 1. Hệ thống hiển thị thông báo lỗi chung: “Đã có lỗi xảy ra. Không thể lưu Quiz. Vui lòng thử lại.”
-------------------	---

Table 2.16: Bảng mô tả use case Add a Quiz



Use Case ID	UC-TEACHER-04
Use Case Name	Update a Quiz
Primary Actors	Giảng viên
Description	Giảng viên chỉnh sửa nội dung hoặc câu hỏi trong bài quiz.
Trigger	Giảng viên nhấp vào nút “Xem” của một Quiz trong danh sách quản lý Quiz
Preconditions	• Giảng viên đã đăng nhập. • Quiz muốn xem phải tồn tại trong hệ thống.
Postconditions	• Thông tin Quiz được cập nhật trong cơ sở dữ liệu. • Hệ thống thông báo thành công.
Normal Flow	1. Giảng viên truy cập mục Quiz trong trang chi tiết của một khóa học 2. Giảng viên bấm vào nút "Xem" của một Quiz 3. Hệ thống truy xuất dữ liệu Quiz hiện tại. 4. Giảng viên thực hiện các thay đổi. 5. Giảng viên chọn “Lưu thay đổi”. 6. Hệ thống xác thực dữ liệu đã thay đổi. 7. Hệ thống cập nhật thông tin Quiz trong cơ sở dữ liệu. 8. Hệ thống hiển thị thông báo “Cập nhật Quiz thành công”. 9. Hệ thống điều hướng về trang chi tiết Quiz hoặc trang quản lý Quiz.

Alternative Flows	A1. Hủy cập nhật: 4a. Giảng viên chọn “Hủy” 4b. Hệ thống hiển thị thông báo xác nhận: “Bạn có chắc muốn hủy? Các thay đổi sẽ không được lưu.” 4c. Giảng viên xác nhận “Đồng ý”. 4d. Hệ thống hủy bỏ các thay đổi và quay lại trang chi tiết Quiz (không thay đổi). Usecase kết thúc.
Exceptions	E1. Xác thực dữ liệu thất bại: Tại bước 6, nếu dữ liệu không hợp lệ - Hệ thống không lưu. - Hiển thị thông báo lỗi tại trường vi phạm. Giảng viên ở lại biểu mẫu chỉnh sửa (bước 4). E2. Cảnh báo chỉnh sửa Quiz đã có người làm: Tại bước 3, nếu hệ thống phát hiện Quiz này đã có dữ liệu làm bài của học viên. - Hệ thống hiển thị cảnh báo: “Quiz này đã có học viên làm bài. Việc thay đổi nội dung câu hỏi có thể ảnh hưởng đến kết quả thống kê và điểm số trước đó.” - Giảng viên có thể chọn “Tiếp tục chỉnh sửa” hoặc “Hủy”. - Nếu tiếp tục, quay lại bước 3

Table 2.17: Bảng mô tả use case Update a Quiz



Use Case ID	UC-TEACHER-05
Use Case Name	Delete a Quiz
Primary Actors	Giảng viên
Description	Giảng viên xóa bài quiz không còn sử dụng.
Trigger	Giảng viên chọn nút “Xóa” của một Quiz cụ thể trong trang quản lý Quiz.
Preconditions	• Giảng viên đã đăng nhập. • Quiz muốn xóa phải tồn tại.
Postconditions	• Quiz được xóa khỏi hệ thống. • Quiz không còn xuất hiện trong danh sách quản lý.
Normal Flow	1. Giảng viên truy cập mục Quiz trong trang chi tiết khóa học 2. Giảng viên bấm vào nút "Xóa" của một Quiz. 3. Hệ thống hiển thị hộp thoại xác nhận: “Bạn có chắc muốn xóa Quiz [Tên Quiz]? Hành động này không thể hoàn tác.” 4. Giảng viên chọn “Xác nhận”. 5. Hệ thống kiểm tra các ràng buộc (ví dụ: Quiz đã có dữ liệu làm bài của học viên chưa). 6. Hệ thống thực hiện xóa Quiz khỏi cơ sở dữ liệu. 7. Hệ thống hiển thị thông báo “Đã xóa Quiz thành công”. 8. Hệ thống tải lại trang quản lý Quiz.
Alternative Flows	A1. Hủy xóa: 4a. Giảng viên chọn “Hủy” trong hộp thoại xác nhận. 4b. Hệ thống đóng hộp thoại.

Table 2.18: Bảng mô tả use case Delete a Quiz



Use Case ID	UC-TEACHER-06
Use Case Name	View a Quiz Analytics Report
Primary Actors	Giảng viên
Description	Giảng viên xem báo cáo thống kê kết quả làm quiz của sinh viên.
Trigger	Giảng viên chọn chức năng “Báo cáo” từ một Quiz cụ thể trong trang Quản lý Quiz.
Preconditions	Giảng viên đã đăng nhập.
Postconditions	Hệ thống hiển thị trang báo cáo phân tích chi tiết cho Quiz đã chọn.
Normal Flow	1. Hệ thống truy vấn cơ sở dữ liệu và tổng hợp tất cả dữ liệu làm bài liên quan đến Quiz này. 2. Hệ thống tính toán các chỉ số thống kê. 3. Hệ thống hiển thị trang báo cáo bao gồm: • Tổng quan: Điểm trung bình, tỷ lệ hoàn thành, điểm cao nhất, điểm thấp nhất, thời gian làm bài trung bình. • Phân bố điểm: Biểu đồ hiển thị phân bố điểm số của học viên. • Phân tích câu hỏi : Tỷ lệ trả lời đúng/sai cho từng câu hỏi (giúp xác định câu hỏi quá khó hoặc quá dễ). • Danh sách kết quả: Bảng liệt kê điểm số chi tiết của từng học viên đã làm Quiz.



Alternative Flows	A1. Lọc báo cáo (Filter): Tại bước 4, Giảng viên sử dụng các bộ lọc (ví dụ: lọc theo nhóm lớp, theo thời gian). 4a. Giảng viên áp dụng bộ lọc. 4b. Hệ thống quay lại bước 2, truy vấn và tính toán lại dữ liệu dựa trên bộ lọc mới và hiển thị lại báo cáo. A2. Xuất báo cáo (Export): Tại bước 4, Giảng viên chọn nút “Xuất” (ví dụ: CSV, PDF). 4a. Hệ thống tạo tệp báo cáo theo định dạng đã chọn. 4b. Hệ thống cung cấp tệp cho Giảng viên tải về.
Exceptions	E1. Quiz chưa có dữ liệu làm bài: Tại bước 2, nếu hệ thống không tìm thấy bất kỳ lượt làm bài nào. 1. Hệ thống hiển thị thông báo: “Chưa có dữ liệu làm bài cho Quiz này để tạo báo cáo.”

Table 2.19: Bảng mô tả use case View a Quiz Analytics Report



Use Case ID	UC-TEACHER-07
Use Case Name	Add an Exam
Primary Actors	Giảng viên
Description	Giảng viên tạo bài thi đánh giá tổng hợp cho khóa học.
Trigger	Giảng viên bấm nút "Tạo mới" trong mục "Bài thi" trong trang chi tiết của một khóa học
Preconditions	Giảng viên đã đăng nhập vào hệ thống
Postconditions	Một bài thi mới của khóa học được tạo thành công và thêm vào danh sách bài thi
Normal Flow	1. Giảng viên truy cập vào trang chi tiết của một khóa học. 2. Giảng viên bấm vào nút "Tạo mới" trong mục "Bài thi" của khóa học 3. Hệ thống hiển thị biểu mẫu "Tạo bài thi mới". Biểu mẫu yêu cầu các thông tin cơ bản như: Tiêu đề, Mô tả. 4. Giảng viên nhập thông tin cơ bản và chọn "Thêm câu hỏi". 5. Hệ thống hiển thị giao diện để thêm câu hỏi vào bài thi. 6. Giảng viên chọn loại câu hỏi (ví dụ: trắc nghiệm, điền từ, tự luận) và nhập nội dung câu hỏi, các phương án trả lời (nếu có), đáp án đúng và điểm số cho từng câu. 7. Giảng viên lặp lại bước 5 cho đến khi hoàn tất việc thêm câu hỏi. 8. Giảng viên chọn "Lưu". 9. Hệ thống xác thực dữ liệu đầu vào (ví dụ: Bài thi phải có tiêu đề, mỗi câu hỏi phải có đáp án đúng). 10. Hệ thống lưu bài thi vào cơ sở dữ liệu. 11. Hệ thống hiển thị thông báo "Tạo bài thi thành công" và điều hướng giảng viên về trang quản lý bài thi.



Alternative Flows	A1. Giảng viên không tạo bài thi nữa: Bất cứ lúc nào từ bước 4 đến 8, giảng viên có thể bấm vào nút "Hủy" để hủy bỏ việc tạo bài thi
Exceptions	E1. Lỗi hệ thống khi tạo bài thi: Hệ thống thông báo lỗi: "Lỗi khi tạo bài thi mới. Vui lòng thử lại sau"

Table 2.20: Bảng mô tả use case Add an Exam



Use Case ID	UC-TEACHER-08
Use Case Name	Update an Exam
Primary Actors	Giảng viên
Description	Giảng viên chỉnh sửa nội dung hoặc cấu trúc của bài thi đánh giá tổng hợp khóa học.
Trigger	Giảng viên chọn tùy chọn “Chỉnh sửa” cho một bài thi cụ thể trong giao diện quản lý nội dung của một khóa học.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đang ở trang quản lý nội dung của khóa học. • Bài thi mà giảng viên muốn cập nhật đã tồn tại trong hệ thống.
Postconditions	• Các thông tin thay đổi (ví dụ: mô tả, yêu cầu, ngày hết hạn) của bài thi được lưu thành công vào cơ sở dữ liệu. • Sinh viên truy cập vào bài thi sẽ thấy thông tin đã được cập nhật.



Normal Flow	1. Giảng viên điều hướng đến trang quản lý nội dung của khóa học. 2. Hệ thống hiển thị danh sách các bài thi hiện có. 3. Giảng viên tìm và chọn bài thi cần cập nhật, sau đó nhấn nút “Chỉnh sửa”. 4. Hệ thống hiển thị một biểu mẫu chỉnh sửa, đã điền sẵn các thông tin hiện tại của bài thi (ví dụ: tiêu đề, mô tả, yêu cầu, ngày hết hạn, tệp đính kèm). 5. Giảng viên thực hiện các thay đổi mong muốn. 6. Sau khi hoàn tất các thay đổi, giảng viên nhấn nút “Cập nhật”. 7. Hệ thống xác thực dữ liệu đầu vào (ví dụ: đảm bảo tiêu đề không bị để trống, ngày hết hạn phải sau ngày bắt đầu). 8. Hệ thống lưu trữ các thay đổi vào cơ sở dữ liệu, ghi đè thông tin cũ. 9. Hệ thống hiển thị thông báo thành công. 10. Hệ thống điều hướng giảng viên trở lại trang chi tiết của bài thi vừa được cập nhật.
Alternative Flows	A1: Giảng viên hủy bỏ thao tác cập nhật 5a. Giảng viên nhấn nút “Hủy”. 5b. Hệ thống hiển thị một hộp thoại xác nhận. 5c. Giảng viên xác nhận “Đồng ý”. 5d. Hệ thống hủy bỏ mọi thay đổi, đóng biểu mẫu chỉnh sửa và quay lại trang danh sách bài tập.



Exceptions	E1: Xác thực dữ liệu thất bại 7a. Hệ thống hiển thị thông báo lỗi cụ thể ngay bên cạnh trường nhập liệu không hợp lệ và cho giảng viên chỉnh sửa lại lỗi
	E2: Lỗi hệ thống khi đang lưu 8a. Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi hệ thống. Không thể cập nhật bài thi. Vui lòng thử lại sau.”

Table 2.21: Bảng mô tả use case Update an Exam



Use Case ID	UC-TEACHER-10
Use Case Name	Add an Assignment
Primary Actors	Giảng viên
Description	Giảng viên tạo bài tập cho sinh viên.
Trigger	Giảng viên chọn tùy chọn “Tạo bài tập mới” trong giao diện quản lý nội dung của một khóa học.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đang ở trang quản lý nội dung của khóa học
Postconditions	• Một bài tập mới được tạo và lưu trữ trong cơ sở dữ liệu. • Hệ thống thông báo tạo thành công.
Normal Flow	1. Giảng viên chọn tùy chọn “Tạo bài tập mới” 2. Hệ thống hiển thị biểu mẫu “Tạo bài tập mới”. Biểu mẫu yêu cầu các thông tin cơ bản (ví dụ: tiêu đề, mô tả, yêu cầu, ngày hết hạn, tệp đính kèm, mức độ khó, các tệp cần nộp). 3. Giảng viên nhập thông tin cơ bản. 4. Giảng viên nhấn nút “Lưu”. 5. Hệ thống xác thực dữ liệu đầu vào (ví dụ: đảm bảo tiêu đề không bị để trống, ngày hết hạn phải sau ngày bắt đầu). 6. Hệ thống lưu bài tập vào cơ sở dữ liệu. 7. Hệ thống hiển thị thông báo “Tạo bài tập thành công” và điều hướng Giảng viên về trang quản lý bài tập



Alternative Flows	A1: Giảng viên hủy bỏ thao tác cập nhật 3a. Giảng viên nhấn nút “Hủy”. 3b. Hệ thống hiển thị một hộp thoại xác nhận. 3c. Giảng viên xác nhận “Đồng ý”. 3d. Hệ thống hủy bỏ mọi thay đổi, đóng biểu mẫu chỉnh sửa và quay lại trang danh sách bài tập.
Exceptions	E1: Xác thực dữ liệu thất bại 5a. Hệ thống hiển thị thông báo lỗi cụ thể ngay bên cạnh trường nhập liệu không hợp lệ và cho giảng viên chỉnh sửa lại lỗi E2: Lỗi hệ thống khi đang lưu 6a. Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi hệ thống. Không thể thêm bài tập. Vui lòng thử lại sau.”

Table 2.22: Bảng mô tả use case Add an Assignment



Use Case ID	UC-TEACHER-09
Use Case Name	View an Exam Analytics Report
Primary Actors	Giảng viên
Description	Giảng viên xem báo cáo phân tích kết quả bài thi của sinh viên.
Trigger	Giảng viên chọn chức năng “Báo cáo” từ giao diện quản lý một bài thi trong khóa học.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đang ở trang quản lý bài thi cần xem báo cáo.
Postconditions	• Hệ thống hiển thị trang báo cáo phân tích chi tiết cho bài thi đã chọn.
Normal Flow	1. Giảng viên điều hướng đến giao diện quản lý bài thi và chọn chức năng “Báo cáo”. 2. Hệ thống truy vấn cơ sở dữ liệu và tổng hợp tất cả dữ liệu liên quan đến bài thi này. 3. Hệ thống hiển thị trang báo cáo bao gồm: • Tổng quan: Điểm trung bình, tỷ lệ hoàn thành, điểm cao nhất, điểm thấp nhất, thời gian làm bài trung bình. • Phân bổ điểm: Biểu đồ hiển thị phân bổ điểm số của học viên. • Phân tích câu hỏi : Tỷ lệ trả lời đúng/sai cho từng câu hỏi. • Danh sách kết quả: Bảng liệt kê điểm số chi tiết của từng học viên đã làm bài thi.



Alternative Flows	A1: Lọc báo cáo (Filter) 3a. Giảng viên áp dụng bộ lọc (ví dụ: lọc theo nhóm lớp, theo thời gian). 3b. Hệ thống quay lại bước 2, truy vấn và tính toán lại dữ liệu dựa trên bộ lọc mới và hiển thị lại báo cáo. A2: Xuất báo cáo 3a. Giảng viên chọn nút “Xuất” (ví dụ: CSV, PDF). 3b. Hệ thống tạo tệp báo cáo theo định dạng đã chọn. 3c. Hệ thống cung cấp tệp cho Giảng viên tải về hoặc in.
Exceptions	E1: Quiz chưa có dữ liệu làm bài 3a. Hệ thống hiển thị thông báo: “Chưa có dữ liệu làm bài cho Quiz này để tạo báo cáo.” 3b. Hệ thống trở về giao diện quản lý bài thi.

Table 2.23: Bảng mô tả use case View an Exam Analytics Report



Use Case ID	UC-TEACHER-11
Use Case Name	Update an Assignment
Primary Actors	Giảng viên
Description	Giảng viên chỉnh sửa yêu cầu hoặc mô tả của bài tập.
Trigger	Giảng viên chọn tùy chọn “Chỉnh sửa” cho một bài tập cụ thể trong giao diện quản lý nội dung của một khóa học.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đang ở trang quản lý nội dung của khóa học. • Bài tập mà giảng viên muốn cập nhật đã tồn tại trong hệ thống.
Postconditions	• Các thông tin thay đổi (ví dụ: mô tả, yêu cầu, ngày hết hạn) của bài tập được lưu thành công vào cơ sở dữ liệu. • Sinh viên truy cập vào bài tập sẽ thấy thông tin đã được cập nhật.



Normal Flow	1. Giảng viên điều hướng đến trang quản lý nội dung của khóa học. 2. Hệ thống hiển thị danh sách các bài tập hiện có. 3. Giảng viên tìm và chọn bài tập cần cập nhật, sau đó nhấn nút “Chỉnh sửa”. 4. Hệ thống hiển thị một biểu mẫu chỉnh sửa, đã điền sẵn các thông tin hiện tại của bài tập (ví dụ: tiêu đề, mô tả, yêu cầu, ngày hết hạn, tệp đính kèm). 5. Giảng viên thực hiện các thay đổi mong muốn. 6. Sau khi hoàn tất các thay đổi, giảng viên nhấn nút “Cập nhật”. 7. Hệ thống xác thực dữ liệu đầu vào (ví dụ: đảm bảo tiêu đề không bị để trống, ngày hết hạn phải sau ngày bắt đầu). 8. Hệ thống lưu trữ các thay đổi vào cơ sở dữ liệu, ghi đè thông tin cũ. 9. Hệ thống hiển thị thông báo thành công. 10. Hệ thống điều hướng giảng viên trở lại trang chi tiết của bài tập vừa được cập nhật.
Alternative Flows	A1: Giảng viên hủy bỏ thao tác cập nhật 5a. Giảng viên nhấn nút “Hủy”. 5b. Hệ thống hiển thị một hộp thoại xác nhận. 5c. Giảng viên xác nhận “Đồng ý”. 5d. Hệ thống hủy bỏ mọi thay đổi, đóng biểu mẫu chỉnh sửa và quay lại trang danh sách bài tập.



Exceptions	E1: Xác thực dữ liệu thất bại • Hệ thống hiển thị thông báo lỗi cụ thể ngay bên cạnh trường nhập liệu không hợp lệ và cho giảng viên chỉnh sửa lại lỗi E2: Lỗi hệ thống khi đang lưu • Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi hệ thống. Không thể cập nhật bài tập. Vui lòng thử lại sau.”
-------------------	---

Table 2.24: Bảng mô tả use case View an Update an Assignment



Use Case ID	UC-TEACHER-12
Use Case Name	Delete an Assignment
Primary Actors	Giảng viên
Description	Giảng viên xóa bài tập không còn cần thiết
Trigger	Giảng viên chọn tùy chọn “Xóa” cho một bài tập cụ thể trong giao diện quản lý nội dung trong một khóa học
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor” • Giảng viên đang xem danh sách các bài tập trong một khóa học mà họ có quyền quản lý • Bài tập mà giảng viên muốn xóa đang tồn tại trong hệ thống
Postconditions	• Bài tập được xóa (hoặc được đánh dấu là “đã xóa”) khỏi khóa học • Bài tập không còn xuất hiện trong lộ trình học tập của sinh viên

Normal Flow	1. Giảng viên điều hướng đến trang quản lý nội dung của khóa học 2. Hệ thống hiển thị danh sách các bài tập hiện có 3. Giảng viên tìm và chọn bài tập cụ thể mà họ muốn xóa 4. Giảng viên nhấn vào nút “Xóa” bên cạnh bài tập đó 5. Hệ thống hiển thị một hộp thoại xác nhận, cảnh báo về hậu quả của việc xóa 6. Giảng viên nhấn nút “Xác nhận xóa” 7. Hệ thống tiến hành xóa bài tập và tất cả dữ liệu liên quan (bài nộp, điểm số) khỏi cơ sở dữ liệu 8. Hệ thống hiển thị thông báo thành công 9. Hệ thống tự động làm mới danh sách bài tập
Alternative Flows	A1: Giảng viên hủy bỏ thao tác xóa 6a. Giảng viên nhấn nút “Hủy” 6b. Hệ thống đóng hộp thoại xác nhận.
Exceptions	E1: Lỗi hệ thống trong quá trình xóa • Hệ thống hiển thị một thông báo lỗi: “Đã xảy ra lỗi. Không thể hoàn tất thao tác xóa. Vui lòng thử lại sau.”

Table 2.25: Bảng mô tả use case Delete an Assignment



Use Case ID	UC-TEACHER-13
Use Case Name	Manage Learning Materials
Primary Actors	Giảng viên
Description	Giảng viên quản lý và cập nhật tài liệu học tập trong khóa học.
Trigger	Giảng viên chọn chức năng “Quản lý nội dung” từ bảng điều khiển của một khóa học
Preconditions	• Giảng viên đã đăng nhập thành công vào hệ thống với vai trò “Tutor”. • Hệ thống đã tải và hiển thị Bảng điều khiển Giảng viên (Instructor Dashboard).
Postconditions	• Tạo mới: Tài liệu học tập mới được lưu vào cơ sở dữ liệu của hệ thống và được liên kết với môn học/chủ đề tương ứng. • Cập nhật: Các thay đổi đối với tài liệu học tập được lưu lại thành công. • Xóa: Tài liệu học tập bị xóa khỏi hệ thống (hoặc được đánh dấu là “không hoạt động”).



Normal Flow	1. Giảng viên chọn môn học cần quản lý nội dung 2. Giảng viên điều hướng đến phần “Quản lý nội dung” của môn học 3. Hệ thống hiển thị danh sách các tài liệu học tập hiện có và các nút hành động (ví dụ: “Tạo mới”, “Chỉnh sửa”, “Xóa”). 4. Giảng viên chọn “Tạo mới” (để thêm tài liệu) hoặc chọn một tài liệu cụ thể và nhấn “Chỉnh sửa” (để cập nhật). 5. Hệ thống hiển thị một biểu mẫu nhập liệu. 6. Giảng viên điền hoặc cập nhật các thông tin cho tài liệu: 7. Giảng viên nhấn “Lưu”. 8. Hệ thống xác thực tất cả dữ liệu đầu vào (ví dụ: đảm bảo các trường bắt buộc đã được điền, URL video hợp lệ). 9. Hệ thống lưu tài liệu (hoặc các thay đổi) vào cơ sở dữ liệu. 10. Hệ thống hiển thị thông báo “Lưu thành công” và chuyển hướng giảng viên trở lại danh sách tài liệu (bước 3), nơi tài liệu mới/cập nhật được hiển thị.
--------------------	---

Alternative Flows	A1: Giảng viên xóa một tài liệu học tập 4a. Giảng viên tìm tài liệu muốn xóa và nhấn nút “Xóa” 4b. Hệ thống hiển thị một hộp thoại xác nhận 4c. Giảng viên nhấn “Xác nhận xóa”. 4d. Hệ thống xóa tài liệu (hoặc đánh dấu là đã xóa/vô hiệu hóa). 4e. Hệ thống hiển thị thông báo “Xóa thành công” và làm mới danh sách tài liệu. A2: Giảng viên hủy bỏ thao tác 7a. Giảng viên nhấn “Hủy” 7b. Hệ thống cảnh báo về việc mất dữ liệu chưa lưu (nếu có) 7c. Hệ thống đóng hộp thoại và quay lại danh sách tài liệu.
Exceptions	E1: Xác thực dữ liệu thất bại - Hệ thống hiển thị thông báo lỗi cụ thể ngay bên cạnh (các trường nhập liệu không hợp lệ (ví dụ: “Vui lòng nhập tiêu đề cho tài liệu này”). E2: Lỗi hệ thống khi đang lưu - Hệ thống hiển thị một thông báo lỗi chung (ví dụ: “Đã xảy ra lỗi hệ thống. Không thể lưu tài liệu. Vui lòng thử lại sau.”).

Table 2.26: Bảng mô tả use case Manage Learning Materials



Use Case ID	UC-TEACHER-14
Use Case Name	Provide Student Learning Feedback
Primary Actors	Giảng viên
Description	Giảng viên gửi nhận xét, đánh giá tiến trình học tập trong một khóa học của sinh viên.
Trigger	Giảng viên chọn một sinh viên cụ thể từ danh sách lớp của khóa học để xem xét và đưa ra phản hồi
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên có quyền quản lý khóa học. • Sinh viên (mà giảng viên muốn đánh giá) đã được ghi danh vào khóa học.
Postconditions	• Phản hồi của giảng viên được lưu lại trong hệ thống và liên kết với hồ sơ của sinh viên đó. • Sinh viên nhận được thông báo về phản hồi mới.

Normal Flow	1. Từ bảng điều khiển, giảng viên chọn khóa học muốn quản lý. 2. Giảng viên chọn mục “Sinh viên”. 3. Hệ thống hiển thị danh sách tất cả sinh viên trong khóa học. 4. Giảng viên nhấn vào tên của một sinh viên cụ thể để xem chi tiết. 5. Hệ thống chuyển hướng đến trang “Chi tiết sinh viên”. 6. Trên trang này, hệ thống hiển thị toàn bộ hồ sơ học tập của sinh viên trong khóa học 7. Giảng viên tìm đến khu vực nhập liệu “Nhận xét của giảng viên”. 8. Giảng viên nhập nội dung đánh giá của mình. 9. Giảng viên nhấn nút “Gửi”. 10. Hệ thống xác thực dữ liệu. 11. Hệ thống lưu phản hồi vào cơ sở dữ liệu. 12. Hệ thống hiển thị thông báo “Gửi phản hồi thành công.”
Exceptions	E1: Xác thực dữ liệu thất bại • Tại bước 10, nếu nội dung phản hồi bị bỏ trống, hệ thống hiển thị thông báo lỗi tại chỗ: “Nội dung phản hồi không được để trống.” E2: Lỗi hệ thống khi đang lưu • Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi, không thể lưu phản hồi. Vui lòng thử lại.”

Table 2.27: Bảng mô tả use case Provide Student Learning Feedback



Use Case ID	UC-TEACHER-15
Use Case Name	View Student Analytics Report
Primary Actors	Giảng viên
Description	Giảng viên xem báo cáo phân tích tổng quan về kết quả học tập của sinh viên.
Trigger	Giảng viên chọn mục “Báo cáo phân tích” của một khóa học cụ thể từ bảng điều khiển.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đã được gán quyền quản lý khóa học mà họ muốn xem báo cáo. • Khóa học đã có sinh viên đăng ký và đã phát sinh dữ liệu học tập.
Postconditions	Hệ thống hiển thị thành công trang báo cáo phân tích cho giảng viên.
Normal Flow	1. Từ bảng điều khiển, giảng viên chọn một khóa học cụ thể. 2. Giảng viên điều hướng đến mục “Báo cáo phân tích” của khóa học đó. 3. Hệ thống gửi yêu cầu tổng hợp dữ liệu liên quan đến khóa học. 4. Hệ thống truy vấn và tổng hợp dữ liệu. 5. Hệ thống xử lý dữ liệu và trình bày dưới dạng một bảng Phân tích tổng quan. 6. Giảng viên xem các biểu đồ và số liệu thống kê để đánh giá hiệu suất tổng thể của lớp.



Alternative Flows	A1: Xuất báo cáo 6a. Giảng viên nhấn nút “Xuất báo cáo”. 6b. Hệ thống hiển thị các tùy chọn định dạng (ví dụ: PDF, CSV). 6c. Giảng viên chọn định dạng mong muốn. 6d. Hệ thống tạo tệp báo cáo ở định dạng đã chọn và cung cấp cho giảng viên để tải xuống.
Exceptions	E1: Lỗi hệ thống khi đang tổng hợp dữ liệu Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi khi tạo báo cáo. Vui lòng thử lại sau.”

Table 2.28: Bảng mô tả use case View Student Analytics Report



Use Case ID	UC-TEACHER-16
Use Case Name	Grade an Assignment
Primary Actors	Giảng viên
Description	Giảng viên chấm điểm và nhận xét kết quả bài tập của sinh viên.
Trigger	Giảng viên chọn một bài tập đã được sinh viên nộp và đang ở trạng thái “Cần chấm điểm”.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đã vào trang quản lý của khóa học. • Bài tập đã được tạo và giao cho sinh viên. • Ít nhất một sinh viên đã nộp bài làm cho bài tập này.
Postconditions	• Điểm số của sinh viên cho bài tập này được lưu vào “Sổ điểm” của hệ thống. • Nhận xét bằng văn bản của giảng viên được lưu và liên kết với bài nộp của sinh viên. • Trạng thái của bài nộp được cập nhật từ “Cần chấm điểm” thành “Đã chấm điểm”. • Sinh viên có thể xem điểm và nhận xét của mình.

Normal Flow	1. Từ bảng điều khiển, giảng viên chọn khóa học và điều hướng đến mục quản lý nội dung. 2. Giảng viên chọn một bài tập cụ thể. 3. Giảng viên bấm vào nút “Danh sách bài nộp”. 4. Hệ thống hiển thị danh sách tất cả sinh viên, cùng với trạng thái bài nộp của họ. 5. Giảng viên chọn một sinh viên có trạng thái “Cần chấm điểm”. 6. Hệ thống mở ra giao diện chấm điểm. 7. Giảng viên xem xét kỹ lưỡng bài nộp của sinh viên. 8. Giảng viên nhập một điểm số vào ô “Điểm số” và viết nhận xét chi tiết vào ô “Nhận xét” hoặc nhấn “Chấm Điểm Tự Động” để gửi yêu cầu đến hệ thống AI và tự động điền điểm số, nhận xét. 9. Giảng viên nhấn nút “Hoàn tất Chấm điểm”. 10. Hệ thống lưu điểm số và nhận xét vào cơ sở dữ liệu. 11. Hệ thống cập nhật trạng thái bài nộp thành “Đã chấm điểm”. 12. Hệ thống hiển thị thông báo thành công.
Exceptions	E1: Lỗi hệ thống khi đang lưu • Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi. Không thể lưu điểm của bạn. Vui lòng thử lại.”
Notes and Issues	Tích hợp AI hỗ trợ đưa ra nhận xét cho giảng viên.

Table 2.29: Bảng mô tả use case Grade an Assignment



Use Case ID	UC-TEACHER-17
Use Case Name	Delete an Exam
Primary Actors	Giảng viên
Description	Giảng viên xóa bài thi đánh giá tổng hợp khóa học không còn cần thiết.
Trigger	Giảng viên chọn tùy chọn “Xóa” của bài thi tổng hợp từ giao diện quản lý nội dung của khóa học.
Preconditions	• Giảng viên đã đăng nhập vào hệ thống với vai trò “Tutor”. • Giảng viên đang ở trong trang quản lý của khóa học mà họ có quyền chỉnh sửa. • Bài thi mà giảng viên muốn xóa đang tồn tại trong hệ thống.
Postconditions	• Bài thi và tất cả các thành phần của nó bị xóa vĩnh viễn (hoặc được vô hiệu hóa/lưu trữ). • Bài thi không còn xuất hiện trong lộ trình học tập hay sổ điểm của sinh viên.

Normal Flow	1. Giảng viên điều hướng đến trang quản lý nội dung trong giao diện quản lý khóa học. 2. Hệ thống hiển thị danh sách tất cả các bài thi và bài kiểm tra đã được tạo cho khóa học. 3. Giảng viên tìm và chọn bài thi tổng hợp mà họ muốn xóa. 4. Giảng viên nhấn vào nút “Xóa”. 5. Hệ thống hiển thị một hộp thoại xác nhận với mức độ cảnh báo cao. 6. Giảng viên đọc cảnh báo và nhấn nút “Xác nhận xóa”. 7. Hệ thống tiến hành xóa bài thi, các câu hỏi trong ngân hàng câu hỏi (nếu liên quan), và tất cả điểm số/bài nộp của sinh viên cho bài thi này. 8. Hệ thống hiển thị thông báo thành công. 9. Hệ thống làm mới danh sách bài thi.
Alternative Flows	A1: Giảng viên hủy bỏ thao tác xóa 5a. Giảng viên nhấn nút “Hủy”. 5b. Hệ thống đóng hộp thoại xác nhận.
Exceptions	E1: Lỗi hệ thống trong quá trình xóa (lỗi cơ sở dữ liệu) • Hệ thống hiển thị một thông báo lỗi chung: “Đã xảy ra lỗi hệ thống khi cố gắng xóa bài thi. Không có thay đổi nào được thực hiện. Vui lòng thử lại sau.”

Table 2.30: Bảng mô tả use case Delete an Exam



Use Case ID	UC-TEACHER-18
Use Case Name	Sign In as Tutor
Primary Actors	Giảng viên
Description	Giảng viên đăng nhập vào hệ thống để truy cập các chức năng giảng dạy.
Trigger	Giảng viên truy cập trang đăng nhập của hệ thống và chọn “Đăng nhập bằng tài khoản giảng viên”
Preconditions	• Giảng viên có một tài khoản hợp lệ. • Tài khoản của giảng viên đã được cấp phép và liên kết với vai trò “Tutor” trong cơ sở dữ liệu của hệ thống.
Postconditions	• Hệ thống xác thực danh tính giảng viên thành công. • Hệ thống cấp quyền truy cập cho giảng viên dựa trên vai trò “Tutor”. • Hệ thống chuyển hướng giảng viên đến bảng điều khiển cá nhân của họ.
Normal Flow	1. Giảng viên truy cập trang đăng nhập của hệ thống. 2. Hệ thống hiển thị các tùy chọn đăng nhập. 3. Học viên nhấn vào nút “Đăng nhập bằng tài khoản giảng viên” 4. Giảng viên nhập thông tin đăng nhập (username, password) của họ. 5. Hệ thống xác thực thông tin đăng nhập. 6. Hệ thống tìm thấy tài khoản, thiết lập phiên làm việc và áp dụng các quyền của vai trò “Tutor”. 7. Hệ thống chuyển hướng giảng viên đến bảng điều khiển.



Exceptions	E1: Xác thực tài khoản thất bại Hệ thống thông báo lỗi đến học viên: “Tài khoản hoặc mật khẩu không chính xác. Vui lòng thử lại sau.”
-------------------	--

Table 2.31: Bảng mô tả use case Sign In as Tutor

3 Kiến Trúc Phần Mềm

3.1 Tổng Quan Kiến Trúc

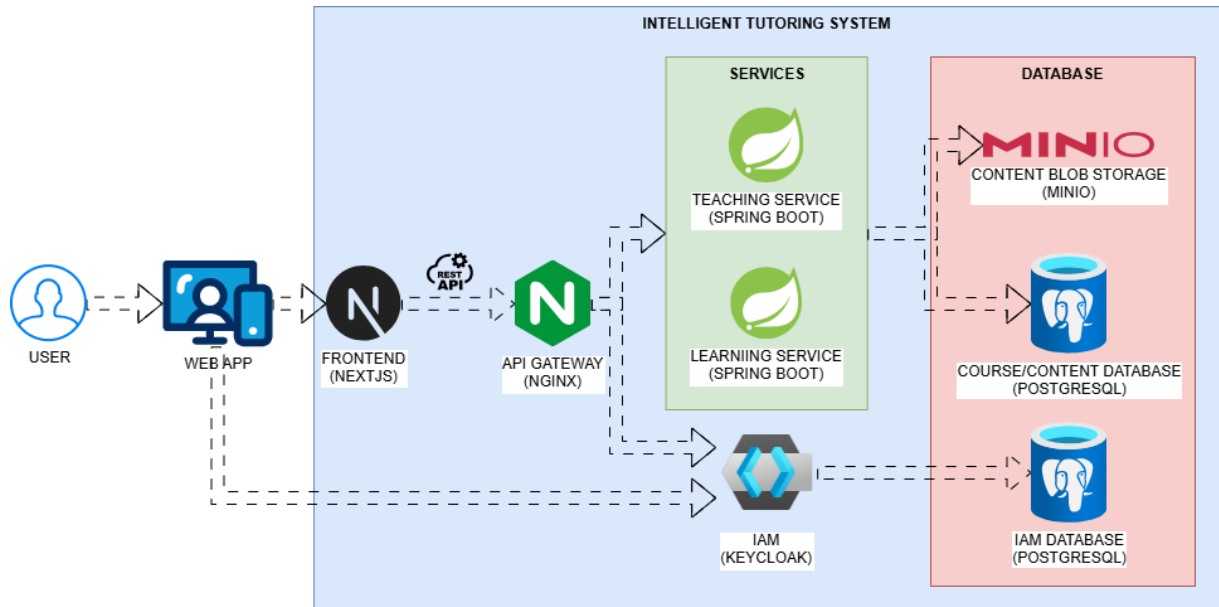


Figure 3.1: Sơ đồ tổng quan kiến trúc

Hệ thống Gia sư Thông minh **IntelliLearn** được xây dựng dựa trên mô hình **Service-Based Architecture**, trong đó các thành phần được tổ chức thành những dịch vụ theo miền nghiệp vụ. Mỗi dịch vụ được triển khai độc lập và giao tiếp thông qua API Gateway, giúp hệ thống linh hoạt, dễ mở rộng và đảm bảo độ tin cậy trong vận hành.

1. **Frontend (Next.js)** Giao diện người dùng được phát triển bằng **Next.js**, hỗ trợ Server-Side Rendering và tối ưu SEO. Người dùng tương tác với hệ thống thông qua Web App; mọi yêu cầu đều được chuyển tiếp đến API Gateway. Quá trình xác thực sẽ chuyển hướng người dùng đến hệ thống IAM để đăng nhập trước khi truy cập các chức năng nội bộ.
2. **API Gateway (Nginx)** API Gateway đóng vai trò điều phối trung tâm, chịu trách nhiệm định tuyến request từ Frontend đến đúng dịch vụ Backend. Bên cạnh đó, Gateway còn hỗ trợ các chức năng như kiểm tra token, lọc request và triển khai các chính sách bảo mật. Tất cả các cuộc gọi từ người dùng vào hệ thống sau khi xác thực đều đi qua Gateway.
3. **Hệ thống Quản lý Định danh (IAM – Keycloak)** Thành phần IAM được triển khai bằng Keycloak, cung cấp cơ chế xác thực và phân quyền theo chuẩn

OAuth2/OpenID Connect. Người dùng được chuyển hướng sang Keycloak để đăng nhập, sau đó nhận token và sử dụng token này trong các yêu cầu gửi đến API Gateway và các dịch vụ backend. Keycloak sử dụng một cơ sở dữ liệu riêng để lưu trữ dữ liệu người dùng, thông tin phân quyền và dữ liệu xác thực.

4. **Backend Services (Spring Boot)** Các dịch vụ backend được phát triển bằng Spring Boot và được phân tách theo miền nghiệp vụ chính:

- **Learning Service:** xử lý các nghiệp vụ liên quan đến học viên như quản lý tiến trình học tập, truy xuất nội dung học liệu và đăng ký khóa học. Dịch vụ truy cập vào Course/Content Database để đọc/ghi dữ liệu học vụ và tương tác với kho nội dung trên MinIO.
- **Teaching Service:** hỗ trợ các tác vụ dành cho giảng viên như quản lý khóa học, nội dung và đánh giá. Dịch vụ này thực hiện truy vấn vào Course/Content Database và gọi đến MinIO khi cần thao tác với tệp học liệu.

5. **Kho Dữ liệu (PostgreSQL và MinIO)** Kiến trúc dữ liệu được tách thành hai phần chính:

- **Course/Content Database (PostgreSQL):** lưu trữ các dữ liệu quan hệ liên quan đến khóa học, nội dung học tập và tiến độ học viên.
- **IAM Database (PostgreSQL):** lưu trữ thông tin người dùng, phân quyền và các dữ liệu phục vụ cho việc xác thực trong Keycloak.
- **Content Blob Storage (MinIO):** lưu trữ các tệp tin, tài liệu học tập và nội dung đa phương tiện. Backend Services truy cập trực tiếp MinIO để đọc và ghi dữ liệu.

Việc tách riêng cơ sở dữ liệu học vụ và cơ sở dữ liệu định danh giúp giảm rủi ro, tối ưu hiệu năng và hỗ trợ áp dụng chính sách bảo mật khác nhau cho từng loại dữ liệu. (Chi tiết tại [ADR-02](#))

Kiến trúc này đảm bảo tính mở rộng, dễ bảo trì và khả năng triển khai độc lập cho từng dịch vụ. Nhờ sử dụng API Gateway và tổ chức hệ thống theo miền nghiệp vụ, IntelliLearn có thể phát triển thêm chức năng mới hoặc mở rộng quy mô hoạt động mà không ảnh hưởng đến các thành phần còn lại.

3.2 Góc Nhìn Phân Rã

3.2.1 Sơ Đồ Chính

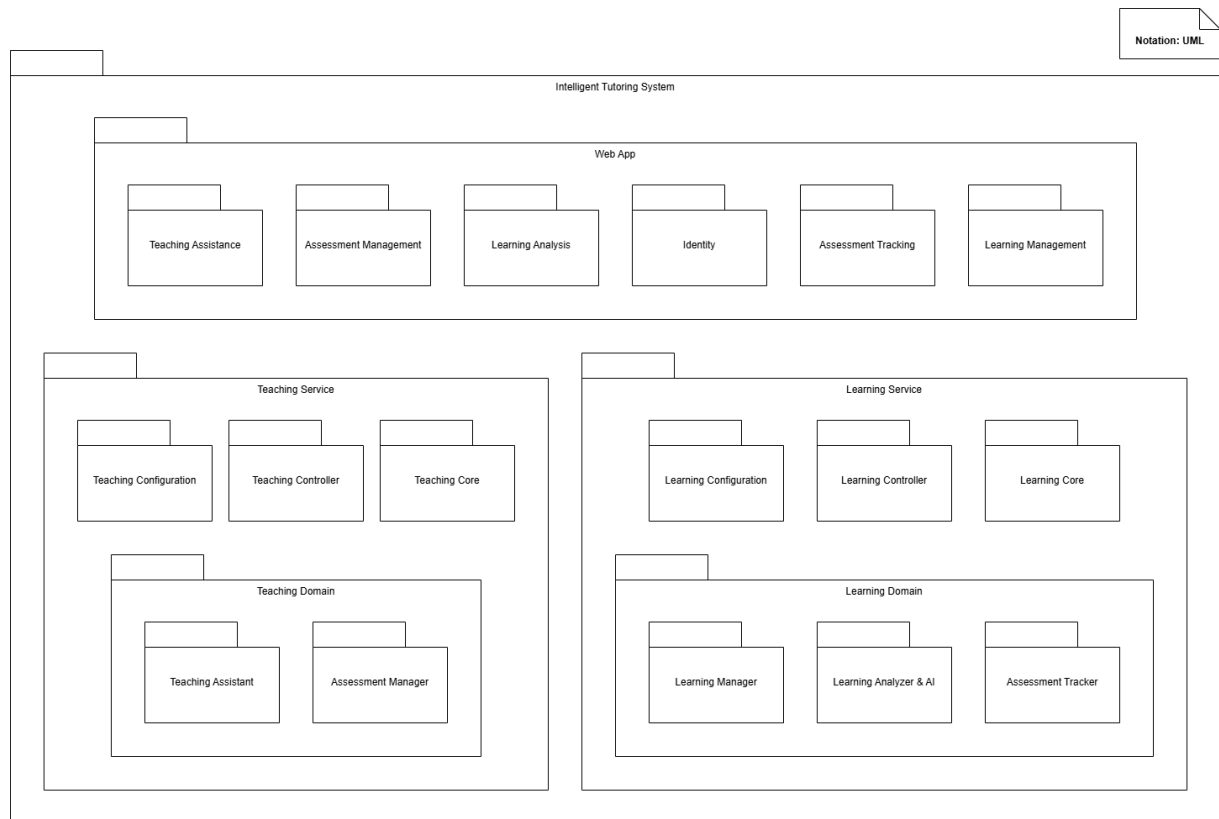


Figure 3.2: Sơ đồ góc nhìn phân rã

3.2.2 Danh Sách Thành Phần

1. **Web App:** Là tầng giao diện người dùng cung cấp môi trường tương tác trực tiếp giữa hệ thống và người dùng cuối, bao gồm giảng viên, học viên và quản trị viên. Tầng này gửi yêu cầu đến các dịch vụ thông qua API và hiển thị kết quả xử lý theo thời gian thực.
 - (a) **Teaching Assistance:** Giao diện hỗ trợ giảng viên trong việc quản lý khóa học, học viên, tài liệu học tập và phản hồi học tập.
 - (b) **Assessment Management:** Cho phép giảng viên tạo, cấu hình và quản lý các bài kiểm tra hoặc bài tập; đồng thời thống kê, phân tích kết quả học tập.
 - (c) **Learning Analysis:** Trình bày biểu đồ, báo cáo và thống kê học tập, hỗ trợ học viên tự đánh giá tiến độ và hiệu quả học tập.



- (d) **Identity:** Đảm nhận các chức năng xác thực và định danh người dùng (đăng nhập, đăng ký, phân quyền).
 - (e) **Assessment Tracking:** Cung cấp giao diện để học viên thực hiện bài kiểm tra, lưu trữ kết quả và nhận phản hồi từ giảng viên hoặc hệ thống.
 - (f) **Learning Management:** Hỗ trợ học viên đăng ký hoặc hủy khóa học, truy cập tài liệu, theo dõi tiến độ học tập và nhận phản hồi.
2. **Teaching Service:** Là tầng xử lý nghiệp vụ giảng dạy, chịu trách nhiệm điều phối các quy trình giảng dạy, cấu hình và tương tác với dữ liệu học tập.
- (a) **Teaching Configuration:** Quản lý tham số cấu hình, chính sách và thông số kỹ thuật của các quy trình giảng dạy.
 - (b) **Teaching Controller:** Cung cấp API để Web App giao tiếp với các nghiệp vụ giảng dạy, tiếp nhận yêu cầu và trả kết quả.
 - (c) **Teaching Core:** Định nghĩa logic xử lý chính cho các hoạt động giảng dạy, đồng thời điều phối các module con trong Teaching Domain.
 - (d) **Teaching Domain:** Bao gồm các module nghiệp vụ cụ thể:
 - **Teaching Assistant:** Quản lý khóa học, học liệu, danh sách học viên và phản hồi giữa giảng viên – học viên.
 - **Assessment Manager:** Xử lý việc tạo, cấu hình, lưu trữ và tổng hợp kết quả đánh giá.
3. **Learning Service:** Là tầng nghiệp vụ học tập, chịu trách nhiệm xử lý các hoạt động liên quan đến cá nhân hóa, tiến độ học tập và phân tích kết quả.
- (a) **Learning Configuration:** Quản lý tham số cấu hình cho hoạt động học tập và các chính sách cá nhân hóa.
 - (b) **Learning Controller:** Đóng vai trò trung gian giữa Web App và logic xử lý của hệ thống học tập, đảm bảo giao tiếp thông qua API thống nhất.
 - (c) **Learning Core:** Chứa các quy trình nghiệp vụ trọng yếu cho cá nhân hóa và quản lý học tập, điều phối hoạt động của Learning Domain.
 - (d) **Learning Domain:** Bao gồm các module nghiệp vụ chính:
 - **Learning Manager:** Quản lý đăng ký, tiến trình học tập và tương tác của học viên với giảng viên.

- **Learning Analyzer & AI:** Ứng dụng kỹ thuật học máy để phân tích và tổng hợp dữ liệu học tập của từng học viên, từ đó đưa ra gợi ý cá nhân hóa về lộ trình và nội dung học tập. Thành phần này đồng thời thống kê và trực quan hóa kết quả học tập.
- **Assessment Tracker:** Theo dõi, ghi nhận và lưu trữ kết quả đánh giá, phục vụ phản hồi và phân tích.

3.2.3 Hướng Dẫn Thay Đổi

- **Thay đổi giao diện người dùng:** Thực hiện tại tầng Web App mà không ảnh hưởng đến tầng dịch vụ. Các API của Controller trong Service vẫn được giữ nguyên.
- **Thêm chức năng mới:** Thêm module tương ứng trong Domain của tầng Service và mở rộng Controller để cung cấp API cho Web App.
- **Thay đổi cấu hình hệ thống:** Thực hiện trong module Configuration của Service tương ứng, không làm gián đoạn hoạt động logic nghiệp vụ.

3.3 Góc Nhìn Sử Dụng

3.3.1 Sơ Đồ Chính

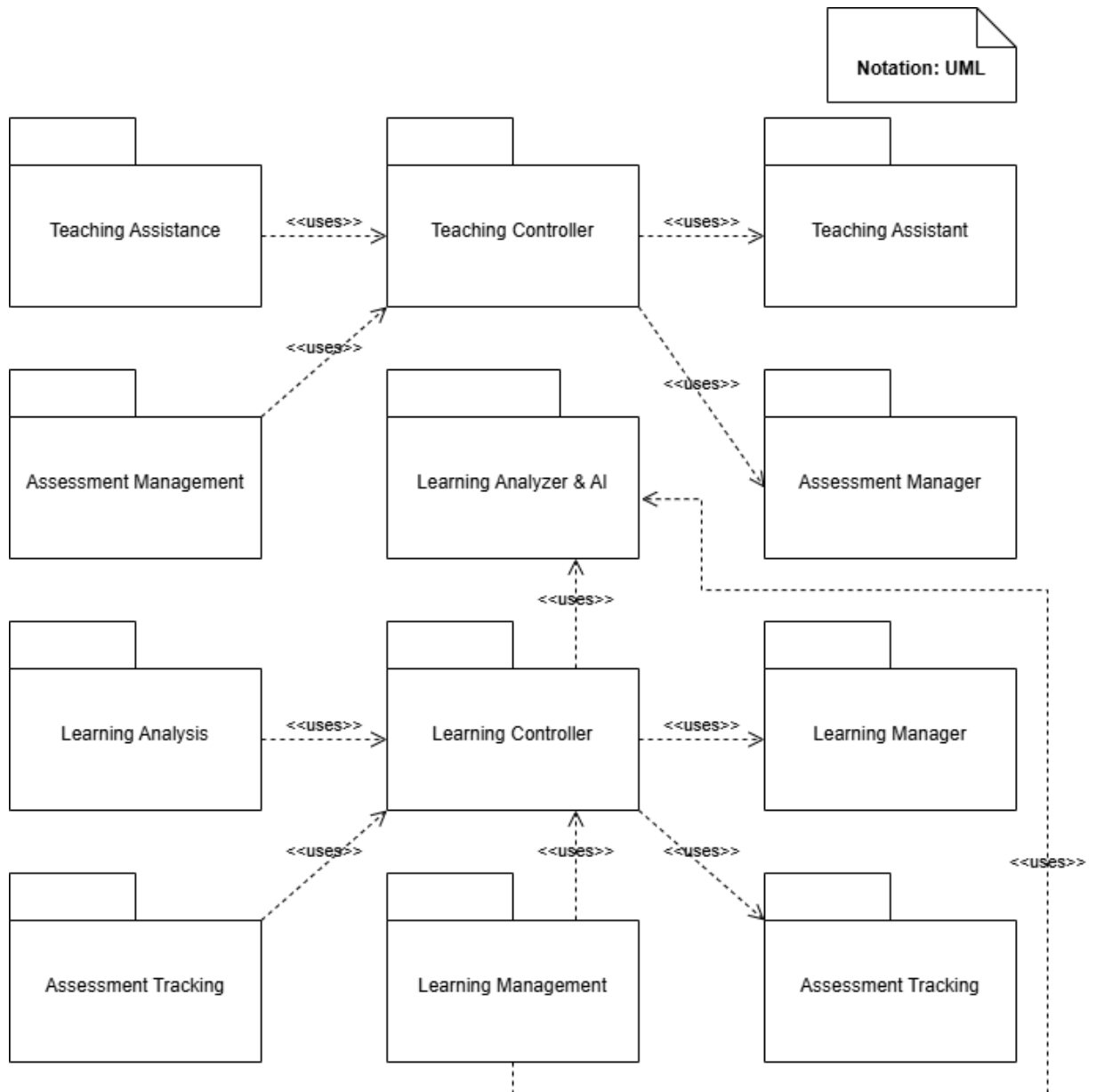


Figure 3.3: Sơ đồ góc nhìn sử dụng

3.3.2 Danh Sách Thành Phần

1. **Teaching Assistance:** Giao diện hỗ trợ giảng viên trong việc quản lý khóa học, học viên, tài liệu học tập và phản hồi học tập.



2. **Assessment Management:** Cho phép giảng viên tạo, cấu hình và quản lý các bài kiểm tra hoặc bài tập; đồng thời thống kê, phân tích kết quả học tập.
3. **Learning Analysis:** Trình bày biểu đồ, báo cáo và thống kê học tập, hỗ trợ học viên tự đánh giá tiến độ và hiệu quả học tập.
4. **Assessment Tracking:** Cung cấp giao diện để học viên thực hiện bài kiểm tra, lưu trữ kết quả và nhận phản hồi từ giảng viên hoặc hệ thống.
5. **Learning Management:** Hỗ trợ học viên đăng ký hoặc hủy khóa học, truy cập tài liệu, theo dõi tiến độ học tập và nhận phản hồi.
6. **Teaching Controller:** Cung cấp API để Web App giao tiếp với các nghiệp vụ giảng dạy, tiếp nhận yêu cầu và trả kết quả.
7. **Teaching Assistant:** Quản lý khóa học, học liệu, danh sách học viên và phản hồi giữa giảng viên – học viên.
8. **Assessment Manager:** Xử lý việc tạo, cấu hình, lưu trữ và tổng hợp kết quả đánh giá.
9. **Learning Controller:** Đóng vai trò trung gian giữa Web App và logic xử lý của hệ thống học tập, đảm bảo giao tiếp thông qua API thống nhất.
10. **Learning Manager:** Quản lý đăng ký, tiến trình học tập và tương tác của học viên với giảng viên.
11. **Learning Analyzer & AI:** Ứng dụng kỹ thuật học máy để phân tích và tổng hợp dữ liệu học tập của từng học viên, từ đó đưa ra gợi ý cá nhân hóa về lộ trình và nội dung học tập. Thành phần này đồng thời thống kê và trực quan hóa kết quả học tập.
12. **Assessment Tracker:** Theo dõi, ghi nhận và lưu trữ kết quả đánh giá, phục vụ phản hồi và phân tích.

3.4 Góc Nhìn Thành Phần - Kết Nối

3.4.1 Sơ Đồ Chính

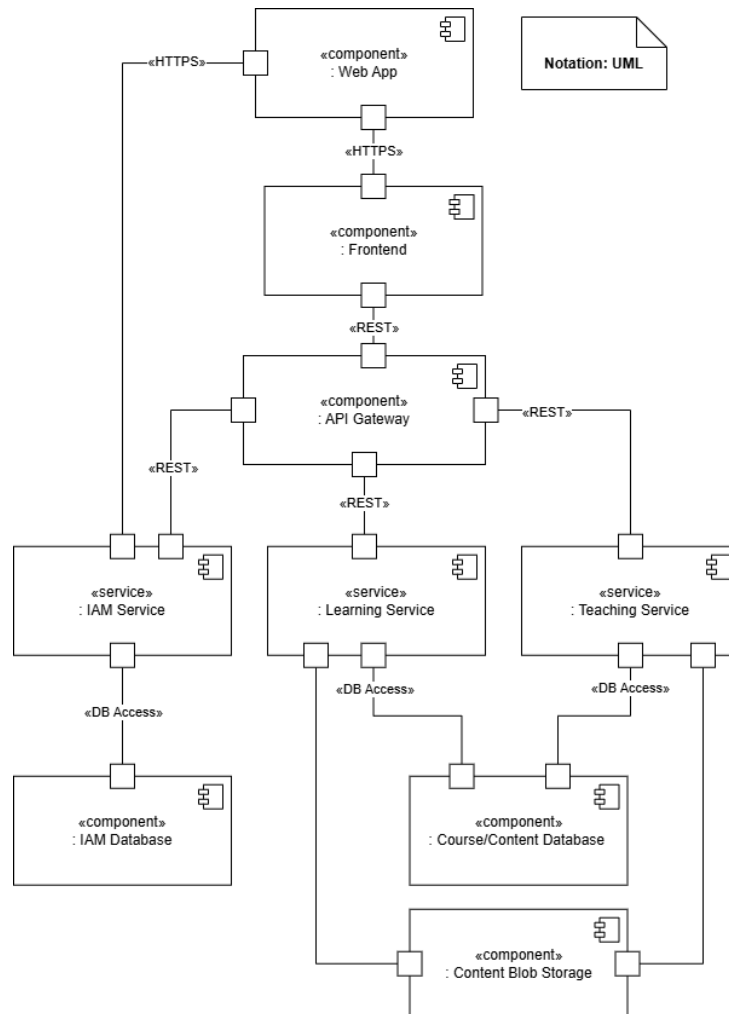


Figure 3.4: Sơ đồ góc nhìn thành phần - kết nối

3.4.2 Danh Sách Thành Phần

- **Web App** Các ứng dụng phía người dùng (Client), cung cấp giao diện cho giảng viên, học viên và quản trị viên.
 - Gửi yêu cầu HTTPS đến Frontend.
 - Thực hiện chuyển hướng người dùng đến IAM Service để đăng nhập.
- **Frontend (Next.js)** Thành phần giao diện đảm nhiệm việc tạo trang trên máy chủ (Server-Side Rendering), sau đó gửi HTML đến trình duyệt.



- Nhận yêu cầu từ người dùng và gửi yêu cầu REST đến API Gateway.
- Thực hiện chuyển hướng người dùng sang IAM Service khi cần xác thực.
- **API Gateway (Nginx)** Thành phần trung tâm định tuyến và bảo vệ backend.
 - Nhận yêu cầu REST từ Frontend.
 - Gửi yêu cầu REST sang Learning Service và Teaching Service.
 - Kết nối tới IAM Service qua REST khi cần xác thực.
- **IAM Service (Keycloak)** Thành phần quản lý định danh và xác thực.
 - Trình duyệt thực hiện chuyển hướng để đăng nhập.
 - API Gateway gửi yêu cầu REST để kiểm tra token hoặc lấy thông tin người dùng.
- **Learning Service (Spring Boot)** Dịch vụ nghiệp vụ cho học viên.
 - Nhận yêu cầu REST từ API Gateway.
 - Truy cập Course/Content Database.
 - Truy xuất học liệu từ Content Blob Storage.
- **Teaching Service (Spring Boot)** Dịch vụ dành cho giảng viên.
 - Nhận yêu cầu REST từ API Gateway.
 - Chia sẻ Course/Content Database với Learning Service.
 - Truy cập nội dung học liệu trên Content Blob Storage.
- **IAM Database (PostgreSQL)** Lưu trữ:
 - Thông tin người dùng,
 - Vai trò và phân quyền,
 - Dữ liệu liên quan đến xác thực.
- **Course/Content Database (PostgreSQL)** Cơ sở dữ liệu quan hệ lưu trữ thông tin khóa học và tiến trình học tập. Learning Service và Teaching Service.
- **Content Blob Storage (MinIO)** Kho lưu trữ các tệp học liệu và nội dung đa phương tiện.

3.4.3 Hướng Dẫn Thay Đổi

- Khi mở rộng chức năng, ưu tiên bổ sung vào dịch vụ hiện có nếu cùng miền nghiệp vụ, tránh tạo mới trừ khi cần xử lý độc lập.
- Chỉ tách thành dịch vụ mới khi cần sự độc lập cao về mở rộng, triển khai hoặc khi phạm vi nghiệp vụ vượt quá khả năng quản lý của dịch vụ hiện tại.

3.4.4 Lý Do Thiết Kế

- API Gateway được sử dụng vì:
 - **Tập trung hoá điểm truy cập:** Mọi yêu cầu từ Frontend đi qua một cổng duy nhất, giúp đơn giản hóa giao tiếp.
 - **Che giấu cấu trúc backend:** Frontend không phụ thuộc vào từng dịch vụ nội bộ.
 - **Tăng cường bảo mật:** Thực hiện kiểm tra token và lọc request trước khi chuyển tiếp đến backend.

(Chi tiết tại phụ lục [ADR-03](#))

- IAM Service được sử dụng vì:
 - Mô hình xác thực tập trung, thống nhất giữa các nhóm dịch vụ,
 - Khả năng quản lý vai trò và phân quyền theo chuẩn OAuth2/OIDC,
 - Giảm tải cho các dịch vụ nghiệp vụ do không phải xử lý logic xác thực và quản lý người dùng.

(Chi tiết tại phụ lục [ADR-02](#))

3.5 Góc Nhìn Triển Khai

3.5.1 Sơ Đồ Chính

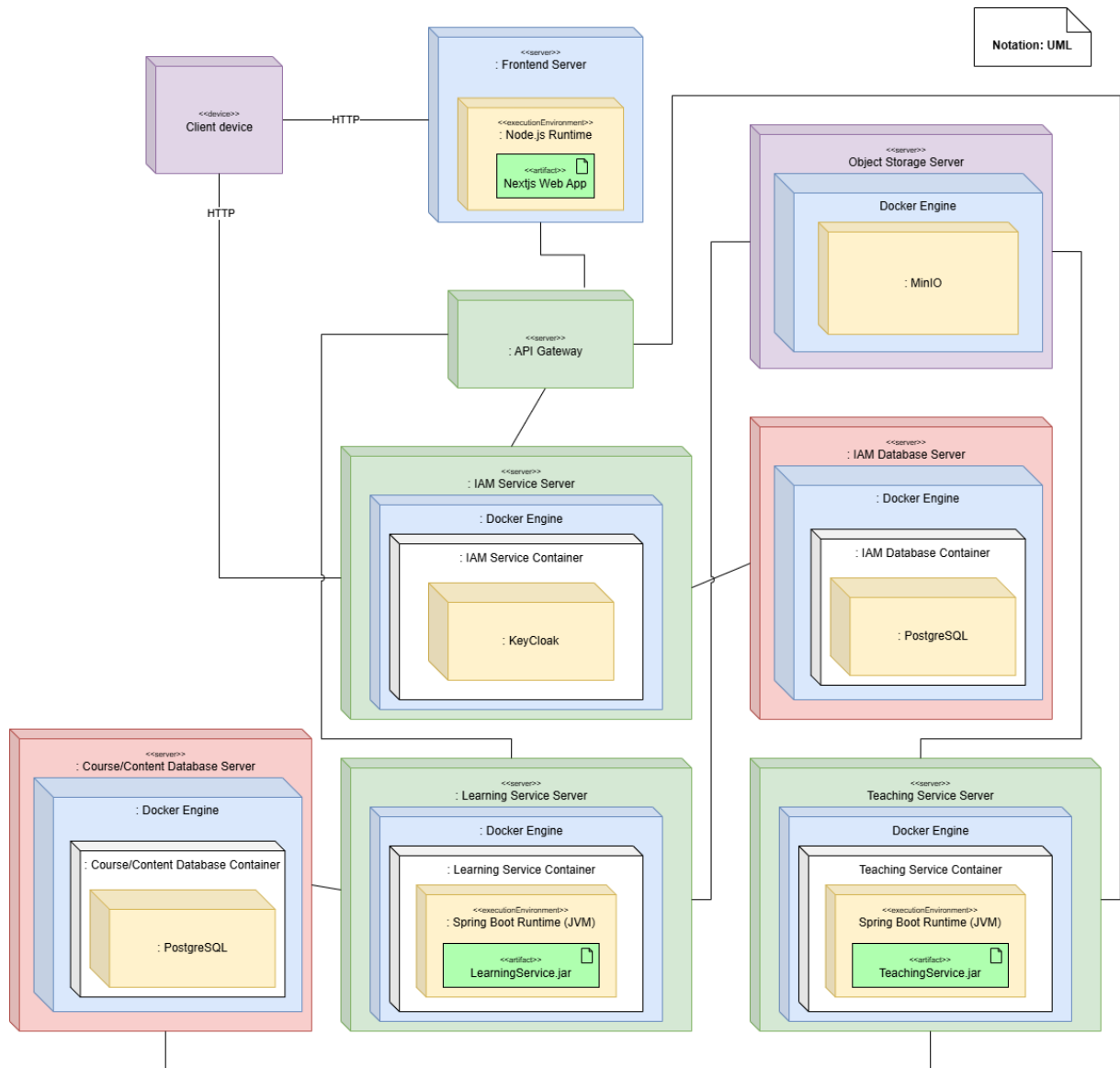


Figure 3.5: Sơ đồ triển khai

3.5.2 Danh Sách Thành Phần

Hệ thống IntelliLearn được triển khai trên nhiều máy chủ, trong đó mỗi thành phần được đóng gói dưới dạng **container** và vận hành bởi **Docker Engine**. Cấu trúc này giúp hệ thống đạt được tính cô lập, dễ quản lý, và thuận tiện cho mở rộng quy mô.

- **Client Device** Thiết bị của người dùng (máy tính, điện thoại), truy cập hệ thống



thông qua giao thức HTTP/HTTPS. Đây là điểm bắt đầu của mọi yêu cầu từ giáo viên và học viên.

- **Frontend Server** Máy chủ giao diện chạy môi trường `Node.js Runtime`, phục vụ ứng dụng `Next.js Web App`. Thành phần này:
 - Tạo giao diện phía máy chủ (Server-Side Rendering) và phân phối tài nguyên tĩnh.
 - Gửi yêu cầu HTTP đến **API Gateway** cho tất cả nghiệp vụ backend.
- **API Gateway** Đóng vai trò điểm vào duy nhất của hệ thống. Thành phần này:
 - Xác thực token người dùng bằng cách kết nối đến **IAM Service**.
 - Định tuyến yêu cầu đến các dịch vụ **Learning**, **Teaching**.
- **IAM Service Server** Máy chủ chạy **Docker Engine** và chứa **IAM Service Container**, trong đó:
 - **Keycloak** đảm nhận quản lý định danh, xác thực người dùng và quản lý phiên đăng nhập.
 - **IAM Service** kết nối trực tiếp tới **IAM Database Server**.
- **IAM Database Server** Máy chủ chứa **PostgreSQL** độc lập để lưu trữ thông tin người dùng, vai trò và phân quyền.
- **Learning Service Server** Máy chủ chạy container **Learning Service**:
 - Chạy **Spring Boot Runtime (JVM)** thực thi tệp `LearningService.jar`.
 - Xử lý nghiệp vụ học tập: đăng ký khóa học, xem học liệu, tiến trình và phân tích kết quả học tập.
 - Kết nối đến **Course/Content Database**.
- **Teaching Service Server** Máy chủ vận hành `TeachingService.jar` trong môi trường **Spring Boot Runtime (JVM)**:
 - Đảm nhận nghiệp vụ giảng dạy: tạo khóa học, quản lý bài giảng, bài tập, đánh giá.
 - Chia sẻ nguồn dữ liệu với **Learning Service** thông qua cùng cơ sở dữ liệu học vụ.



- **Course/Content Database Server** Máy chủ lưu trữ dữ liệu thông tin khóa học, tài nguyên học tập, bài kiểm tra và đánh giá, chứa PostgreSQL bên trong một container riêng.
- **Object Storage Server** Máy chủ lưu trữ nội dung số (video, tài liệu PDF, bài nộp,...), chạy container MinIO.

3.5.3 Hướng Dẫn Thay Đổi

- **Cập nhật dịch vụ:** Việc nâng cấp ứng dụng được thực hiện bằng cách cập nhật Docker Image và khởi động lại container — không làm gián đoạn các dịch vụ khác.
- **Mở rộng hiệu năng:** Tăng tài nguyên (CPU/RAM) cho từng máy chủ hoặc triển khai nhiều instance của cùng dịch vụ.
- **Thay đổi quy tắc định tuyến:** Cập nhật cấu hình của API Gateway khi thêm endpoint hoặc khi dịch vụ được tách nhỏ.



3.6 Phân Công Nhiệm Vụ

Thành viên	MSSV	Nhiệm vụ	Đóng góp (%)
Phạm Gia Bảo	2210254	Phát triển module Teaching Assistant	12.5
Nguyễn Trương Thái Bảo	2210251	Phát triển module Teaching Assistance	12.5
Châu Quốc Bảo	2210195	Phát triển module Learning Manager	12.5
Võ Lê Quang Vinh	2213982	Phát triển module Learning Management	12.5
Phan Đình Khôi Nguyên	2212318	Phát triển góc nhìn Thành Phần - Kết Nối, góc nhìn Triển Khai & Triển khai hệ thống	12.5
Lâm Vũ	2213994	Phát triển tài liệu	12.5
Võ Quang Đại Việt	2213954	Quản lý cơ sở dữ liệu	12.5
Nguyễn Duy Vũ	2213997	Phát triển góc nhìn Module, Class Diagram & Bảo mật hệ thống	12.5

Table 3.1: Bảng phân công nhiệm vụ

3.7 Lý Do Thiết Kế

Trong quá trình thiết kế hệ thống **IntelliLearn**, ba mô hình kiến trúc phù hợp nhất đã được xem xét: **Layered Architecture**, **Service-Based Architecture**, và **Microservices Architecture**. Mỗi mô hình phản ánh cách tổ chức phần mềm khác nhau, phù hợp với các mức độ phức tạp hệ thống và khả năng mở rộng khác nhau. Phần dưới đây mô tả chi tiết từng kiến trúc trước khi phân tích và lựa chọn mô hình phù hợp.

- Kiến trúc Layered:** Đây là mô hình kiến trúc truyền thống và được sử dụng rộng rãi, trong đó hệ thống được phân tách thành các tầng kỹ thuật. Mỗi tầng đảm nhiệm một chức năng riêng biệt, đồng thời quy định rõ ràng phạm vi giao tiếp: tầng phía trên chỉ được tương tác với tầng trực tiếp bên dưới (Closed Layer) hoặc gọi xuống một tầng sâu hơn (Open Layer).
 - **Ưu điểm:** Kiến trúc này có tính đơn giản và dễ triển khai.
 - **Nhược điểm:** Do luồng xử lý bắt buộc phải đi qua các tầng theo thứ tự cố định, các tầng trở nên dễ bị phụ thuộc chặt chẽ. Khi hệ thống mở rộng, mức độ phụ thuộc này làm giảm tính linh hoạt, gây khó khăn cho việc mở rộng từng phần.
- Kiến trúc Microservices:** Microservices là kiến trúc phân tán hiện đại, trong đó hệ thống được chia thành các dịch vụ nhỏ, độc lập hoàn toàn, mỗi dịch vụ có thể được triển khai riêng, mở rộng riêng và có cơ sở dữ liệu riêng.
 - **Ưu điểm:** Kiến trúc này có mức độ linh hoạt và mở rộng vượt trội, cho phép nhiều nhóm phát triển độc lập.
 - **Nhược điểm:** Chi phí xây dựng và vận hành cao do yêu cầu hạ tầng phức tạp.
- Kiến trúc Service-Based:** Kiến trúc Service-Based là kiến trúc phân tán tương tự như Microservices, hệ thống được tổ chức thành các dịch vụ theo miền nghiệp vụ, nhưng không chi tiết như Microservices. Các dịch vụ có mức độ độc lập tương đối và có thể triển khai tách biệt tùy theo nhu cầu.
 - **Ưu điểm:** Kiến trúc này làm giảm độ phức tạp so với Microservices cũng như hỗ trợ mở rộng từng dịch vụ.
 - **Nhược điểm:** Độ độc lập của dịch vụ không hoàn toàn, nguy cơ chia sẻ dữ liệu dẫn đến phụ thuộc, khi số lượng dịch vụ tăng có thể tiến gần độ phức tạp của Microservices.

Bảng sau trình bày sự so sánh khái quát giữa ba mô hình kiến trúc:

Tiêu chí	Layered Architecture	Service-Based Architecture	Microservice Architecture
Cấu trúc tổ chức	Theo tầng kỹ thuật	Theo nhóm nghiệp vụ	Nhiều dịch vụ nhỏ, độc lập hoàn toàn
Độ tách biệt chức năng	Thấp	Trung bình	Cao
Khả năng mở rộng	Hạn chế	Mở rộng theo nhóm dịch vụ	Mở rộng từng dịch vụ độc lập
Khả năng bảo trì	Dễ bắt đầu nhưng khó mở rộng về lâu dài	Dễ bảo trì theo module, giảm phụ thuộc	Dễ bảo trì từng dịch vụ, nhưng phức tạp toàn hệ thống
Độ phức tạp triển khai	Thấp	Trung bình	Cao
Phù hợp quy mô	Nhỏ–trung bình	Trung bình–lớn	Rất lớn, nhiều nhóm phát triển độc lập

Kiến trúc **Service-Based** mang lại sự cân bằng giữa tính tách biệt và tính khả thi triển khai. So với Layered Architecture, kiến trúc này giảm thiểu phụ thuộc ngang và tăng khả năng phát triển song song giữa các nhóm chức năng. So với Microservice Architecture, Service-Based đơn giản hơn về hạ tầng, chi phí vận hành và yêu cầu kỹ thuật, nhưng vẫn đảm bảo khả năng mở rộng và bảo trì tốt cho hệ thống có quy mô trung bình như IntelliLearn. Do đó, lựa chọn kiến trúc Service-Based là phù hợp để đáp ứng nhu cầu phát triển dài hạn trong khi vẫn duy trì sự ổn định và hiệu quả vận hành của hệ thống.

Lưu ý: Lý do lựa chọn kiến trúc Service-Based được phân tích đầy đủ trong hồ sơ quyết định kiến trúc, được trình bày tại phụ lục [ADR-01](#).

4 Thiết Kế Chi Tiết

4.1 Thiết Kế Module

4.1.1 Module Learning Manager

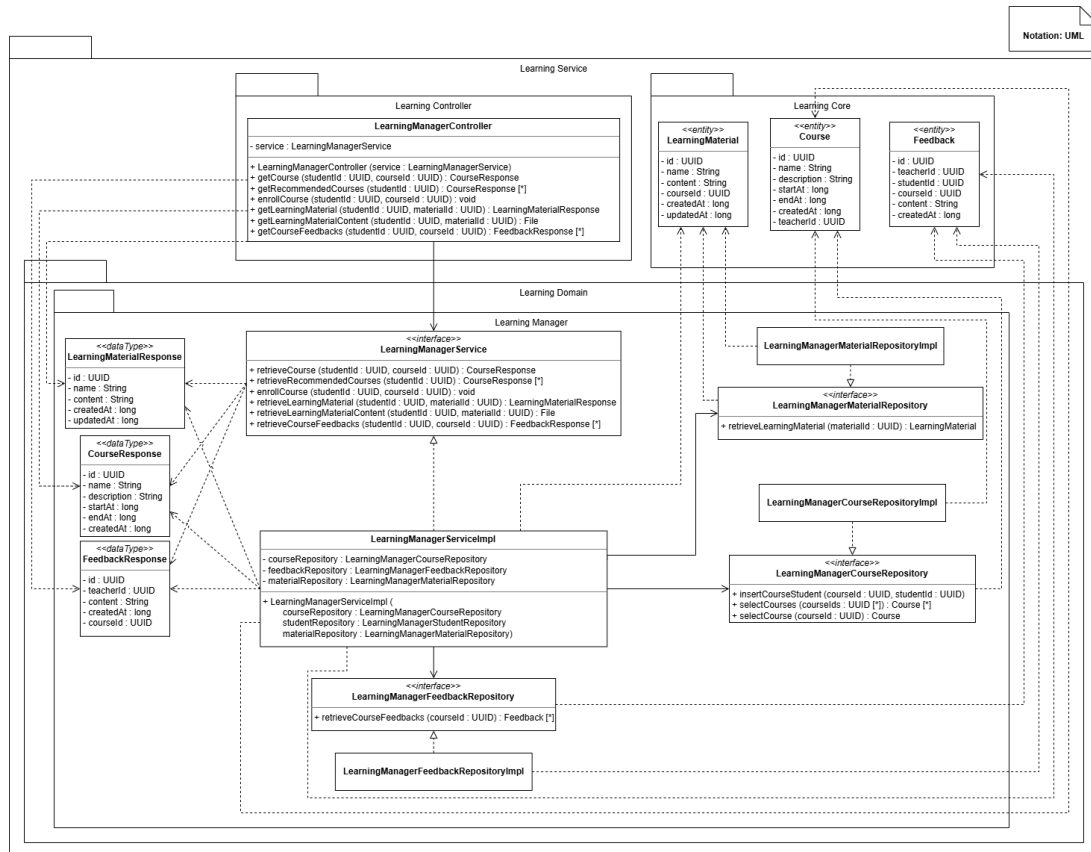


Figure 4.1: Sơ đồ lớp module Learning Manager

Danh sách thành phần:

- **LearningManagerController:** Lớp điều khiển chịu trách nhiệm xử lý các yêu cầu REST liên quan đến hoạt động học tập của học viên, bao gồm đăng ký khóa học, truy xuất tài liệu, gợi ý khóa học và truy xuất phản hồi từ giảng viên.
- **LearningManagerService:** Giao diện định nghĩa các chức năng nghiệp vụ chính của module, như truy xuất thông tin khóa học, tài liệu học tập và phản hồi từ giảng viên.
- **LearningManagerServiceImpl:** Lớp hiện thực giao diện **LearningManagerService**, chịu trách nhiệm thực thi logic nghiệp vụ, điều phối giữa các kho dữ liệu (Repository)

và đảm bảo tính toàn vẹn dữ liệu trong quá trình xử lý.

- **LearningManagerCourseRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu liên quan đến khóa học, như truy vấn, đăng ký học viên và chọn khóa học.
- **LearningManagerCourseRepositoryImpl**: Lớp hiện thực của **LearningManagerCourseRepository**, cung cấp các thao tác cụ thể với dữ liệu khóa học.
- **LearningManagerMaterialRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu tài liệu học tập.
- **LearningManagerMaterialRepositoryImpl**: Lớp hiện thực của **LearningManagerMaterialRepository**, cung cấp các thao tác cụ thể với dữ liệu tài liệu học tập.
- **LearningManagerFeedbackRepository**: Giao diện định nghĩa các phương thức lấy phản hồi của học viên đối với khóa học.
- **LearningManagerFeedbackRepositoryImpl**: Lớp hiện thực của **LearningManagerFeedbackRepository**, thực hiện các thao tác truy xuất phản hồi từ cơ sở dữ liệu.
- **Course**: Thực thể biểu diễn thông tin một khóa học trong hệ thống, bao gồm tên, mô tả, thời gian bắt đầu và kết thúc.
- **LearningMaterial**: Thực thể biểu diễn tài liệu học tập, chứa nội dung, tên tài liệu, thời gian tạo và cập nhật.
- **Feedback**: Thực thể biểu diễn phản hồi từ giảng viên cho học viên trong một khóa học cụ thể, bao gồm nội dung phản hồi, mã khóa học, mã học viên và thời gian tạo.
- **CourseResponse**: Kiểu dữ liệu phản hồi chứa thông tin chi tiết của một khóa học, dùng cho việc hiển thị hoặc truyền tải dữ liệu qua API.
- **LearningMaterialResponse**: Kiểu dữ liệu phản hồi chứa thông tin chi tiết của một tài liệu học tập.
- **FeedbackResponse**: Kiểu dữ liệu phản hồi chứa thông tin phản hồi từ giảng viên cho học viên đối với khóa học.

Áp Dụng Nguyên Tắc SOLID

Thiết kế của Module **Learning Manager** được xây dựng nhằm tuân thủ các nguyên tắc thiết kế SOLID.



1. Single Responsibility Principle (SRP)

- `LearningManagerController`: chịu trách nhiệm tiếp nhận và phản hồi các yêu cầu REST từ học viên; không chứa logic nghiệp vụ.
- `LearningManagerService` / `LearningManagerServiceImpl`: xử lý toàn bộ nghiệp vụ liên quan đến hoạt động học tập của học viên và điều phối các repository cần thiết.

2. Open/Closed Principle (OCP)

- Kiến trúc module sử dụng các abstraction như `LearningManagerService`, `LearningManagerCourseRepository`, `LearningManagerMaterialRepository` và `LearningManagerFeedbackRepository` để tách biệt phần giao diện và phần hiện thực.
- Nhờ đó, có thể bổ sung các lớp hiện thực mới—ví dụ một repository tối ưu hơn hoặc một service mở rộng chức năng—mà không cần sửa đổi các lớp đang dùng abstraction trong sơ đồ, chẳng hạn `LearningManagerServiceImpl`.

3. Liskov Substitution Principle (LSP)

- Mọi implementation của repository/service đều tuân thủ cùng một giao diện, đảm bảo có thể thay thế lẫn nhau mà không phá vỡ hành vi hệ thống.

4. Interface Segregation Principle (ISP)

- Module chia nhỏ interface theo nhu cầu:
 - `LearningManagerCourseRepository` – thao tác khóa học,
 - `LearningManagerMaterialRepository` – thao tác tài liệu,
 - `LearningManagerFeedbackRepository` – thao tác phản hồi.
- Giúp các lớp hiện thực chỉ phải hiện thực các chức năng liên quan trực tiếp.

5. Dependency Inversion Principle (DIP)

- `LearningManagerServiceImpl` phụ thuộc vào abstraction (các giao diện `Repository`), không phụ thuộc implementation cụ thể.

4.1.2 Module Teaching Assistant

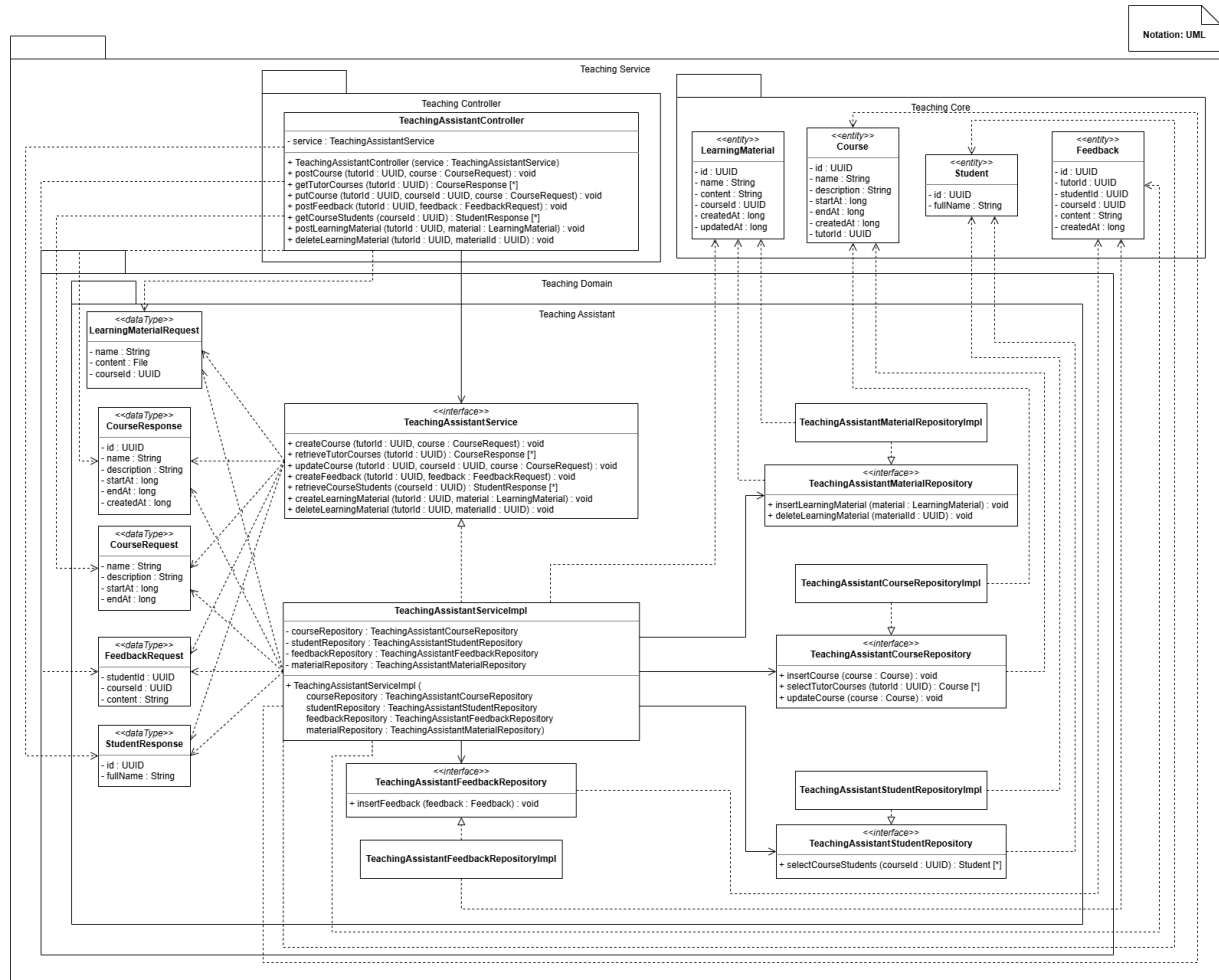


Figure 4.2: Sơ đồ lớp module Teaching Assistant

Danh sách thành phần:

- **TeachingAssistantController**: Lớp điều khiển chịu trách nhiệm xử lý các yêu cầu REST liên quan đến hoạt động giảng dạy của giảng viên, bao gồm quản lý khóa học, tài liệu học tập, danh sách học viên và phản hồi.
- **TeachingAssistantService**: Giao diện định nghĩa các chức năng nghiệp vụ chính của module, như tạo khóa học, quản lý tài liệu, truy xuất danh sách học viên và xử lý phản hồi giữa giảng viên và học viên.
- **TeachingAssistantServiceImpl**: Lớp hiện thực giao diện TeachingAssistantService, chịu trách nhiệm thực thi logic nghiệp vụ, điều phối dữ liệu giữa các kho lưu



trữ và đảm bảo nhất quán trong quá trình cập nhật, truy xuất thông tin khóa học và phản hồi.

- **TeachingAssistantCourseRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu khóa học, bao gồm thêm, cập nhật và truy vấn danh sách khóa học theo giảng viên phụ trách.
- **TeachingAssistantCourseRepositoryImpl**: Lớp hiện thực của TeachingAssistantCourseRepository, thực hiện các thao tác cụ thể với dữ liệu khóa học trong cơ sở dữ liệu.
- **TeachingAssistantMaterialRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu tài liệu học tập, cho phép thêm mới và xóa tài liệu liên quan đến từng khóa học.
- **TeachingAssistantMaterialRepositoryImpl**: Lớp hiện thực của TeachingAssistantMaterialRepository, cung cấp các thao tác cụ thể với dữ liệu tài liệu học tập.
- **TeachingAssistantFeedbackRepository**: Giao diện định nghĩa các phương thức thao tác dữ liệu phản hồi của học viên, cho phép lưu phản hồi từ giảng viên cho học viên đối với từng khóa học.
- **TeachingAssistantFeedbackRepositoryImpl**: Lớp hiện thực của TeachingAssistantFeedbackRepository, thực thi các thao tác thêm phản hồi vào cơ sở dữ liệu.
- **TeachingAssistantStudentRepository**: Giao diện định nghĩa các phương thức truy xuất danh sách học viên thuộc một khóa học.
- **TeachingAssistantStudentRepositoryImpl**: Lớp hiện thực của TeachingAssistantStudentRepository, cung cấp thao tác cụ thể để lấy danh sách học viên tham gia khóa học.
- **Course**: Thực thể biểu diễn thông tin khóa học, bao gồm tên, mô tả, thời gian bắt đầu, thời gian kết thúc và mã định danh của giảng viên phụ trách.
- **LearningMaterial**: Thực thể biểu diễn tài liệu học tập, bao gồm nội dung, mã khóa học và thời gian tạo hoặc cập nhật.
- **Student**: Thực thể biểu diễn thông tin học viên, bao gồm mã định danh và tên đầy đủ.



- **Feedback:** Thực thể biểu diễn phản hồi của giảng viên cho học viên, chứa nội dung phản hồi, mã học viên và thời gian tạo.
- **CourseRequest:** Kiểu dữ liệu yêu cầu chứa thông tin để tạo mới hoặc cập nhật khóa học.
- **CourseResponse:** Kiểu dữ liệu phản hồi chứa thông tin chi tiết của khóa học, phục vụ việc hiển thị hoặc truyền dữ liệu qua API.
- **LearningMaterialRequest:** Kiểu dữ liệu yêu cầu chứa thông tin về tài liệu học tập được thêm hoặc cập nhật.
- **FeedbackRequest:** Kiểu dữ liệu yêu cầu chứa thông tin phản hồi của giảng viên cho học viên.
- **StudentResponse:** Kiểu dữ liệu phản hồi chứa thông tin học viên trong một khóa học cụ thể.

Áp Dụng Nguyên Tắc SOLID

Thiết kế của Module **Teaching Assistant** được xây dựng nhằm tuân thủ các nguyên tắc thiết kế SOLID.

1. Single Responsibility Principle (SRP)

- **TeachingAssistantController:** chịu trách nhiệm tiếp nhận và phản hồi các yêu cầu REST từ giảng viên; không chứa logic nghiệp vụ.
- **TeachingAssistantService / TeachingAssistantServiceImpl:** xử lý các nghiệp vụ liên quan đến hoạt động giảng dạy (tạo khóa học, cập nhật tài liệu, quản lý học viên, phản hồi).

2. Open/Closed Principle (OCP)

- Module sử dụng các abstraction như **TeachingAssistantService**, **TeachingAssistantCourseRepository**, **TeachingAssistantMaterialRepository**, **TeachingAssistantFeedbackRepository** và **TeachingAssistantStudentRepository** để tách biệt phần giao diện với phần hiện thực.
- Nhờ đó, có thể thay thế hoặc mở rộng các lớp hiện thực—ví dụ như một repository tối ưu cho truy vấn khóa học—mà không cần sửa các lớp phụ thuộc như **TeachingAssistantServiceImpl**.



3. Liskov Substitution Principle (LSP)

- Mọi lớp `RepositoryImpl` trong sơ đồ đều tuân thủ đúng giao diện tương ứng, giúp có thể thay thế implementation mà không làm thay đổi `TeachingAssistantServiceImpl`.

4. Interface Segregation Principle (ISP)

- Module tách interface thành các nhóm nhỏ dựa trên nghiệp vụ:
 - `TeachingAssistantCourseRepository` – thao tác khóa học,
 - `TeachingAssistantMaterialRepository` – thao tác tài liệu,
 - `TeachingAssistantFeedbackRepository` – thao tác phản hồi,
 - `TeachingAssistantStudentRepository` – truy xuất danh sách học viên.
- Điều này giúp các lớp hiện thực chỉ cần hiện thực các chức năng mà chúng thực sự cần.

5. Dependency Inversion Principle (DIP)

- `TeachingAssistantServiceImpl` phụ thuộc vào abstraction (các interface repository), không phụ thuộc các lớp `RepositoryImpl`.

4.1.3 Module Assessment Tracker

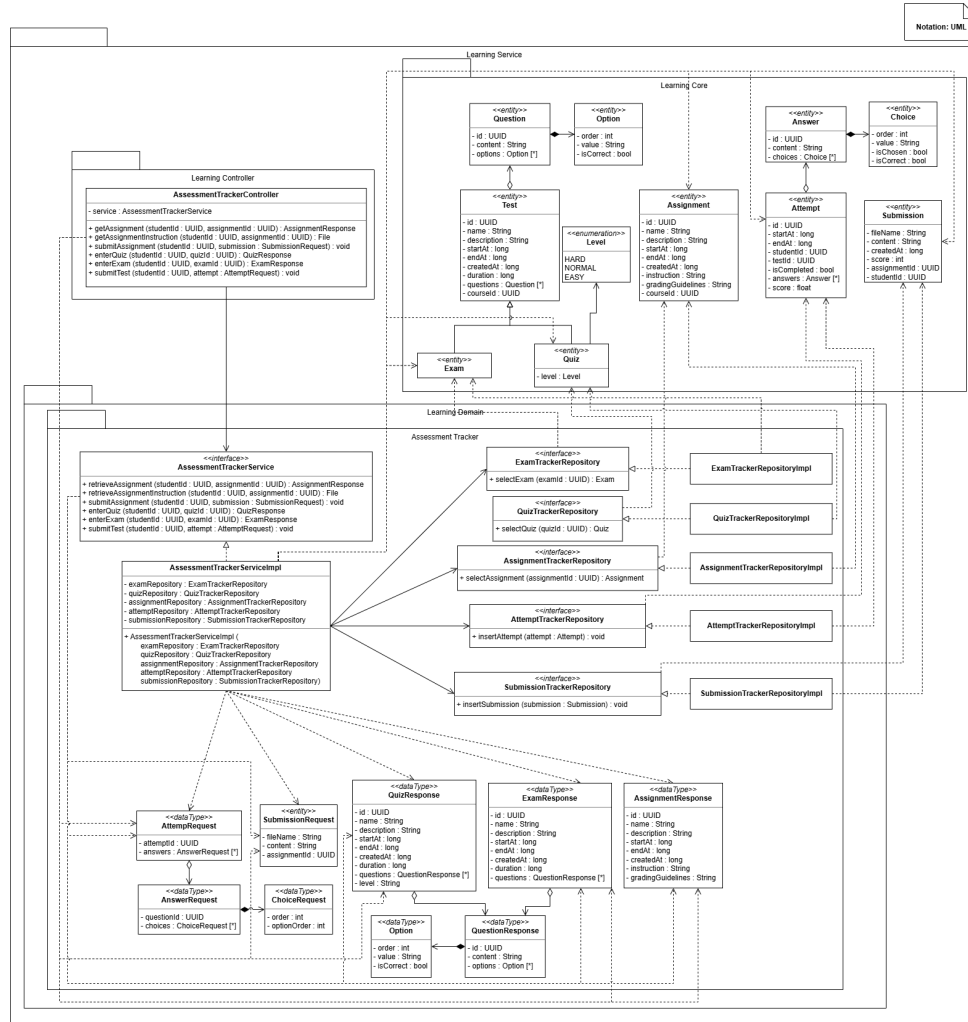


Figure 4.3: Sơ đồ lớp module Assessment Tracker

Các lớp chính:

- **AssessmentTrackerController**: Lớp điều khiển chịu trách nhiệm xử lý các yêu cầu REST liên quan đến việc truy xuất, nộp và theo dõi kết quả đánh giá của học viên, bao gồm bài tập, bài kiểm tra và bài thi.
- **AssessmentTrackerService**: Giao diện định nghĩa các chức năng nghiệp vụ cốt lõi của module, như lấy thông tin bài đánh giá, ghi nhận kết quả làm bài và truy xuất lịch sử nộp bài.
- **AssessmentTrackerServiceImpl**: Lớp hiện thực của **AssessmentTrackerService**,

chịu trách nhiệm triển khai logic nghiệp vụ, điều phối giữa các kho dữ liệu (Repository) và đảm bảo tính toàn vẹn của kết quả đánh giá.

- **ExamTrackerRepository**: Giao diện định nghĩa các phương thức thao tác với dữ liệu bài thi, bao gồm truy xuất, lưu trữ và cập nhật thông tin kỳ thi.
- **ExamTrackerRepositoryImpl**: Lớp hiện thực của ExamTrackerRepository, cung cấp các thao tác cụ thể với dữ liệu bài thi trong cơ sở dữ liệu.
- **QuizTrackerRepository**: Giao diện định nghĩa các phương thức thao tác với dữ liệu bài kiểm tra ngắn, cho phép truy xuất và lưu kết quả làm bài.
- **QuizTrackerRepositoryImpl**: Lớp hiện thực của QuizTrackerRepository, chịu trách nhiệm thực thi các truy vấn liên quan đến bài kiểm tra.
- **AssignmentTrackerRepository**: Giao diện định nghĩa các phương thức thao tác với dữ liệu bài tập, bao gồm truy vấn, cập nhật và lấy thông tin chấm điểm.
- **AssignmentTrackerRepositoryImpl**: Lớp hiện thực của AssignmentTrackerRepository, cung cấp các thao tác lưu trữ và cập nhật thông tin bài tập trong hệ thống.
- **AttemptTrackerRepository**: Giao diện định nghĩa các phương thức ghi nhận và truy xuất thông tin các lần làm bài của học viên.
- **AttemptTrackerRepositoryImpl**: Lớp hiện thực của AttemptTrackerRepository, chịu trách nhiệm lưu trữ chi tiết các lượt làm bài và kết quả tương ứng.
- **SubmissionTrackerRepository**: Giao diện định nghĩa các phương thức lưu trữ và quản lý bài nộp của học viên, bao gồm tệp đính kèm và thông tin chấm điểm.
- **SubmissionTrackerRepositoryImpl**: Lớp hiện thực của SubmissionTrackerRepository, cung cấp thao tác ghi nhận và truy xuất bài nộp từ cơ sở dữ liệu.
- **Assignment, Quiz, Exam**: Các thực thể biểu diễn các hình thức đánh giá khác nhau, bao gồm thông tin về tiêu đề, mô tả, mức độ, hướng dẫn và tập câu hỏi.
- **Question, Option, Answer, Choice**: Các thực thể mô tả cấu trúc câu hỏi, tùy chọn trả lời và lựa chọn của học viên, hỗ trợ chấm điểm tự động và phân tích kết quả.
- **Attempt và Submission**: Thực thể lưu trữ thông tin về quá trình làm bài và bài nộp cuối cùng của học viên, bao gồm thời gian, điểm số và mã định danh.



- **AssignmentResponse**, **ExamResponse**, **QuizResponse**: Kiểu dữ liệu phản hồi chứa thông tin chi tiết về bài đánh giá, bao gồm nội dung câu hỏi, lựa chọn và hướng dẫn chấm điểm.
- **QuestionResponse** và **ChoiceResponse**: Kiểu dữ liệu phản hồi mô tả chi tiết từng câu hỏi và các lựa chọn của học viên trong quá trình làm bài.

Áp Dụng Nguyên Tắc SOLID

Thiết kế của Module **Assessment Tracker** tuân thủ chặt chẽ các nguyên tắc SOLID.

1. Single Responsibility Principle (SRP)

- **AssessmentTrackerController**: chịu trách nhiệm tiếp nhận yêu cầu REST từ học viên và trả về dữ liệu đánh giá; không chứa logic nghiệp vụ.
- **AssessmentTrackerService** / **AssessmentTrackerServiceImpl**: quản lý toàn bộ nghiệp vụ đánh giá, bao gồm truy xuất bài đánh giá, ghi nhận lần làm bài (**Attempt**) và ghi nhận bài nộp (**Submission**); đồng thời điều phối các repository liên quan.

2. Open/Closed Principle (OCP)

- Module được xây dựng xoay quanh các abstraction như **AssessmentTrackerService**, **ExamTrackerRepository**, **QuizTrackerRepository**, **AssignmentTrackerRepository**, **AttemptTrackerRepository**, **SubmissionTrackerRepository**.
- Nhờ cấu trúc này, các lớp implement—ví dụ như một repository—mà không cần sửa các lớp phụ thuộc như **AssessmentTrackerServiceImpl**.

3. Liskov Substitution Principle (LSP)

- Mọi implementation cho các repository đều có thể thay thế nhau trong **AssessmentTrackerServiceImpl**.

4. Interface Segregation Principle (ISP)

- Module tách từng nhóm nghiệp vụ đánh giá thành các interface nhỏ, độc lập:
 - **ExamTrackerRepository** – thao tác bài thi,
 - **QuizTrackerRepository** – thao tác bài kiểm tra,
 - **AssignmentTrackerRepository** – thao tác bài tập,
 - **AttemptTrackerRepository** – thao tác lượt làm bài,



– `SubmissionTrackerRepository` – thao tác bài nộp.

- Điều này giúp các lớp hiện thực chỉ cần quan tâm đến kiểu dữ liệu mình xử lý, không bị buộc hiện thực các phương thức không liên quan.

5. Dependency Inversion Principle (DIP)

- `AssessmentTrackerServiceImpl` chỉ phụ thuộc vào abstraction (các interface repository), không phụ thuộc vào các lớp cụ thể như `QuizTrackerRepositoryImpl` hay `AttemptTrackerRepositoryImpl`.

4.1.4 Module Assessment Manager

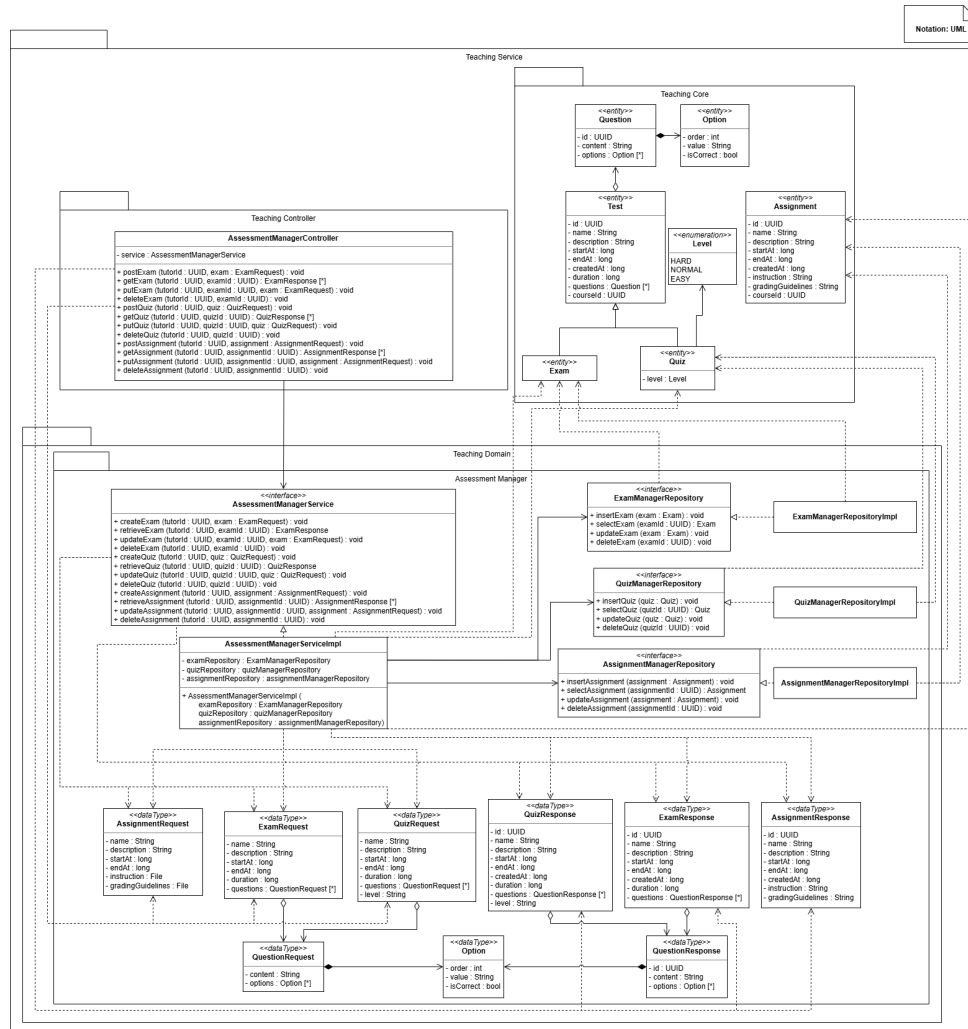


Figure 4.4: Sơ đồ lớp module Assessment Manager

Các lớp chính:

- **AssessmentManagerController**: Lớp điều khiển chịu trách nhiệm xử lý các yêu cầu REST liên quan đến việc tạo, chỉnh sửa, truy xuất và xóa các bài đánh giá như bài thi, bài kiểm tra và bài tập.
- **AssessmentManagerService**: Giao diện định nghĩa các chức năng nghiệp vụ chính của module, bao gồm tạo mới, cập nhật, truy xuất và xóa các bài đánh giá.
- **AssessmentManagerServiceImpl**: Lớp hiện thực của AssessmentManagerService, chịu trách nhiệm triển khai logic nghiệp vụ và điều phối giữa các kho dữ liệu (Repository) để đảm bảo tính toàn vẹn thông tin của bài đánh giá.



- **ExamManagerRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu bài thi, bao gồm thêm mới, chỉnh sửa, truy xuất và xóa thông tin bài thi.
- **ExamManagerRepositoryImpl**: Lớp hiện thực của ExamManagerRepository, thực hiện các thao tác lưu trữ và truy vấn dữ liệu bài thi trong hệ thống.
- **QuizManagerRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu bài kiểm tra ngắn, cho phép thêm, sửa, xóa và truy xuất thông tin.
- **QuizManagerRepositoryImpl**: Lớp hiện thực của QuizManagerRepository, cung cấp các thao tác cụ thể với dữ liệu bài kiểm tra.
- **AssignmentManagerRepository**: Giao diện định nghĩa các phương thức thao tác với cơ sở dữ liệu bài tập, bao gồm lưu trữ, truy xuất và cập nhật hướng dẫn chấm điểm.
- **AssignmentManagerRepositoryImpl**: Lớp hiện thực của AssignmentManagerRepository, cung cấp các thao tác cụ thể để quản lý dữ liệu bài tập.
- **Exam, Quiz, Assignment**: Các thực thể biểu diễn các loại hình bài đánh giá, bao gồm thông tin về tên, mô tả, độ khó, thời gian bắt đầu – kết thúc, hướng dẫn chấm điểm và danh sách câu hỏi.
- **Question và Option**: Các thực thể mô tả cấu trúc câu hỏi và các tùy chọn trả lời, hỗ trợ nhiều dạng câu hỏi khác nhau.
- **ExamRequest, QuizRequest, AssignmentRequest**: Kiểu dữ liệu yêu cầu dùng để gửi thông tin khi tạo hoặc chỉnh sửa bài đánh giá.
- **ExamResponse, QuizResponse, AssignmentResponse**: Kiểu dữ liệu phản hồi chứa thông tin chi tiết của từng loại bài đánh giá, phục vụ cho việc hiển thị hoặc truyền dữ liệu qua API.
- **QuestionRequest và QuestionResponse**: Các kiểu dữ liệu trung gian dùng để mô tả và phản hồi thông tin chi tiết của câu hỏi và các tùy chọn.

Áp dụng nguyên tắc SOLID Thiết kế của Module **Assessment Manager** tuân thủ các nguyên tắc SOLID.

1. Single Responsibility Principle (SRP)



- **AssessmentManagerController**: tiếp nhận và phản hồi các yêu cầu REST liên quan đến quản lý bài kiểm tra từ giảng viên; không chứa logic nghiệp vụ.
- **AssessmentManagerService / AssessmentManagerServiceImpl**: chịu trách nhiệm logic nghiệp vụ cho việc tạo, cập nhật và quản lý bài kiểm tra; điều phối các repository tương ứng.

2. Open/Closed Principle (OCP)

- Module phân tách rõ abstraction qua các interface: **AssessmentManagerService**, **ExamManagerRepository**, **QuizManagerRepository**, **AssignmentManagerRepository**.
- Khi cần mở rộng (ví dụ thêm hỗ trợ loại câu hỏi mới, caching, audit hoặc một storage tối ưu hơn), ta chỉ bổ sung implementation mới (**NewExamRepositoryImpl**, **CachedQuizRepositoryImpl**) mà không cần thay đổi logic trong **AssessmentManagerService** hoặc controller.

3. Liskov Substitution Principle (LSP)

- Mọi lớp **RepositoryImpl** tuân thủ contract của interface tương ứng, do đó có thể thay thế lẫn nhau trong service.
- Ví dụ, **ExamManagerRepositoryImpl** có thể được thay bằng một implementation khác mà **AssessmentManagerServiceImpl** không cần biết chi tiết.

4. Interface Segregation Principle (ISP)

- Module chia nhỏ interface theo trách nhiệm cụ thể:
 - **ExamManagerRepository** — thao tác CRUD cho **Exam**,
 - **QuizManagerRepository** — thao tác cho **Quiz**,
 - **AssignmentManagerRepository** — thao tác cho **Assignment**.

5. Dependency Inversion Principle (DIP)

- **AssessmentManagerServiceImpl** phụ thuộc vào các abstraction (các interface repository) thay vì các lớp hiện thực cụ thể.

4.1.5 Module Learning Analyzer

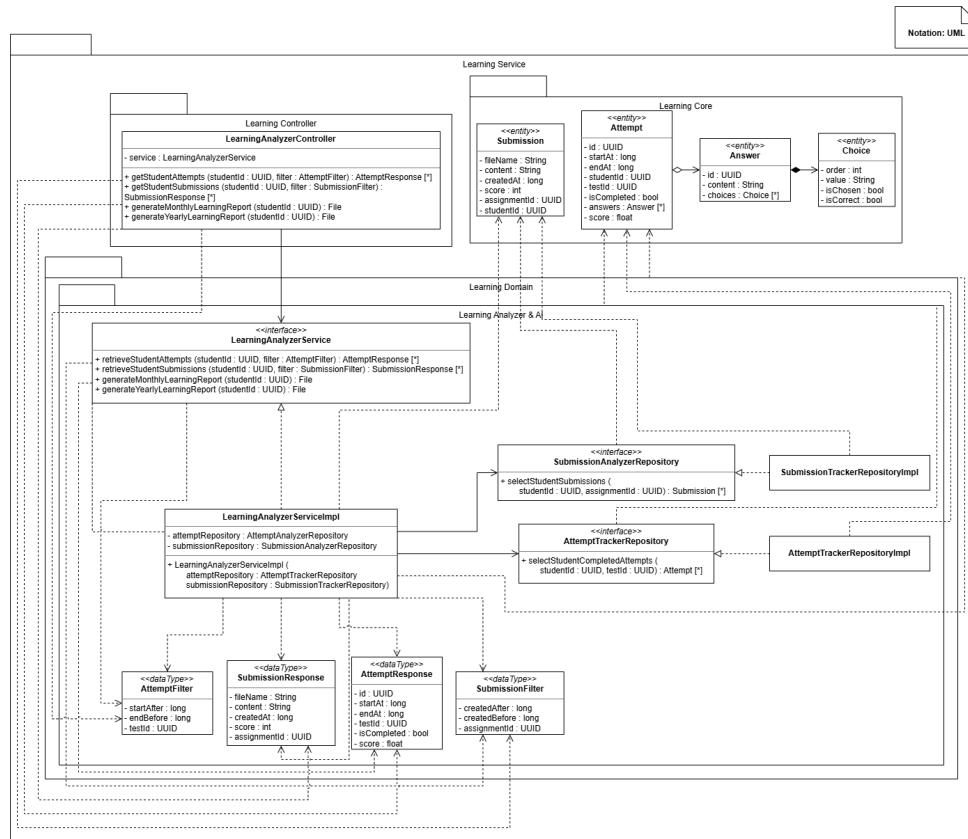


Figure 4.5: Sơ đồ lớp module Learning Analyzer

Các lớp chính:

- **LearningAnalyzerController:** Lớp điều khiển chịu trách nhiệm xử lý các yêu cầu REST liên quan đến việc phân tích dữ liệu học tập của học viên, bao gồm truy xuất lịch sử làm bài, kết quả nộp bài và tạo báo cáo học tập định kỳ.
- **LearningAnalyzerService:** Giao diện định nghĩa các chức năng nghiệp vụ chính của module, như truy xuất dữ liệu học tập, thống kê kết quả học viên và sinh báo cáo tổng hợp.
- **LearningAnalyzerServiceImpl:** Lớp hiện thực của LearningAnalyzerService, chịu trách nhiệm triển khai logic phân tích, kết nối các kho dữ liệu và thực thi quá trình tổng hợp, đánh giá hiệu suất học tập.
- **AttemptAnalyzerRepository:** Giao diện định nghĩa các phương thức thao tác với dữ liệu về các lần làm bài của học viên, bao gồm truy vấn kết quả và lọc theo bài kiểm tra.

- **AttemptTrackerRepositoryImpl**: Lớp hiện thực của **AttemptAnalyzerRepository**, cung cấp các thao tác cụ thể để truy xuất dữ liệu từ hệ thống lưu trữ kết quả làm bài.
- **SubmissionAnalyzerRepository**: Giao diện định nghĩa các phương thức thao tác với dữ liệu bài nộp, cho phép truy vấn các bài nộp của học viên theo tiêu chí tìm kiếm nhất định.
- **SubmissionTrackerRepositoryImpl**: Lớp hiện thực của **SubmissionAnalyzerRepository**, chịu trách nhiệm thực thi các truy vấn dữ liệu liên quan đến bài nộp học tập.
- **Attempt**: Thực thể biểu diễn thông tin một lần làm bài của học viên, bao gồm mã học viên, bài kiểm tra, thời gian, trạng thái hoàn thành và điểm số.
- **Submission**: Thực thể biểu diễn bài nộp của học viên cho một bài tập cụ thể, bao gồm tệp nộp, điểm và thời gian nộp.
- **Answer** và **Choice**: Các thực thể mô tả câu trả lời và lựa chọn của học viên trong một bài làm, hỗ trợ phân tích chi tiết mức độ hiểu và kỹ năng.
- **AttemptFilter** và **SubmissionFilter**: Kiểu dữ liệu lọc dùng để giới hạn phạm vi truy xuất dữ liệu học tập theo khoảng thời gian, mã bài kiểm tra hoặc bài tập cụ thể.
- **AttemptResponse** và **SubmissionResponse**: Kiểu dữ liệu phản hồi chứa thông tin chi tiết về kết quả làm bài và bài nộp, dùng cho hiển thị báo cáo hoặc phân tích trong giao diện người dùng.

Áp dụng nguyên tắc SOLID Thiết kế của **Module Learning Analyzer** tuân thủ các nguyên tắc SOLID.

1. Single Responsibility Principle (SRP)

- **LearningAnalyzerController**: tiếp nhận và phản hồi các yêu cầu REST từ học viên liên quan đến phân tích học tập (lấy lịch sử lượt làm bài, nộp bài, sinh báo cáo); không chứa logic phân tích.
- **LearningAnalyzerService** / **LearningAnalyzerServiceImpl**: chịu trách nhiệm logic phân tích và tổng hợp dữ liệu cho học viên (thực hiện truy vấn, tính toán chỉ số, tạo báo cáo); điều phối các repository cần thiết.

2. Open/Closed Principle (OCP)



- Module tách rõ abstraction qua các interface: `LearningAnalyzerService`, `AttemptAnalyzerRepository`, `SubmissionAnalyzerRepository`.
- Khi cần mở rộng (ví dụ bổ sung thuật toán phân tích mới, thêm chỉ số báo cáo hoặc tối ưu truy vấn), có thể thêm implementation mới mà không thay đổi controller hay service hiện tại.

3. Liskov Substitution Principle (LSP)

- Mọi lớp `AnalyzerRepositoryImpl` tuân thủ contract của interface tương ứng, do đó có thể thay thế lẫn nhau trong `LearningAnalyzerServiceImpl`.

4. Interface Segregation Principle (ISP)

- Module chia nhỏ interface theo trách nhiệm phân tích:
 - `AttemptAnalyzerRepository` — truy vấn lượt làm bài.
 - `SubmissionAnalyzerRepository` — truy vấn bài nộp.
- Điều này giúp các lớp hiện thực chỉ cần hiện thực các phương thức liên quan.

5. Dependency Inversion Principle (DIP)

- `LearningAnalyzerServiceImpl` phụ thuộc vào abstraction (các interface repository), không phụ thuộc vào các lớp hiện thực như `AttemptTrackerRepositoryImpl` hay `SubmissionTrackerRepositoryImpl`.

4.2 Thiết Kế Hành Vi

Dưới đây trình bày một số hoạt động nghiệp vụ chính của hệ thống dưới dạng sơ đồ hoạt động. Chi tiết nghiệp vụ được mô tả đầy đủ trong bảng Use Case tương ứng ở phần [Mô Hình Hóa Hệ Thống](#)

1. Xem khóa học được đề xuất

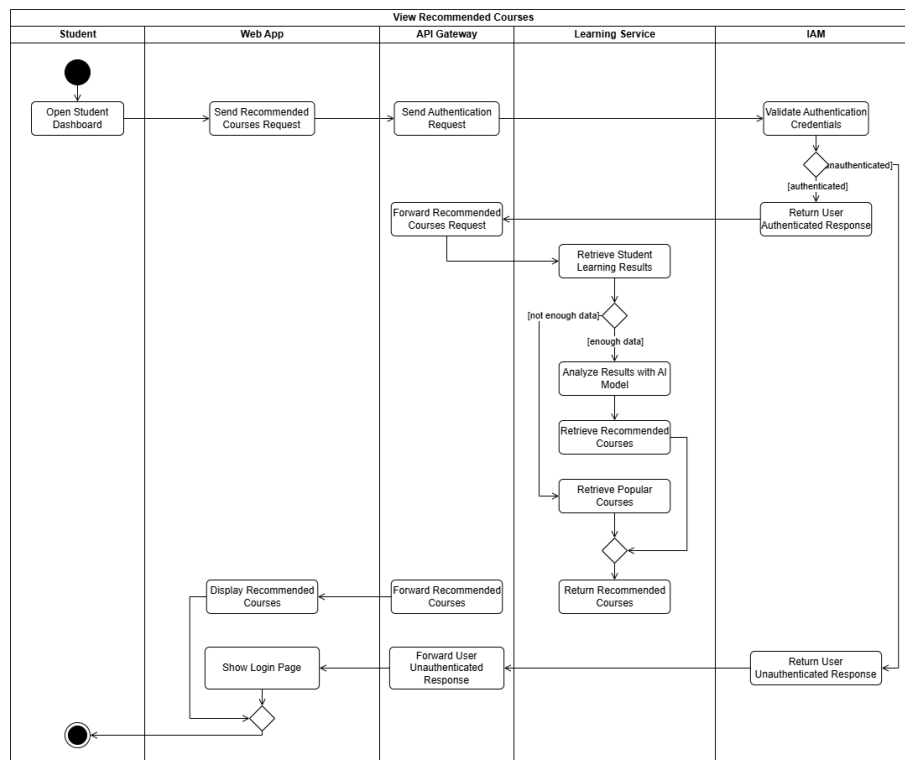


Figure 4.6: Sơ đồ hoạt động use case View Recommended Courses

(Tham khảo Use Case: [View Recommended Courses](#))

- Học viên mở Student Dashboard và khởi tạo yêu cầu xem các khóa học được gợi ý.
- Web App gửi yêu cầu đến API Gateway để xử lý xác thực và định tuyến.
- API Gateway gửi yêu cầu xác thực đến IAM.
- Nếu xác thực không thành công, hệ thống trả phản hồi không hợp lệ và chuyển hướng học viên đến trang đăng nhập.
- Nếu xác thực thành công, API Gateway chuyển tiếp yêu cầu đến Learning Service.



- Learning Service truy xuất dữ liệu học tập của học viên và đánh giá mức độ đầy đủ của thông tin hiện có.
- Hệ thống xử lý theo hai trường hợp:
 - Dữ liệu đầy đủ: hệ thống kích hoạt mô hình AI để phân tích và truy xuất danh sách khóa học được cá nhân hóa.
 - Dữ liệu chưa đủ: hệ thống tạo danh sách các khóa học phổ biến hoặc các khóa phù hợp cho người mới.
- Learning Service trả về danh sách khóa học đề xuất cho API Gateway.
- API Gateway chuyển tiếp kết quả đến Web App.
- Web App hiển thị danh sách khóa học gợi ý cho học viên.

2. Đăng ký khóa học

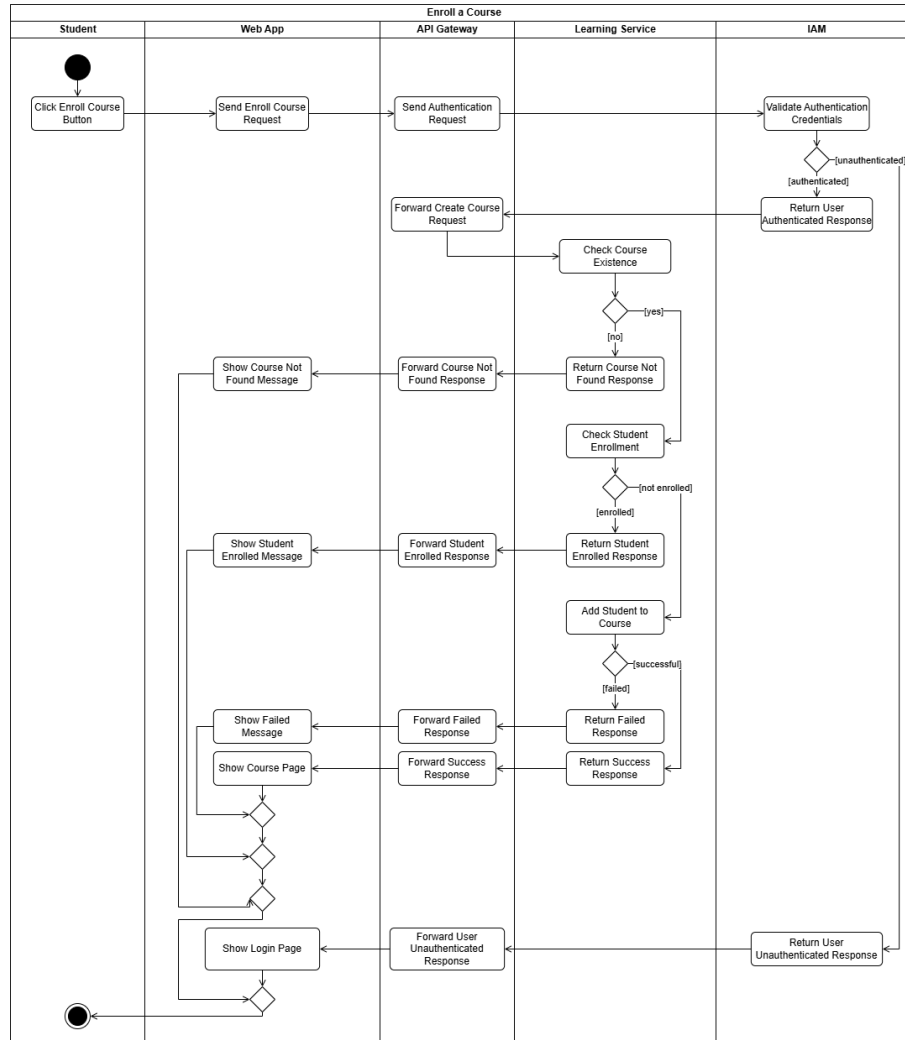


Figure 4.7: Sơ đồ hoạt động use case Enroll a Course

(Tham khảo Use Case: [Enroll a Course](#))

- Học viên gửi yêu cầu đăng ký khóa học qua giao diện web.
- API Gateway tiếp nhận, xác thực người dùng thông qua IAM.
- Learning Service kiểm tra:
 - Khóa học có tồn tại hay không,
 - Học viên đã đăng ký trước đó chưa.
- Nếu hợp lệ, hệ thống thực hiện ghi nhận việc đăng ký vào cơ sở dữ liệu khóa học. Kết quả (thành công hoặc thất bại) được trả về qua API Gateway và hiển thị trên giao diện.

3. Thực hiện bài kiểm tra/thi

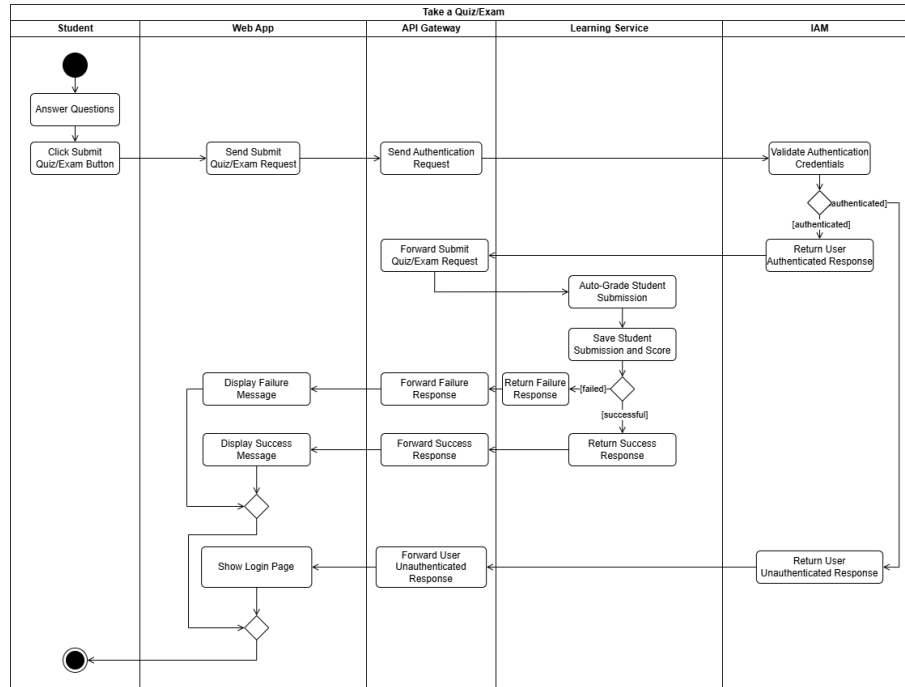


Figure 4.8: Sơ đồ hoạt động use case Take a Quiz/Exam

(Tham khảo Use Case: [Take a Quiz/Exam](#))

- Học viên trả lời các câu hỏi và gửi yêu cầu nộp bài qua Web App.
- Web App chuyển yêu cầu đến API Gateway để xác thực và định tuyến.
- API Gateway gửi yêu cầu xác thực đến IAM:
 - Nếu không được xác thực, hệ thống trả phản hồi không hợp lệ và Web App chuyển hướng học viên đến trang đăng nhập.
 - Nếu xác thực hợp lệ, yêu cầu được chuyển tiếp đến Learning Service.
- Learning Service tự động chấm bài dựa trên câu trả lời học viên.
- Hệ thống lưu kết quả bài làm và điểm số:
 - Lưu thất bại: hệ thống trả phản hồi lỗi và Web App hiển thị thông báo thất bại.
 - Lưu thành công: hệ thống trả kết quả chấm điểm và Web App hiển thị thông báo thành công.

4. Nộp bài tập

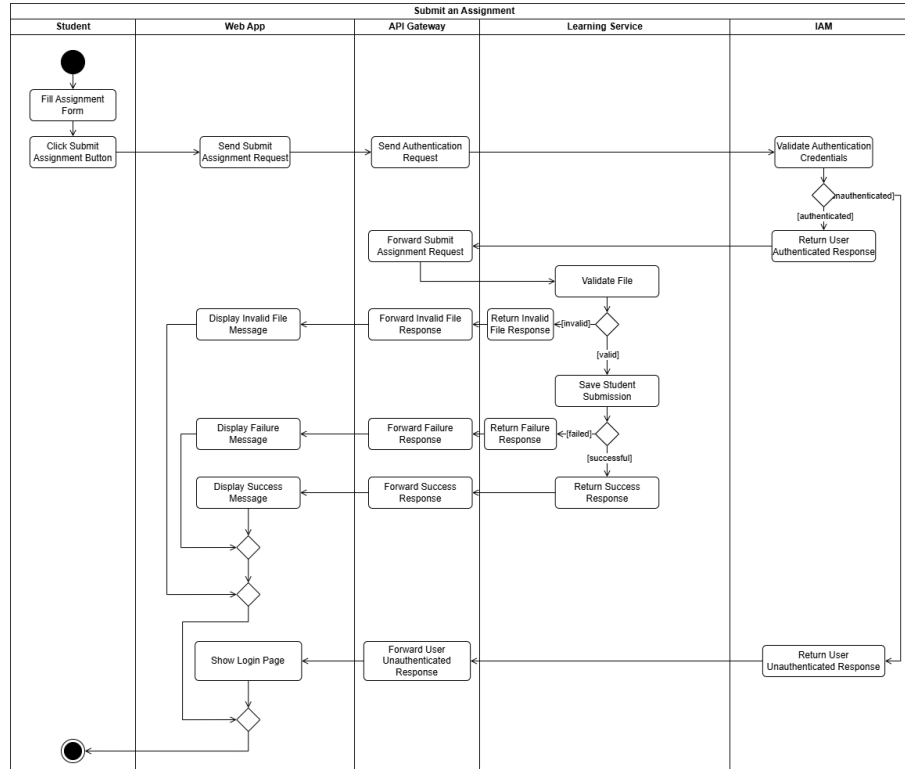


Figure 4.9: Sơ đồ hoạt động use case Submit an Assignment

(Tham khảo Use Case: [Submit an Assignment](#))

- Học viên điền thông tin và tệp bài làm, sau đó gửi yêu cầu nộp bài từ giao diện Web App.
- Web App chuyển yêu cầu đến API Gateway để xác thực và định tuyến.
- API Gateway gửi yêu cầu xác thực đến IAM.
- Nếu xác thực không hợp lệ, hệ thống trả về phản hồi không được cấp quyền và Web App chuyển hướng học viên đến trang đăng nhập.
- Nếu xác thực hợp lệ, API Gateway chuyển tiếp yêu cầu nộp bài đến Learning Service.
- Learning Service kiểm tra tính hợp lệ của tệp bài nộp (định dạng, kích thước).
- Hệ thống xử lý theo hai nhánh:
 - Tệp không hợp lệ: Learning Service trả về thông báo lỗi và Web App hiển thị thông báo tệp không hợp lệ.



- Tập hợp lệ: Learning Service lưu bài nộp của học viên vào hệ thống.
- Sau khi lưu, hệ thống xác định kết quả xử lý:
 - Lưu thất bại: Web App hiển thị thông báo lỗi.
 - Lưu thành công: Web App hiển thị thông báo nộp bài thành công.

5. Xem báo cáo phân tích học tập

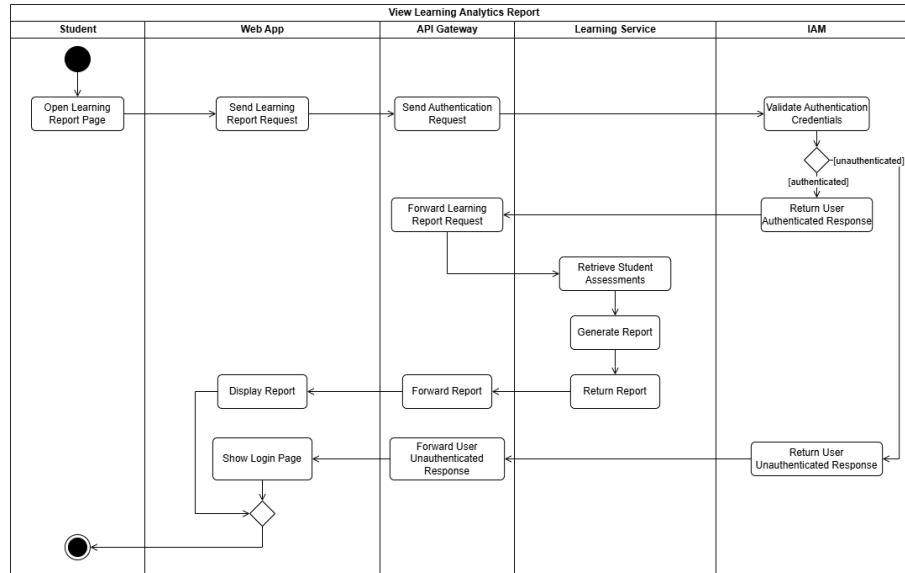


Figure 4.10: Sơ đồ hoạt động use case View Learning Analytics Report

(Tham khảo Use Case: [View Learning Analytics Report](#))

- Học viên gửi yêu cầu xem báo cáo học tập theo tháng hoặc theo năm.
- API Gateway xác thực người dùng thông qua IAM.
- Learning Analyzer Service:
 - Truy xuất dữ liệu các lần làm bài, bài nộp và điểm số,
 - Tổng hợp dữ liệu theo bộ lọc (thời gian, môn học),
 - Trả về báo cáo tổng hợp và phân tích kết quả học tập.
- API Gateway chuyển kết quả về cho giao diện để hiển thị báo cáo.

6. Tạo khóa học mới

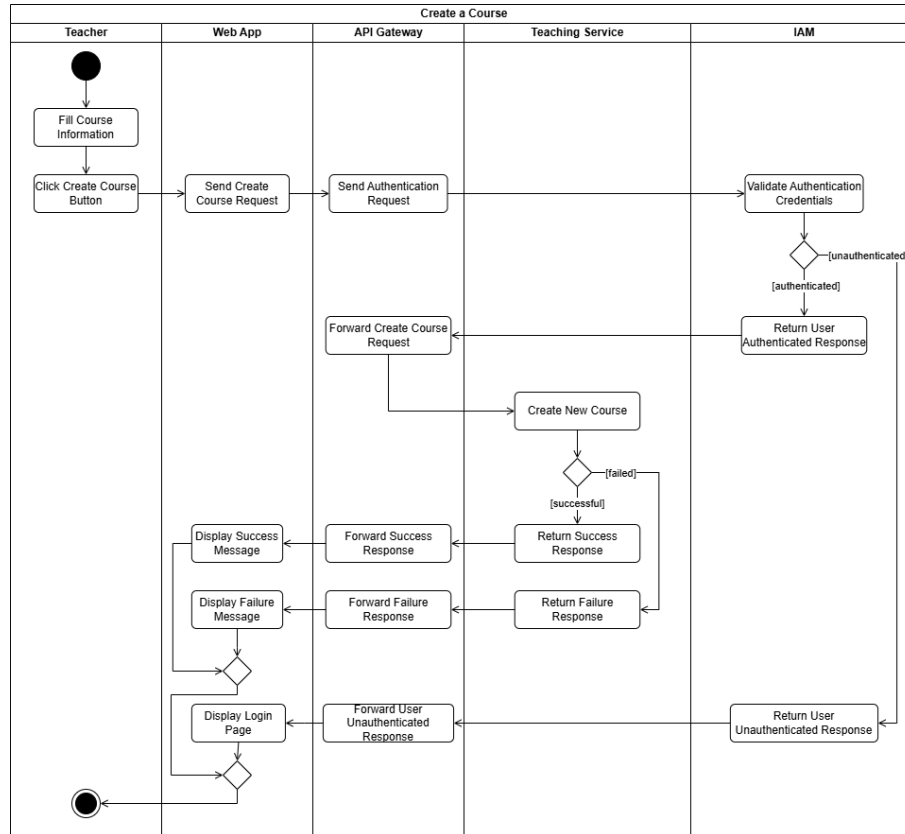


Figure 4.11: Sơ đồ hoạt động use case Create a Course

(Tham khảo Use Case: [Create a Course](#))

- Giảng viên điền thông tin khóa học và gửi yêu cầu tạo khóa học mới với thông tin môn học và mô tả.
- API Gateway kiểm tra xác thực và quyền truy cập thông qua IAM.
- Teaching Service:
 - Kiểm tra thông tin đầu vào,
 - Tạo khóa học trong cơ sở dữ liệu,
 - Liên kết khóa học với giảng viên phụ trách.
- Hệ thống trả kết quả thành công hoặc thông báo lỗi để hiển thị cho giảng viên.

7. Tạo bài kiểm tra mới

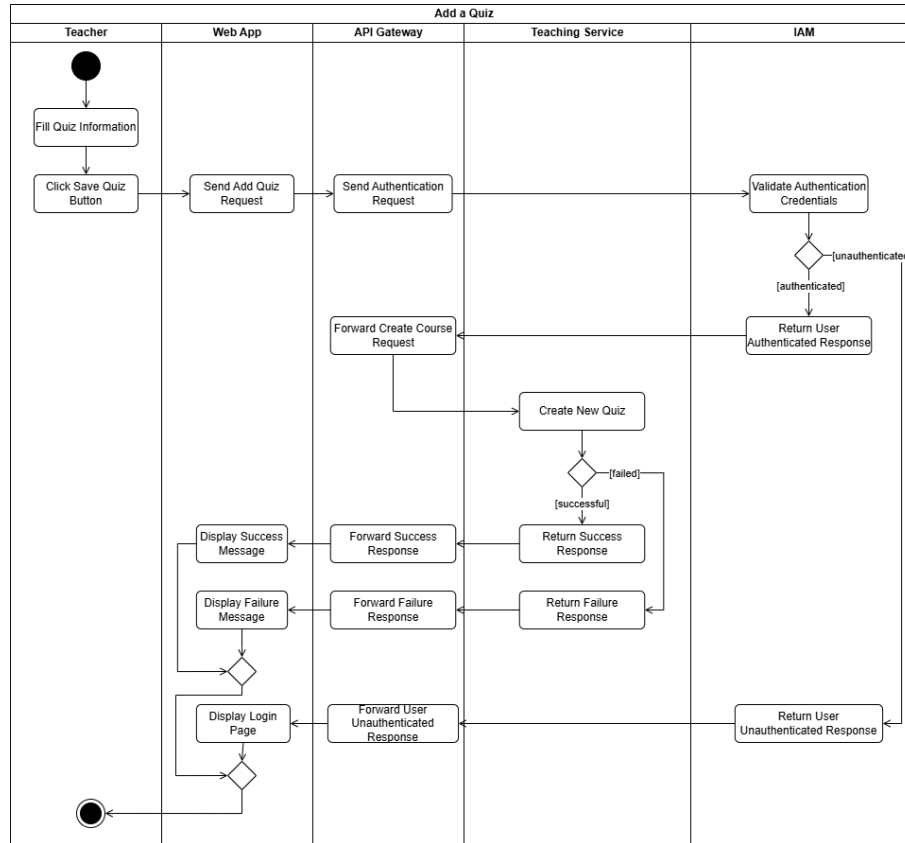


Figure 4.12: Sơ đồ hoạt động use case Add a Quiz

(Tham khảo Use Case: [Add a Quiz](#))

- Giảng viên nhập thông tin bài kiểm tra và gửi yêu cầu tạo mới thông qua Web App.
- Web App gửi yêu cầu đến API Gateway để thực hiện bước xác thực và định tuyến.
- API Gateway gửi yêu cầu xác thực đến hệ thống IAM:
 - Nếu xác thực thất bại, hệ thống trả phản hồi không hợp lệ và Web App chuyển giảng viên đến trang đăng nhập.
 - Nếu xác thực thành công, yêu cầu được chuyển tiếp đến Teaching Service.
- Teaching Service tiếp nhận yêu cầu, kiểm tra dữ liệu và tiến hành tạo bài kiểm tra trong cơ sở dữ liệu:



- Nếu lưu dữ liệu thất bại, hệ thống gửi phản hồi lỗi để Web App hiển thị thông báo tương ứng.
- Nếu tạo thành công, hệ thống trả phản hồi thành công và Web App hiển thị thông báo bài kiểm tra đã được tạo.

8. Tạo bài thi mới

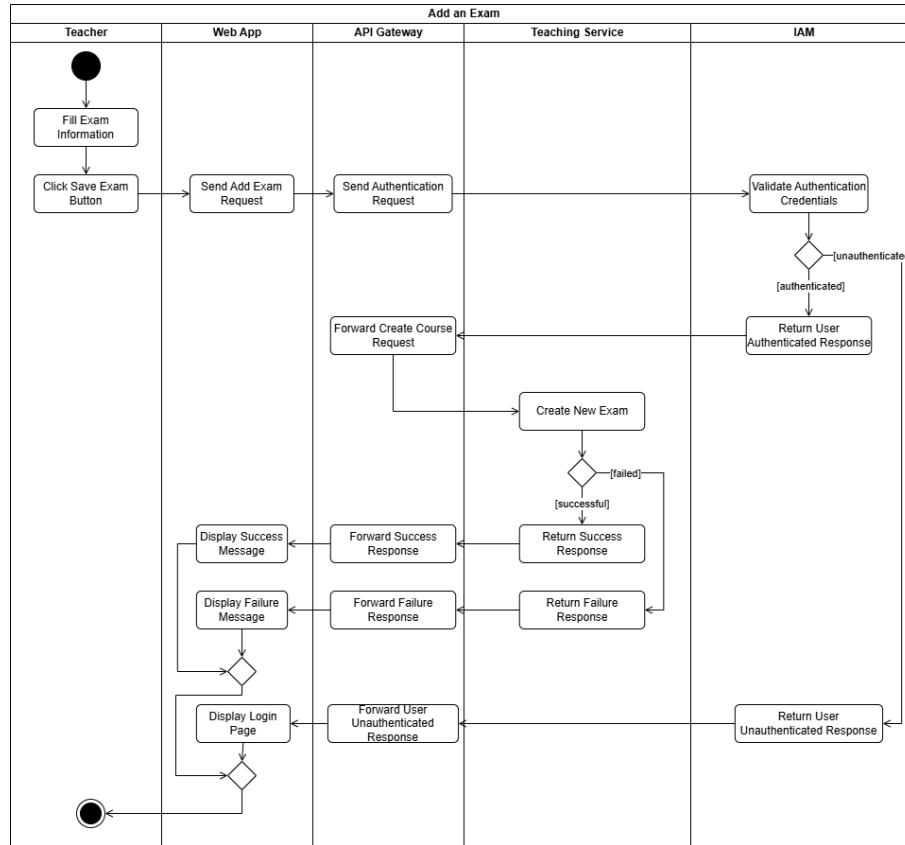


Figure 4.13: Sơ đồ hoạt động use case Add an Exam

(Tham khảo Use Case: [Add an Exam](#))

- Giảng viên điền thông tin bài thi và gửi yêu cầu thêm bài thi với cấu trúc gồm câu hỏi, lựa chọn và mức độ.
- API Gateway xác thực token với IAM trước khi chuyển yêu cầu đến Teaching Service.
- Teaching Service:
 - Kiểm tra tính hợp lệ của đề thi,
 - Lưu đề thi vào cơ sở dữ liệu,
 - Phản hồi trạng thái tạo mới thành công hoặc lỗi.
- API Gateway gửi kết quả về frontend để hiển thị cho giảng viên.

9. Tạo bài tập mới

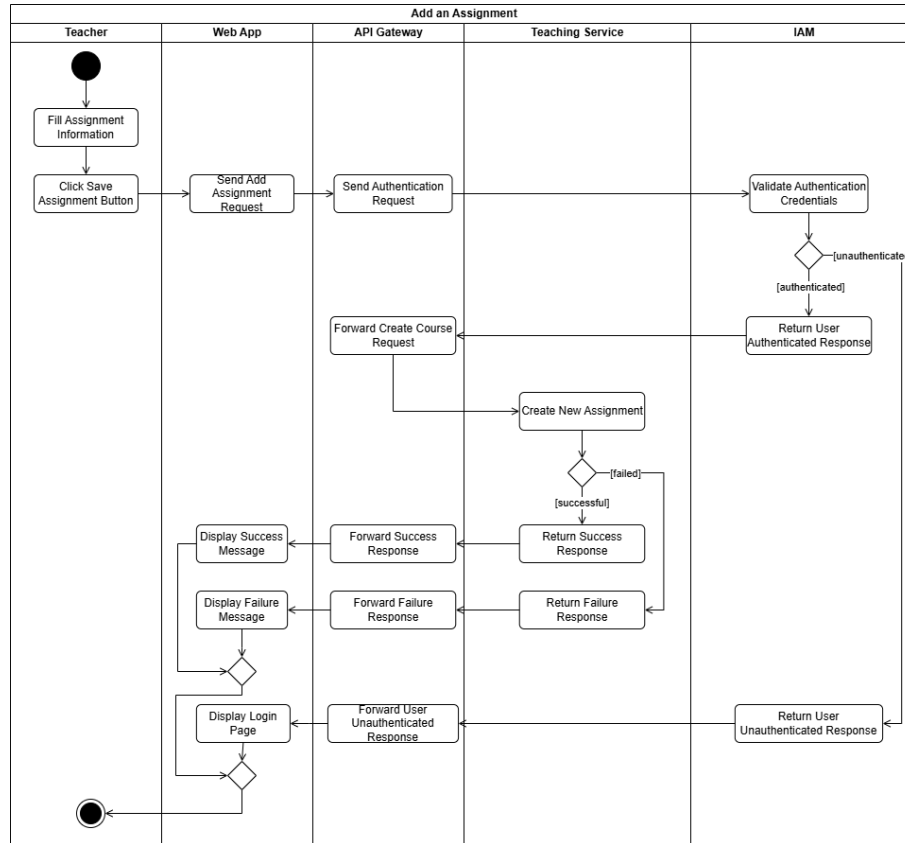


Figure 4.14: Sơ đồ hoạt động use case Add an Assignment

(Tham khảo Use Case: [Add an Assignment](#))

- Giảng viên nhập thông tin bài tập và gửi yêu cầu tạo mới thông qua Web App.
- Web App chuyển yêu cầu đến API Gateway để xác thực và định tuyến đến Teaching Service.
- API Gateway gửi yêu cầu xác thực tới IAM:
 - Nếu không được xác thực, hệ thống trả phản hồi không hợp lệ và Web App chuyển hướng giảng viên đến trang đăng nhập.
 - Nếu xác thực hợp lệ, yêu cầu được chuyển tiếp đến Teaching Service.
- Teaching Service xử lý dữ liệu đầu vào và tạo bài tập mới trong cơ sở dữ liệu:
 - Tạo thất bại: hệ thống trả phản hồi lỗi để Web App hiển thị thông báo tương ứng.



- Tạo thành công: hệ thống trả phản hồi thành công để Web App hiển thị thông báo tạo bài tập mới thành công.

10. Chấm điểm bài tập

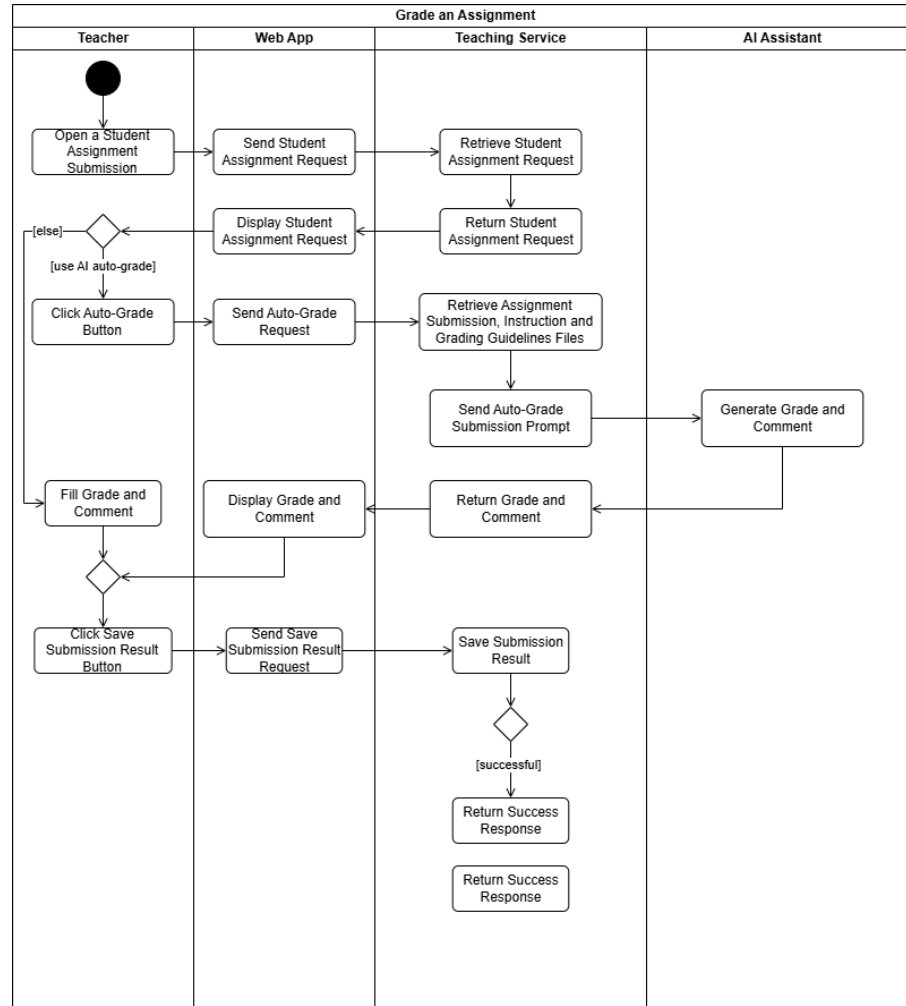


Figure 4.15: Sơ đồ hoạt động use case Grade an Assignment

(Tham khảo Use Case: [Grade an Assignment](#))

- Giảng viên mở bài nộp của sinh viên từ giao diện Web App.
- Web App gửi yêu cầu truy xuất bài nộp đến Teaching Service thông qua API Gateway.
- Teaching Service lấy dữ liệu bài nộp và trả về cho Web App để hiển thị cho giảng viên.
- Giảng viên quyết định chấm thủ công hoặc sử dụng chức năng chấm tự động bằng AI:



- Nếu chọn chấm tự động, Web App gửi yêu cầu đến Teaching Service, sau đó Teaching Service gửi dữ liệu bài nộp, hướng dẫn và tiêu chí chấm đến AI Assistant.
- AI Assistant sinh ra điểm số và nhận xét, gửi lại Teaching Service, rồi phản hồi cho Web App.
- Giảng viên xem điểm và nhận xét, có thể chỉnh sửa nếu cần.
- Khi giảng viên nhấn nút lưu kết quả, Web App gửi yêu cầu lưu điểm và nhận xét đến Teaching Service.
- Teaching Service lưu kết quả vào cơ sở dữ liệu và cập nhật trạng thái bài nộp thành “Đã chấm điểm”.
- Hệ thống gửi phản hồi thành công để Web App hiển thị thông báo cho giảng viên.

5 Hiện Thực Hệ Thống

5.1 Mã Nguồn

Toàn bộ mã nguồn được lưu trữ tại:

<https://github.com/kng1259/SArch-IntelliLearn-251.git>

5.2 Slide Thuyết Trình

Slide thuyết trình về kiến trúc hệ thống được lưu trữ tại:

https://www.canva.com/design/DAG6tBg1Z3k/vR7JhM5J3yW8yocFHNgYLw/view?utm_content=DAG6tBg1Z3k&utm_campaign=designshare&utm_medium=link2&utm_source=uniqueLinks&utm_id=hc8c0287d88

5.3 Các Module Được Hiện Thực

Các module được hiện thực bao gồm:

- Module giao diện:
 - **Teaching Assistance:** Cung cấp giao diện cho giảng viên trong việc quản lý khóa học, học viên, tài liệu giảng dạy và phản hồi học tập.
 - **Learning Management:** Hỗ trợ học viên đăng ký hoặc hủy khóa học, truy cập tài liệu, theo dõi tiến độ và nhận phản hồi.
- Module nghiệp vụ:
 - **Teaching Assistant Module:** Cung cấp chức năng cho giảng viên quản lý khóa học, học viên, tài liệu và phản hồi.
 - **Learning Management Module:** Hỗ trợ học viên đăng ký khóa học, xem tài liệu, theo dõi tiến trình, và nhận phản hồi.

6 Hướng Phát Triển Trong Tương Lai

Nhờ kiến trúc Service-Based được thiết kế theo hướng mô-đun hoá và tuân thủ chặt chẽ các nguyên tắc SOLID, hệ thống **IntelliLearn** cho phép phát triển và mở rộng trong tương lai mà không yêu cầu tái cấu trúc lớn. Một số hướng mở rộng đáng chú ý bao gồm:

1. Mở rộng chức năng phân tích học tập:

- Tăng khả năng xử lý dữ liệu lớn (big data) để phân tích hành vi học tập trên quy mô lớn.
- Tích hợp kho dữ liệu học tập (Learning Data Warehouse) phục vụ cho báo cáo đa chiều.
- Bổ sung cơ chế trực quan hóa dữ liệu theo thời gian thực nhằm hỗ trợ giáo viên và quản trị viên ra quyết định nhanh chóng.

2. Chuyển đổi sang kiến trúc Microservices khi quy mô tăng mạnh

- Tách các module lớn hiện tại gồm **Learning Service** và **Teaching Service** thành các dịch vụ độc lập để tăng khả năng mở rộng và triển khai.

3. Mở rộng chức năng cho giáo viên, học viên và quản trị viên

- Phát triển các tính năng quản lý lớp học, quản lý lịch dạy, tổng hợp đánh giá chất lượng giảng dạy.
- Bổ sung giao diện quản trị giúp theo dõi tình hình học tập tổng quát trong toàn bộ hệ thống.

4. Tăng cường khả năng bảo mật và tuân thủ

- Áp dụng xác thực đa yếu tố (MFA) cho giáo viên và quản trị viên.
- Tăng cường cơ chế mã hóa dữ liệu và tuân thủ các tiêu chuẩn an toàn thông tin quốc tế.

Những hướng mở rộng này có thể được triển khai theo từng giai đoạn tùy theo nhu cầu thực tế. Với nền tảng kiến trúc linh hoạt đã được xây dựng, hệ thống **IntelliLearn** có khả năng tiếp nhận các thay đổi trong tương lai một cách hiệu quả và bền vững.

7 Kết Luận

Báo cáo này đã trình bày quá trình xây dựng kiến trúc phần mềm cho hệ thống IntelliLearn, từ việc phân tích yêu cầu, lựa chọn kiến trúc, thiết kế module cho đến hiện thực hoá. Việc lựa chọn kiến trúc **Service-Based** đã mang lại sự cân bằng giữa tính linh hoạt, khả năng mở rộng và khả năng quản lý, đồng thời cho phép các thành phần trong hệ thống hoạt động độc lập nhưng vẫn gắn kết chặt chẽ.

Việc áp dụng các nguyên tắc **SOLID** trong quá trình thiết kế hệ thống IntelliLearn đã mang lại nhiều tác động tích cực đến cấu trúc tổng thể của hệ thống, đặc biệt trong bối cảnh ITS là một môi trường phức tạp với các miền nghiệp vụ khác nhau và yêu cầu hỗ trợ nhiều nhóm người dùng.

Trước hết, **Single Responsibility Principle (SRP)** đóng vai trò quan trọng trong việc xác định ranh giới trách nhiệm của từng lớp trong mỗi module. Sự tách biệt hợp lý này giúp cải thiện khả năng kiểm thử đơn vị, dễ dàng xác định nguồn gốc lỗi và thuận tiện cho việc mở rộng chức năng.

Tiếp theo, các nguyên tắc **Open/Closed Principle (OCP)** và **Dependency Inversion Principle (DIP)** góp phần tạo nên sự linh hoạt trong thiết kế. Việc sử dụng abstraction thông qua interface cho phép mở rộng hành vi mới mà không cần sửa đổi mã hiện có, chẳng hạn như bổ sung một cơ chế lưu trữ mới hay thêm nghiệp vụ phân tích học tập. Đây là yếu tố quan trọng giúp hệ thống có thể thích ứng với sự thay đổi của yêu cầu trong tương lai mà vẫn đảm bảo tính ổn định của các module cốt lõi.

Interface Segregation Principle (ISP) cũng đóng vai trò không nhỏ trong việc đảm bảo cấu trúc module trở nên tinh gọn và dễ duy trì. Việc chia nhỏ các interface theo đúng phạm vi trách nhiệm của chúng giúp giảm phụ thuộc không cần thiết và hỗ trợ các lớp triển khai trở nên dễ hiểu, dễ mở rộng và giảm thiểu rủi ro lỗi phát sinh.

Bên cạnh đó, **Liskov Substitution Principle (LSP)** giúp đảm bảo rằng mọi lớp hiện thực của các interface như Repository hoặc Service có thể thay thế cho nhau mà không phá vỡ tính đúng đắn của hệ thống. Việc duy trì sự nhất quán về contract giữa các lớp đã giúp quá trình thay đổi hoặc mở rộng hệ thống diễn ra an toàn và ít rủi ro hơn.

Ngoài những lợi ích rõ ràng, việc áp dụng SOLID cũng đưa ra một số thách thức. Một trong những khó khăn lớn nhất là xác định mức trừu tượng phù hợp cho từng module: abstraction quá rộng dẫn đến dư thừa và khó duy trì, trong khi abstraction quá hẹp lại không hỗ trợ mở rộng hiệu quả. Ngoài ra, việc áp dụng SOLID đôi khi đòi hỏi tách lớp quá nhiều, gây cảm giác phức tạp hoá cấu trúc ban đầu. Tuy nhiên, thông qua quá trình tinh chỉnh và xem xét lại thiết kế, những thách thức này trở thành cơ hội giúp hoàn thiện hơn kỹ năng tổ chức lớp và phân tích nghiệp vụ.



Nhìn chung, quá trình hiện thực hệ thống IntelliLearn đã phản ánh rõ lợi ích của việc kết hợp tư duy kiến trúc phần mềm với thực tiễn phát triển hiện đại. Kiến trúc Service-Based kết hợp việc áp dụng SOLID không chỉ giúp hệ thống vận hành ổn định mà còn tạo nền tảng vững chắc cho việc mở rộng trong tương lai, chẳng hạn như tích hợp thêm các mô-đun phân tích học tập nâng cao, hoặc mở rộng quy mô người dùng. Đây là minh chứng cho việc một kiến trúc được thiết kế tốt sẽ hỗ trợ quá trình phát triển dài hạn và đảm bảo giá trị bền vững cho toàn bộ hệ thống.



8 Tài Liệu Tham Khảo

9 Phụ Lục

9.1 Hồ Sơ Quyết Định Kiến Trúc

ADR-01: Lựa Chọn Kiến Trúc Service-Based

1. **Vấn đề:** Vấn đề đặt ra là lựa chọn một kiến trúc phù hợp cho hệ thống Gia sư Thông minh IntelliLearn. Hệ thống phải hỗ trợ học viên và giảng viên truy cập nội dung học tập, quản lý khóa học và theo dõi tiến độ. Kiến trúc cần đảm bảo khả năng mở rộng theo nhu cầu sử dụng thực tế (số lượng người học có thể tăng), khả năng bảo trì dài hạn, và không gây phức tạp hoá quá mức hạ tầng hoặc quy trình triển khai. Do đó, một quyết định kiến trúc phải được đưa ra ngay giai đoạn thiết kế ban đầu để giảm rủi ro tái cấu trúc về sau.
2. **Quyết định:** Chọn kiến trúc **Service-Based** cho IntelliLearn. Kiến trúc này tổ chức hệ thống thành các dịch vụ theo miền nghiệp vụ, mang lại sự phân tách hợp lý mà không làm tăng độ phức tạp vận hành. Trong bối cảnh của IntelliLearn, kiến trúc được áp dụng theo các định hướng chính sau:
 - **Tổ chức theo miền nghiệp vụ:** Hệ thống được chia thành các dịch vụ có ranh giới rõ ràng và bao phủ một nhóm chức năng tương đối rộng gồm:
 - Learning Service: quản lý học liệu, tiến độ học và tương tác của học viên.
 - Teaching Service: quản lý giảng dạy, bài tập, chấm điểm và hoạt động của giảng viên.
 - **Giao tiếp thông qua API Gateway:** API Gateway đóng vai trò điểm truy cập duy nhất của hệ thống, giúp điều phối yêu cầu, chuẩn hoá giao tiếp và tạo nền tảng cho khả năng mở rộng.
 - **Chia sẻ tài nguyên lưu trữ chung:** Các dịch vụ cùng sử dụng:
 - PostgreSQL cho dữ liệu có cấu trúc.
 - MinIO cho lưu trữ tệp tin và học liệu.
 - **Định hướng phát triển dài hạn:** Kiến trúc Service-Based có thể phát triển lên Microservices trong tương lai nếu nhu cầu mở rộng và tách biệt dữ liệu tăng lên, giúp giảm thiểu tình trạng tái cấu trúc về sau.

3. **Trạng thái: ĐÃ PHÊ DUYỆT**

4. **Giả định:** Việc lựa chọn kiến trúc dựa trên các giả định:

- Quy mô hệ thống ở giai đoạn đầu ở mức trung bình, số lượng người dùng vừa phải nhưng có khả năng tăng trong tương lai.
- Hạ tầng triển khai dự kiến chạy trên môi trường container hoặc máy chủ ảo hóa, cho phép mở rộng theo dịch vụ.
- Hệ thống chưa yêu cầu tách cơ sở dữ liệu hoàn toàn cho từng dịch vụ (nhu cầu phân tán dữ liệu chưa cao).
- Yêu cầu vận hành cần giữ ở mức đơn giản, chi phí hợp lý và không phát sinh overhead trong quản lý hạ tầng.

5. **Các lựa chọn thay thế:** Các lựa chọn thay thế hợp lệ đã được xem xét gồm:

(a) **Kiến trúc Layered:** Đây là mô hình truyền thống, tổ chức hệ thống theo các tầng chức năng như giao diện, xử lý nghiệp vụ và truy cập dữ liệu.

- Ưu điểm:
 - Cấu trúc đơn giản, dễ triển khai và dễ đào tạo đội ngũ phát triển.
 - Quy trình phát triển tuyến tính, phù hợp với ứng dụng có phạm vi chức năng nhỏ.
- Hạn chế:
 - Các tầng phụ thuộc chặt, dẫn tới chi phí thay đổi cao khi hệ thống mở rộng.
 - Khó mở rộng một phần hệ thống mà không ảnh hưởng đến toàn bộ ứng dụng.

(b) **Kiến trúc Microservices:** Mô hình phân tán chia hệ thống thành các dịch vụ nhỏ, độc lập hoàn toàn.

- Ưu điểm:
 - Mỗi dịch vụ có thể phát triển, triển khai và mở rộng một cách độc lập.
 - Khả năng tách biệt dữ liệu mạnh, phù hợp với hệ thống quy mô lớn và thay đổi nhanh.

- Hạn chế:
 - Yêu cầu hạ tầng và DevOps phức tạp (monitoring, discovery, observable systems...).
 - Chi phí vận hành cao, đặc biệt trong giai đoạn đầu của dự án.
 - Đòi hỏi đội ngũ giàu kinh nghiệm và quy trình triển khai phức tạp.

6. **Lập luận:** Việc chọn kiến trúc Service-Based mang lại cân bằng tối ưu giữa chất lượng kỹ thuật và khả năng vận hành cho IntelliLearn:

- **Đáp ứng nhu cầu mở rộng theo dịch vụ:** Learning Service và Teaching Service có thể mở rộng độc lập dựa trên số lượng người học hoặc khối lượng nội dung giảng dạy. Điều này phù hợp với mô hình tăng trưởng dự kiến.
- **Giảm phụ thuộc so với Layered Architecture:** Mỗi dịch vụ quản lý một tập nghiệp vụ rõ ràng, giảm phụ thuộc và cải thiện khả năng bảo trì.
- **Không quá phức tạp như Microservices:** Tránh chi phí vận hành của các nghiệp vụ phức tạp vốn không cần thiết cho hệ thống quy mô trung bình.
- **Đánh giá giải pháp:**
 - **Khả năng bảo trì:** Dự kiến thấp hơn 20–30% chi phí bảo trì so với Layered vì phạm vi dịch vụ nhỏ và rõ ràng.
 - **Khả năng mở rộng:** Có thể mở rộng theo chiều ngang từng dịch vụ tùy theo tải, thay vì mở rộng toàn hệ thống.
 - **Độ phức tạp vận hành:** Thấp hơn Microservices khoảng 50–60% do không yêu cầu hạ tầng phức tạp.
 - **Độ tin cậy:** Do các dịch vụ có kích thước lớn nên các giao tiếp qua mạng giảm đáng kể. Điều này giảm thiểu nguy cơ lỗi mạng, độ trễ, và tránh các giao dịch phân tán phức tạp. Bên cạnh đó, khi một dịch vụ gặp sự cố, phạm vi ảnh hưởng được cô lập, giúp hệ thống tổng thể có thể duy trì vận hành ổn định.

7. **Các quyết định liên quan:**

- **Hệ quả:** [ADR-03](#) - Lựa Chọn Sử Dụng API Gateway.

ADR-02: Tách Biệt Dịch Vụ IAM và Cơ Sở Dữ Liệu

1. **Vấn đề:** Hệ thống IntelliLearn quản lý đồng thời hai loại dữ liệu có tính chất khác nhau: (1) dữ liệu định danh và xác thực người dùng (tài khoản, vai trò, thông tin bảo mật), (2) dữ liệu học vụ thuộc về các miền nghiệp vụ như khóa học, tiến trình học tập và bài đánh giá.

Việc giữ chung hai loại dữ liệu này trong một cơ sở dữ liệu hoặc để các dịch vụ nghiệp vụ tự xử lý xác thực sẽ làm tăng rủi ro an ninh, phức tạp hóa bảo trì và khó đáp ứng yêu cầu tuân thủ bảo mật.

Vì vậy, cần đưa ra quyết định về việc có tách biệt một dịch vụ **IAM** độc lập cùng với một cơ sở dữ liệu riêng để quản lý danh tính và quyền truy cập hay không.

2. **Quyết định:** Triển khai một dịch vụ **IAM (Keycloak)** độc lập cùng với cơ sở dữ liệu riêng, tách biệt hoàn toàn khỏi cơ sở dữ liệu nghiệp vụ. IAM chịu trách nhiệm toàn bộ việc xác thực, quản lý người dùng, phân quyền và tạo token OIDC/OAuth2, trong khi các dịch vụ nghiệp vụ chỉ xử lý logic nghiệp vụ và xác minh token qua API Gateway.

3. **Trạng thái:** **ĐÃ PHÊ DUYỆT**

4. **Giả định:**

- Hệ thống hướng đến số lượng người dùng lớn với nhiều nhóm vai trò (học viên, giảng viên, quản trị viên).
- IAM (Keycloak) cung cấp đầy đủ cơ chế xác thực, phân quyền, quản lý phiên hoạt động và tạo token tiêu chuẩn.
- Hệ thống dự kiến mở rộng trong tương lai, bao gồm khả năng tích hợp với dịch vụ bên ngoài (SSO, nhà cung cấp OAuth2).
- Cơ sở dữ liệu IAM có cấu trúc và yêu cầu bảo mật riêng, tách biệt với dữ liệu khóa học và nội dung học tập.

5. **Các lựa chọn thay thế:** Các lựa chọn thay thế đã được xem xét:

- (a) **Tự xây IAM trong mỗi dịch vụ backend:**

- Ưu điểm: toàn quyền tùy chỉnh; không cần thêm dịch vụ ngoài.

- Nhược điểm: chi phí phát triển và duy trì cực kỳ cao; nhân bản logic xác thực ở nhiều dịch vụ; không đáp ứng tiêu chuẩn bảo mật; khó triển khai SSO hoặc xác thực token thống nhất.

6. **Lập luận:** Việc tách biệt dịch vụ IAM và cơ sở dữ liệu được chọn vì:

- **Tăng cường bảo mật:**
 - Dữ liệu định danh được cô lập khỏi dữ liệu nghiệp vụ.
 - Các dịch vụ nghiệp vụ không thể truy cập trực tiếp vào thông tin nhạy cảm (mật khẩu, token, quyền người dùng).
- **Đơn giản hóa backend nghiệp vụ:**
 - Backend chỉ cần xác minh token thay vì tự triển khai logic xác thực.
 - Giảm mã trùng lặp và rủi ro bảo trì giữa Learning Service và Teaching Service.
- **Khả năng mở rộng và tích hợp trong tương lai:**
 - Hỗ trợ sẵn tích hợp với các nhà cung cấp SSO, LDAP, OAuth2.
 - IAM có thể mở rộng độc lập theo tải đăng nhập và xác thực.
- **Tối ưu khả năng giám sát và vận hành:** IAM tập trung vào xác thực, audit và log bảo mật; backend tập trung vào logic nghiệp vụ.

7. **Các quyết định liên quan:**

- **Tiền đề:** [ADR-01](#) - Lựa chọn kiến trúc Service-Based.
- **Bổ trợ:** [ADR-03](#) - Lựa Chọn Sử Dụng API Gateway.

ADR-03: Lựa Chọn Sử Dụng API Gateway

1. **Vấn đề:** Hệ thống IntelliLearn bao gồm nhiều dịch vụ backend độc lập (Learning Service, Teaching Service), mỗi dịch vụ phụ trách một miền nghiệp vụ riêng. Frontend (Next.js) cần giao tiếp với các dịch vụ này thông qua một cơ chế thống nhất, đảm bảo bảo mật, định tuyến đúng miền nghiệp vụ và che giấu cấu trúc nội bộ của hệ thống. Nếu không có một điểm truy cập tập trung, việc tích hợp sẽ phức tạp, khó mở rộng và gây rủi ro về an ninh, do mỗi dịch vụ phải tự xử lý xác thực, phân quyền và quản lý lưu lượng. Do đó, cần phải đưa ra quyết định về việc có áp dụng API Gateway hay không và chọn cơ chế triển khai phù hợp.
2. **Quyết định:** Sử dụng **API Gateway** làm điểm truy cập tập trung cho toàn bộ yêu cầu từ Frontend. API Gateway chịu trách nhiệm định tuyến, xác thực token (qua IAM), giới hạn truy cập, lọc yêu cầu và chuyển tiếp yêu cầu đến các dịch vụ backend tương ứng.
3. **Trạng thái:** **ĐÃ PHÊ DUYỆT**
4. **Giải định:**
 - Hệ thống có cấu trúc Service-Based, với 2 dịch vụ chính: Learning Service, Teaching Service.
 - Số lượng người dùng có thể tăng dần theo thời gian (học viên, giảng viên), cần cơ chế mở rộng theo nhu cầu.
 - Frontend (Next.js) sử dụng REST API để giao tiếp với backend.
 - Yêu cầu bảo mật cao: mọi request đều phải xác thực bằng token OIDC/OAuth2 từ IAM (Keycloak).
 - Hệ thống triển khai trên môi trường container.
 - Các dịch vụ backend không nên được truy cập trực tiếp từ Internet nhằm giảm thiểu bề mặt tấn công và hạn chế nguy cơ xâm nhập trái phép vào hệ thống.
5. **Các lựa chọn thay thế:** Các lựa chọn thay thế đã được xem xét:
 - (a) **Kết nối trực tiếp Frontend với từng dịch vụ backend:** Frontend gọi trực tiếp nhiều endpoint khác nhau mà không qua cổng tập trung.

- Ưu điểm: đơn giản, không cần thêm tầng trung gian.
- Nhược điểm: phức tạp khi mở rộng; Frontend phải biết toàn bộ cấu trúc backend; khó kiểm soát bảo mật; khó áp dụng rate limiting; dễ rò rỉ endpoint nội bộ.

(b) **Backend for Frontend (BFF)**: Tạo một service riêng dành cho frontend, thay Gateway làm lớp điều phối.

- Ưu điểm: tùy biến API theo từng loại client (web/mobile).
- Nhược điểm: không phù hợp vì hệ thống chỉ có một loại client chính (Web App); chi phí vận hành cao hơn API Gateway; tận dụng kém hơn các cơ chế lọc yêu cầu và định tuyến.

6. **Lập luận**: Lý do lựa chọn sử dụng API Gateway:

- **Ổn định và dễ triển khai**: API Gateway như Nginx phù hợp với quy mô kiến trúc Service-Based hiện tại, không yêu cầu hạ tầng phức tạp.
- **Tăng cường bảo mật**:
 - Mọi yêu cầu đều được kiểm tra token OIDC qua IAM (Keycloak).
 - Che giấu toàn bộ endpoint nội bộ khỏi Internet, giảm bề mặt tấn công.
- **Đơn giản hoá tích hợp Frontend–Backend**: Frontend chỉ cần gọi duy nhất một endpoint Gateway, không cần biết kiến trúc backend.
- **Tăng khả năng mở rộng**: Dễ dàng thêm dịch vụ mới vào Gateway mà không cần sửa Frontend.
- **Tối ưu vận hành**: API Gateway cho phép thiết lập load balancing, rate limiting và giám sát một cách tập trung.

7. **Các quyết định liên quan**:

- **Tiền đề**: [ADR-01](#) - Lựa chọn kiến trúc Service-Based.
- **Bổ trợ**: [ADR-02](#) - Tách Biệt Dịch Vụ IAM và Cơ Sở Dữ Liệu.